
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 22:44:30
Файлов исходного кода: 111
Суммарный объём кода: 1.98 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot: ')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends \
            # На Ubuntu политика ImageMagick иногда запрещает
            # разрешаем то, что нужно пайплайну (text -> pdf)
            for policy in /etc/ImageMagick-6/policy.xml /etc/ImageMagick-6/policy.xml
            if [ -f "$policy" ]; then
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-страниц"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height" />
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
    <title> Дискретка для тупых – Интерактивный гид по дискретке
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, height=device-height" />
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%/public/favicon.svg" />
     <title> Дискретка для тупых – Интерактивный гид по дискретке
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
},

```



```

 className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
 activeTab === 'compiler'
 ? 'bg-emerald-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 }}
 >
 <Terminal className="w-4 h-4" /> Python
</button>
<a
 id="download-pdf-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
 title="Скачать полный PDF сборник всех стра
 >
 <FileDown className="w-4 h-4" /> Скачать PD

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center

```



```
export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }, [code, stdin]);
```

```

 <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запускаю..."}
 {running ? "Запускаю..." : "Запустить"}
 </button>
</div>
</div>

<textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
 placeholder="Напишите Python-код..."

```

```

 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-indigo-600" >
 Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-amber-600" >
 Первый запуск скачивает примерно несколько килобайт кода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4" >
 <Info className="w-4 h-4 shrink-0 text-emerald-600" >
 Код выполняется в браузере и не отправляется на сервер.

 </div>
 </div>
</div>
</main>
);
}
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    return "Ваш код выполнен успешно!"
+
+main()
+
+`;
```



```

+
+  const runCode = useCallback(async () => {
+    if (running) return;
+
+    setRunning(true);
+    setOutput("");
+    setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+    const stdout: string[] = [];
+    const stderr: string[] = [];
+    const inputLines = stdin.split(/\r?\n/);
+
+    try {
+      const runtime = await getPythonRuntime();
+      setRuntimeReady(true);
+      setRuntimeMessage("Python готов – код выполняется");
+
+      runtime.setStdout({ batched: (message) => stdout
+      runtime.setStderr({ batched: (message) => stderr
+      runtime.setStdin({ stdin: () => (inputLines.length
+      await runtime.runPythonAsync(code);
+
+      const text = [...stdout, ...stderr].join("");
+      setOutput(text || "Готово: программа ничего не выв
+    } catch (error) {
+      const captured = [...stdout, ...stderr].join("")
+      setOutput(`${captured}${captured && !captured.en
+      setRuntimeMessage("Не удалось выполнить код – пр
+    } finally {
+      setRunning(false);
+    }
+  }, [code, stdin, running, runtimeReady]);
+
+  const reset = () => {
+    setCode(STARTER_CODE);
+    setStdin("");
+    setOutput("Нажмите «Запустить», чтобы увидеть резу.
+  };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```

+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...

```

`vite.config.ts`

Полное содержимое после изменений

```

````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},

```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживаешься готовых шаблонов разметки, описанных в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-шаблоны, описанные в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в файле ``.ai_skills/universal_guide_authoring/SKILL.md`` для вставки в ``.src/data/content.json`` или ``.src/data/content.ts``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-blue-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 },
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не зациклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

| Глава                                      | id                    | Комментарий             |
|--------------------------------------------|-----------------------|-------------------------|
| 15. Планарность: K5, K3,3 и формула Эйлера | `planarity-euler`     | т «на пальцах».         |
| 16. Флойд (Флойд–Уоршелл)                  | `floyd`               | Есть матричный алгоритм |
| 17. Алгоритм Джонсона                      | `johnson-algo`        | Перевзвешивание         |
| Обзор: Кратчайшие пути и Графы             | `intro`               | Вводная страница        |
| Эйлер: Путь ≠ Цикл                         | `euler-path-vs-cycle` | Ключевая концепция      |

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сухари»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока не реализована

| Глава                                | id                   | Чего не хватает              |
|--------------------------------------|----------------------|------------------------------|
| 3. Декартово дерево (Treap)          | `treap`              | Нет ни визуализации, ни кода |
| 8. Графы. Компоненты связности       | `graph-components`   | Нет кода                     |
| 24. Классы сложности, сведение задач | `complexity-classes` | Нет кода                     |

## Сводная таблица по всем главам

| #   | Глава | Аналогий | 2D-виз. | SVG | Картинка |
|-----|-------|----------|---------|-----|----------|
| --- | ---   | ---      | ---     | --- | ---      |



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []));
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.in
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[i

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 {/* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}`}
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 {/* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className=
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсальн
 </div>
 <h2 className="text-base sm:text-xl fon
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
)}
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/g, "\\$&");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы — чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок накрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрыва
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{k_x} \times 2^{k_y}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block; width: 100px; height: 100px; border-radius: 5px; background-color: #f0f0f0; text-align: center; vertical-align: middle;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4">ум на его части не вычислишь. Приходится хранить матрицу. log N × log M</p>  
xt-emerald-300">log N × log M</span>. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лицу**</p>  
лицу</b> – перекрывающиеся блоки посчитают  
</b> – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text">

<li><b>Сумма через разреженную таблицу префиксные (билет 5).</b></li>

<li><b>Память.</b> N = M = 1000 даём  
считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> В st[...]  
st[...]</span>  
</li>

<li><b>Выход за границу в запросе.</b> 1</span> и 1</span>

<li><b>Нужны обновления.</b> Разреженная матрица (билет 1), а не она.</li>

</ul>

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму по всем уровням, а запросов очень много.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму по всем уровням, а запросов очень много.

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text-sm">
 </div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">С
 х». Спуск от корня: узел, который вместе с
 скрытия – досматривается только одна ветвь
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">М
 аг – одно сравнение: чей у у корня больше,
 сохраняет порядок х). Итого $O(h)$ сравнений
 </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```

 <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
 Код: универсальная вставка, merge
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-xs text-amber-300"><
 </p>
 <pre class="bg-slate-950 p-4 rounded">
def insert(root, node):
 if not root: return node
 L, R = split(root, node.key)
 return merge(merge(L, node), R)</pre>
 <p class="text-xs text-slate-400"><
 и элементы подаются на вход, будучи заранее
 <p>
 <pre class="bg-slate-950 p-4 rounded">
def merge(a, b):
 # все x в a < все y в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key):
 # удалить произвольный элемент
 L, R = split(root, key)
 L_left, L_target = split(L, key - 1)
 return merge(L_left, R)</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Самонастраивающееся дерево: поднимаем",
 category: "Продвинутые структуры",
 },

```

```
<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
```

Аналогия 1: Стопка бумаг

```
</div>
 <p class="text-slate-300 text-sm">Сначала не ведёшь тировку, но через неделю самое нулевое. </p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #800080; border-radius: 10px; box-shadow: 5px 5px 0px #800080;">
 <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #800080; border-radius: 10px; box-shadow: 5px 5px 0px #800080;">
```

Аналогия 2: Тропинка через газон

```
</div>
 <p class="text-slate-300 text-sm">Иногда приходится прокладывать тропинку вдвое. Первый проход дорогой, зато оплачивается. </p>
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">Повороты
<p class="text-slate-300 text-sm mb-3">Повороты бывают:
 Среднее поддерево β (между x и p) – единственное, которое не имеет поворотов. </p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto font-family: monospace; font-size: 0.9em;">
 x p
 / \ / \
x y x y
 / \ / \
a b a b
<----->
 p выше
<----->
 x выше
</pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обход дерева поиска поворотами невозможно в момент «отстёжки», а врезка «Механика одного поворота».
```

```
<h4 class="text-lg font-bold text-white mt-3">Смотреть
<p class="text-slate-300 text-sm mb-3">Смотреть
<div class="overflow-x-auto -mx-2 px-2">
 <table class="w-full text-xs sm:text-sm">
```

-----  
Zig-Zig (p, потом x):  $3 \times \text{Zig-Zig} + \text{Zig} \rightarrow$  высота 8 →  
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<

```
<p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 ro
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4">с
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
lay последнего на пути); insert – BS
сумма левой половины не будет правого сына –
а под «горячие» ключи, дешёвые split/merge.
```

```

 rotate_up(x)
 return x

def find(root, key):
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None

```

Одна строка – и вся амортизация.

```

 </table>
 <p class="text-slate-400 text-xs mt-2">
ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4
rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
о накопилось от угла (0,0) до меня». Хочешь
у и верни дважды отрезанный угол. Вычитание
адывать, но и убирать обратно. Поэтому хран
 <div class="rounded-lg bg-slate-950
 <div><span class="text-slate-500

```

Тонкая: одна матрица

```

</div>
<p class="text-slate-300 text-sm mb-1">
одной матрицы (плюс рамка из нулей). Никаких
прямоугольника от угла. Всё остальное полу
</div>
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычес
 </div>
 <p class="text-slate-300 text-sm mb-1">
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data-
платит она за это памятью <span class="font-
 </div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 <tr class="border-b border-slate-700">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 <td class="py-2 pr-3 font-normal">
 </tr>
 </tbody>
 </table>

```

```

 Код: 1D (тот же, что выполняет панель
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный массив
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">Панель
- часть демонстрации: лента подсвечивает, из
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
- b border-slate-700/50">
 Код: 2D – построение и запрос прямоугольника
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3]]
n, m = 4, 4

S = [] # рамка из нулей
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] - S[i-1][j-1]

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]
print("итого:", ans)</code></pre>
 <p class="text-slate-300 text-sm">Панель
вигают 2D-вкладку демонстрации: при построении
при запросе подсвечиваются четыре угла формы

```

```

 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска графов</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">
 1. Если граф не планарный, то его нельзя раскрасить 4 цветами.
 2. Если граф планарный, то его можно раскрасить 4 цветами.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Границы не пересекаются в одной точке, поэтому
 можно раскрасить карты 4 цветами.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения."
 }

```



```

{
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju-Стерна",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 SCC
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Макро-вершины — это компоненты сильной связности (SCC) графа.
 <p class="text-xl font-mono text-white">Они представляют собой максимальные подграфы, в которых каждая пара вершин взаимно достижима.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе город, где все улицы имеют одностороннее движение. Если вы можете пройти из точки A в точку B, и из B в A, то A и B находятся в одной «макро-вершине» (SCC).
 </div>
 <p class="text-slate-300 text-sm mb-4">В графе с односторонними рёбрами (орграфе) мы можем разбить вершины на группы, где каждая группа — это SCC. Внутри каждой группы можно ходить в любом направлении, но между группами движение строго упорядочено.
 <p class="text-slate-300 text-sm mb-4">Этот процесс называется «разбиением графа на SCC». Он полезен для многих задач, связанных с ориентированными графами, таких как поиск циклов, топологическая сортировка и минимизация времени выполнения задач.
 </div>
 </div>
 </div>
</section>`,
},
{
 id: "graph-bridges",
 title: "11. Графы. Мосты",

```

```

<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
</div>
</div>
</div>
</section>`,
},
{
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4" style="background-color: #1e293b; border-radius: 10px;">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg" style="background-color: #2d3748; border-radius: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">

```

```

 <p class="text-slate-300 text-sm mb-1">
 идишь "АБРА". Если ты ошибешься при поиске
 концовка "АБРА" совпадает с началом "АБРА"
 моя перепроверок!</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

```

```

{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-base sm:text-lg font-monospace">
 P: класс decision-задач, решаемый за полиномиальное время.
 NP: класс decision-задач с сертификатом.
 </p>
 Сведение (Reduction) $A \leq_p B$: задача A сводится к задаче B.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Судоку (P vs NP)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс NP – класс задач, требующих
 енную сетку (ответ), он БЫСТРО (за класс P) решить.
 </div>
 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 2: Задача о рюкзаке (NP vs P)
 </div>
 <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс NP – класс задач, требующих
 енную сетку (ответ), он БЫСТРО (за класс P) решить.
 </div>
 </div>
 </div>
 </div>
`

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded">
 <p class="text-sm text-blue-300">
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </p>
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-900
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная)
 Домой вернуться не смог, потому что
 </p>
 </div>
</div>

<div class="bg-slate-900 p-6 rounded-lg border">

```

```

{
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся — граф р
 </p>
</div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```



```
using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежных
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный вызов
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
 tin.assign(n, -1);
 low.assign(n, -1);
}
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
 Следствие: Для планарно
ode>.
```

```
 Если ребер больше – граф гарантированно
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-
```

```
 <div class="bg-slate-800 p-6 rounded-lg bor-
```

```
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
```

```
 K5: Полный граф на 5 вершинах
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
```

```
 Все 5 вершин соединены со всеми. На
```

```
 Теорема Куратовского: K5 и K3,3 – д
```

```
 Если внутри графа спрятано подра
– плоской раскладки не будет.
```

```
 </p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg bor-
```

```
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
```

```
 K3,3: 3 дома и 3 колодца
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 Три дома хотят соединиться с тремя
```

```
 Нарисовать без пересечений невозможно
```

```
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
 Почему? Потому что у двудольного гр
```

```
 Отсюда более строгое ограничение: <
```

```
 <code class="text-white">9 > 8</code>
```

```
<div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

Аналогия 1: Расписание экзаменов

```
</div>
```

```
<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить расписание
«берём предметы в каком-то порядке и ставим их в слоты»
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
```

```
<div class="absolute -top-3 left-4 border border-fuchsia-500 shadow-md">
```

Аналогия 2: Частоты вышек

```
</div>
```

```
<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left">
```

```
<thead>
```

```
<tr class="text-slate-500 border-b">
```

```
<th class="py-2 pr-3 font-bold">Время
```

```
<th class="py-2 pr-3 font-bold">Предметы
```

```
<th class="py-2 font-bold">Помехи
```

```
</tr>
```

```
</thead>
```

```
<tbody class="text-slate-300">
```

```
<tr class="border-b border-slate-500">
```

```
><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$
а свободен</td></tr>
```

```
<tr class="border-b border-slate-500">
```

```
d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$
</td></tr>
```

```
<tr class="border-b border-slate-500">
```

```
<td class="py-2 pr-3 font-mono"> $\chi = 2 \Rightarrow$ нет
) – единственная «лёгкая» раскраска</td></tr>
```

```

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
 я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободный
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жадн

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет
 dq.append(to)
 elif col[to] == col[v]: # одноцветные
 bipartite = False</pre>

```

еются к точкам сочленения. То есть, **её** - это тупик со степенью 1).

</p>

</div>

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
```

Аналогия: Главный перекресток

</div>

<p class="text-slate-300 text-sm mb-4">

Представь город, где два района соединены перекрестком. Если удалить вершину перекрестка, то система будет **хрупкой**.

</p>

</div>

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
```

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной сети, соединенных в звезду. Если выключить центральный свитч, то сеть перестанет работать (мостом). Сами свитчи — это точки сочленения.

</p>

</div>

</div>

```
<details class="bg-slate-900/40 rounded-lg border border-slate-500">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
```

<div class="p-5 text-sm text-slate-300 space-y-2">

<p class="text-blue-400 font-bold">Особые примечания

</p>

```

 <strong class="text-emerald-400">SCC
 анных графов. Сжатие SCC даёт ориентированный граф
 Внимание: Нельзя применять алгоритмы
 тические структуры для разных типов графов.

</div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 145 | РАЗМЕР: 1.1 КБ

```

import { Chapter } from "../types";
import { graphChapters } from "./graphContent";
import { additionalTickets } from "./additionalTickets";
import { tickets14to20 } from "./tickets/ticket14_20";
import { tickets21to24 } from "./tickets/ticket21_24";
import { tickets6to13 } from "./tickets/ticket6_13";
import { ticket1Segtree } from "./tickets/ticket1_segtree";
import { tickets18to20Mst } from "./tickets/ticket18_20_mst";
import { chapters as newChapters } from "./content_temp";

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает последняя
 * версия главы. `newChapters` (content_temp) перетирала предыдущие
 * поздние переписанные страницы (Эйлер, Мосты, Планарные графы).
 *
 * Страниц в содержании меньше, чем тем: одна страница может содержать
 * несколько билетов сразу (18–20 «Основное дерево: Крайние случаи
 * три алгоритма на одном графе с тремя вкладками в визуализации»).
 */

```

```
// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 2
 */
export const chapterTopics: Record<string, number[]> =
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],
 "graph-components": [8],
 "top-sort": [9],
 "scc-kosaraju": [10],
 "bridges-code": [11],
 "graph-articulation": [12],
 "euler-path-vs-cycle": [13],
```

```
export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler",
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s",
 "dynamic-programming", "johnson-algo", "graph-articulat",
 "complexity-classes", "treap", "splay-tree", "prefix-sum"
];
```

ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 212 | РАЗМ

```
/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

export type DebugValue = null | boolean | number | string;

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("" – верхний уровень) */
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**
 * JSON-снимок доступных имён (globals + locals). В о
 * можно напрямую отдать визуализации: i/j остаются ч
 */
 values?: Record<string, DebugValue>;
}
```



```

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

def __visible(items):
 return {
 k: v for k, v in items
 if not k.startswith("__") and type(v).__name__ != "Traceback"
 }

```

```

 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
Финальные локалы программы (после последней строки кода)
if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 __init_globals = {
 "__name__": "__main__",
 "A": [5, 8, 3, 12, 7, 2, 0, 0],
 "a": [5, 8, 3, 12, 7, 2, 0, 0],
 }
 exec(compile(__src, "<user-code>", "exec"), __init_globals, __init_globals)
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})
print(__OUT)

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРӨК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/MiniI
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

```
 "Волна от источника: сначала все соседи, потом со
```

```

 formal:
 "Полное бинарное дерево в массиве: дети узла i лежат в $2i$ и $2i+1$
complexity: "вставка/извлечение $O(\log n)$, «поиск по ключу» $O(\log n)$ ",
demo: "heap",
simulator: "heap-beam-search",
},
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
 formal: "push / pop / top за $O(1)$. На стеке живёт рекурсия",
 demo: "stack",
 simulator: "stack-dfs",
},
{
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот первым и ест",
 formal: "push_back / pop_front за $O(1)$. Основа BFS и поиска в ширину",
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "CHM", "система непересекающихся множеств"],
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего старосту",
 formal:
 "find со сжатием путей + union по рангу/размеру. Реализация на $O(n)$ ",
 complexity: "почти $O(1)$ — обратная функция Аккермана",
 demo: "dsu",
 simulator: "mst",
},
{
 id: "trie",

```

```

 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набегало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]",
 complexity: "препроцессинг O(n), запрос O(1)",

```

```

 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "т"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения. Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи",
 "амортизированное", "амортизированного", "амортизи",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи",
 "ассив.",

```

```

 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-функ",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начинающихся",
 "с каждой буквы.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i..n]$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "динамическое",
 "мемоизация", "мемоизацию", "подзадач", "подзадачу",
],
 title: "Динамическое программирование",
 short:

```

```

 "На каждом шаге хватаем то, что лучше прямо сейчас",
 formal:
 "Корректность обычно доказывается «леммой о разрезе»",
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь",
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени ≤ 2",
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги",
 formal:
 " $V - E + F = 2$ для связного планарного графа. Курсив — для связного графа",
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вытягивание",
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню",
 formal:
 "Три случая: zig (родитель — корень), zig-zig (узел — левый ребенок — левый ребенок), zig-zag (узел — левый ребенок — правый ребенок)",

```



```

 ось — есть отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше.",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд–Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки

 шить пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядк.",
 "ами.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажд
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"]
 }

```

```

 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "графа",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города и дороги.",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двусторонняя улица).",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить только порядковый номер.",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова дерево"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить через них. Ожидаемая высота $O(\log n)$).",
 complexity: "ожидаемо $O(\log n)$ ",
 demo: "heap",
}
```

```

 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строки"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где встречается

 (Z-функция) и где встречаются слова словаря (бор).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "повороты"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на место

 дившемуся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x$.

 g и Zig-Zag.",
 complexity: "O(1)",
 demo: "zig",
 simulator: "splay-tree",
 },
],

```

```

export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {
 /** Все метки, отсортированные от длинных к коротким (ч
 export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
 }).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 520 | РАЗМЕР: 10.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `<section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Введение
 <h2 class="text-3xl font-bold text-white">Базовая база
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база – это
 дерево как массив, куча, очередь и стек, бинарное
 Кодим по минимуму: там, где за нас уже написали
 </p>
 </div>

 <!-- Таблица мнемоники графовых алгоритмов -->
 <div class="my-8 overflow-x-auto">
 <h3 class="text-xl font-bold text-white mb-4">Графовые

```

```

</tr>
<tr class="hover:bg-slate-800/50 bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 text-slate-500 italic">
</tr>
<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 font-mono text-emerald-500">
</tr>
<tr class="hover:bg-slate-800/50 bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 font-mono text-emerald-500">
</tr>
<tr class="hover:bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 text-slate-500 italic">
</tr>
<tr class="hover:bg-slate-800/50 bg-slate-800/50">
 <td class="p-3 font-bold text-white">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">
 <td class="p-3 border-r border-slate-800/50">

```

```

 n /
 па.</p>
 <pre class="bg-slate-950 border border-slate-700
tree = [None, 40, 20, 60, 10, 30, 50, 70] # tree[0] None
корень n=1
дети 40: 2*1=2 → 20 и 2*1+1=3 → 60
дети 20: 2*2=4 → 10 и 2*2+1=5 → 30
родитель 30: 5 // 2 = 2 → 20</pre>
 <p class="text-slate-300 text-sm mb-4">Цена: пусть
 > деревья – кучу (ниже) и дерево отрезков из
 emerald-300">2n+1 и <span class="font-mono
 ">(n-1)//2.</p>
 <details class="bg-slate-900 border border-slate-700
 <summary class="cursor-pointer px-4 py-2 text-slate-300
 <div class="px-4 pb-4 text-slate-300 text-sm">
 На уровне k
 /span> вершин, а перед ним суммарно k
 class="font-mono text-emerald-300">n и n
 0">2n и n
 де, включая heapq.
 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-emerald-400 mb-4">
 <p class="text-slate-300 text-sm mb-4">Куча
 детей. Минимум всегда лежит в k
 an>}, а вставка и извлечение минимума стоят k
 учи из готового массива (k
 ran> времени. Поиск <i>произвольного</i> элемен
 0">0(n) линейного перебора!</p>
 <pre class="bg-slate-950 border border-slate-700
import heapq
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого об

```

```

<pre class="bg-slate-950 border border-slate-700 p-4">
sorted([3, 1, 2]) # [1, 2, 3] – новый список
sorted([(1, "b"), (1, "a")], key=lambda t: t[0]) # по первому элементу
<p class="text-slate-300 text-sm">Что лежит внутри?
 отвечают: слияние двух рангов длины k стоит O(k log k)
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-l border-r border-slate-700">
 <h3 class="text-xl font-bold text-purple-400 mb-4">Балансировка AVL-дерева
 <p class="text-slate-300 text-sm mb-4">AVL-дерево – бинарное дерево, в котором
 для каждой вершины выполнено условие баланса:
 $abs(h_L - h_R) \leq 1$, где h_L и h_R – высоты левых и правых поддеревьев.
 Если условие не выполняется, то дерево называется несбалансированным.
 Для восстановления баланса выполняются повороты:
 <pre class="bg-slate-950 border border-slate-700 p-4">
 Малый поворот (цепочка вправо-вправо стрелочками):

```

```

 z x
 /> x /> \
 /> y z y

```

Зигзаг (вправо-влево): сначала малый поворот нижнего из детей, потом малый поворот верхнего – две перерисовки стрелок:

```

 z z x
 /> x /> x /> \
 /> y /> y z y

```

```

<p class="text-slate-300 text-sm mb-4">Поворот вправо вокруг
 родителя y среднего звена и перевесить одно из поддеревьев
 отцов вдоль пути вставки, и условие
<details class="bg-slate-900 border border-slate-700 p-4">
 <summary class="cursor-pointer px-4 py-2 text-slate-300">Алгоритм
 <div class="px-4 pb-4 text-slate-300 text-sm">
 x = right(z)

 right(z) = left(x) # внутреннее поддеревое
 left(x) = z # z становится левым ребенком
 высоты пересчитываются только у z и x – остальные не меняются
 </div>

```

```

 <p class="text-slate-300 text-sm">Куча
 ь, не нужно переигрывать все матчи. Снял по
 тавшихся.</p>
 </div>
 </div>
 </div>

 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-indigo-400 mb
 <p class="text-slate-300 text-sm leading-relaxe
 База закрыта — теперь ею пользуются остальные
 Кратчайшие пути живут в билетах 14–16 (<a data-
 derline font-semibold cursor-pointer">Дейкс
 300 hover:text-white underline font-semibol
 "text-indigo-300 hover:text-white underline
 остовные деревья — в билетах 18–20 (<a data-g
 semibold cursor-pointer">Краскал, Прим, Бор
 обходы и связность — в билетах 6–13 (<a data-
 white underline font-semibold cursor-pointe
 indigo-300 hover:text-white underline font-
 dges-code" class="text-indigo-300 hover:tex
 строки — в билетах 21–23 (<a data-goto="strin
 ont-semibold cursor-pointer">КМП, <a da
 xt-white underline font-semibold cursor-poi
 text-indigo-300 hover:text-white underline
 Начинать удобно с <a data-goto="graph-dfs-bfs
 t-semibold cursor-pointer font-bold">билета
 </p>
 </div>
 </div>

 </div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",

```



```

<details class="bg-slate-900/40 rounded-lg border-
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Строгое доказательство (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь (min-heap). Гарантирует корректность только для не-
 g V). Гарантирует корректность только для не-
 </div>
</details>

<div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Форд-Беллман
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-
 <h3 class="text-xl font-bold text-rose-400">
 Строгое доказательство (Скрыто)

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-indigo-400">Аналогия 1: Автомагистрали

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-4">
 евле долететь из Москвы в Париж, если мы сд
 <div id="slot-chapter-image-floyd" class="w-full h-40">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 три вложенных цикла. Сложность $O(V^3)$. Если
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{
 id: "aho-corasick",

```

```

 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 терминальные ссылки (term_link) – прямые мос
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 outl
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скры
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>
 ссылку <code>term_link</code> (ближайшая мен
 <div class="bg-black/50 p-4 rounded-lg font-mon
struct Node {
 map<char, int> go; // Предвычисленные пер
 int link = 0; // Суффиксная ссылка: куда
 int term_link = 0; // Терминальная ссылка: матр
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();

```

```
 вы от 0 до N, и уже по ним строим графы.</p>
 </div>
</div>
</section>`
}
];
```

ФАЙЛ 23/111: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 200 | РАЗМ

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "sparse-table": [
 {
 question: "Сколько памяти занимает двумерная разрезка матрицы?",
 options: [
 "N × M – столько же, сколько сама матрица",
 "N × M × log N × log M: на каждом уровне почти N × M",
 "N × M × 4 – по одной ячейке на каждый угол записанного элемента",
],
 correctIndex: 1,
 explanation: "Уровней log N по строкам и log M по столбцам. В сумме получается примерно 10 × 10 = 100 копий – ровно такова цена хранения матрицы.",
 },
 {
 question: "Можно ли считать сумму на прямоугольнике за O(1)?",
 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие можно только за O(1) на каждый блок",
],
 },
],
```

```

 correctIndex: 0,
 explanation: "Сумма накапливается: каждая клетка

 ная здесь как раз разреженная таблица (билет 2)
 },
 {
 question: "Запрос суммы прямоугольника (r1,c1)...(r2,c2)",
 options: [
 "S[r2][c2] - S[r1][c2] - S[r2][c1] + S[r1][c1]",
 "S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]",
 "S[r2+1][c2+1] - S[r1][c1]"
],
 correctIndex: 1,
 explanation: "С рамкой из нулей S[i][j] - это сумма

 ариант без +1 даёт ответ со сдвигом на клетку - 1
 },
 {
 question: "Почему левый верхний угол в формуле про

 options: [
 "Он входит и в левую, и в верхнюю полосу - его

 "Он всегда равен нулю, поэтому прибавление безоп

 "Так компенсируется рамка из нулей"
],
 correctIndex: 0,
 explanation: "Отрезая полосу слева и полосу сверху

 аланс: это и есть формула включений-исключений.
 },
 {
 question: "Можно ли тем же приёмом считать миниму

 options: [
 "Да, если заменить вычитание на min",
 "Нет: у минимума нет обратной операции, нужна р

 "Да, но только для неотрицательных чисел"
],
 correctIndex: 1,
 explanation: "Весь приём держится на вычитании. З

 ной операции нет. Для min, max и gcd берут разр
 },
 {

```

```

 "Такого графа не существует",
 "Эйлерова пути нет – нужно удалять лишние рёбра
],
 correctIndex: 2,
 explanation: "Если нечётных вершин больше 2, эйлер
 ровно 2 нечётные или 0."
 }
],
"bridges-code": [
 {
 question: "После DFS у нас low[u] = 4, а tin[v] =
 options: [
 "Да, так как low[u] (4) > tin[v] (2)",
 "Нет, low[u] должно быть меньше или равно",
 "Только если граф связный и неориентированный"
],
 correctIndex: 0,
 explanation: "Условие моста: low[u] > tin[v]. В н
 },
 {
 question: "Для чего нужен массив low[v]?",
 options: [
 "Для того чтобы было больше переменных в коде",
 "Чтобы знать минимальный tin предка, до которого
 "Чтобы хранить глубину рекурсии"
],
 correctIndex: 1,
 explanation: "low[v] хранит минимальное время вхо
 поиска мостов и точек сочленения."
 },
 {
 question: "Какова временная сложность алгоритма п
 options: [
 "O(V^2) – ищем мосты в квадрате",
 "O(V + E) – каждый элемент один раз",
 "O(2^V) – полный перебор"
],
 correctIndex: 1,
 }
]

```

---

ФАЙЛ 24/111: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТР0К: 292 | РАЗМ

---

```
import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";
```

```
/**
```

```
 * Прямая связь «компилятор ↔ открытая визуализация».
```

```
 *
```

```
 * Компилятор передаёт не «магический номер строки эталонного
```

```
 * выполненного Python-кода. Визуализация читает захардкоженные
```

```
 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие
```

```
 * любой пользовательский код работает, если в нём использованы
```

```
 */
```

```
export const VizChapterContext = createContext("");
```

```
const CHANNEL = "algo:viz-state";
```

```
export interface VizStateEventDetail {
```

```
 chapterId: string;
```

```
 /** Строка и функция реально выполненного пользователя */
```

```
 line?: number;
```

```
 func?: string;
```

```
 /** Машинно-читаемые globals + locals: числа остаются
```

```
 variables?: Record<string, DebugValue>;
```

```
 /** Имена, изменённые выделенной строкой. */
```

```
 changed?: string[];
```

```
}
```

```
/** Последний снимок хранится, чтобы поздно смонтировать
```

```
const latestByChapter = new Map<string, VizStateEventDetail>();
```

```
/** Транслировать реальные переменные визуализациям текста
```

```
export function emitVizState(
```

```

export type VizStepSelector = (variables: Record<string>

/**
 * Подписка старых покадровых демонстраций. Если переда
 * номер кадра выбирается по именам/значениям переменны
 */
export function useVizStepSync(
 currentStep: number,
 goToStep: (n: number) => void,
 maxStep: number,
 selectByVariables?: VizStepSelector
) {
 const chapterId = useContext(VizChapterContext);

 useEffect(() => {
 if (!chapterId) return;

 const apply = (detail?: VizStateEventDetail) => {
 if (!detail || detail.chapterId !== chapterId) re
 // Без явного селектора компонент либо рисует use
 // либо остаётся ручным. Номер Python-строки нико
 const requested =
 selectByVariables && detail.variables ? selectB
 // null означает: компонент рисует runtime-перемен
 // не увидел знакомых захардкоженных имён.
 if (requested === null || !Number.isFinite(requested))
 const target = Math.max(0, Math.min(Math.max(0, m
 if (target !== currentStep) goToStep(target);
 };

 apply(latestByChapter.get(chapterId));
 const handler = (e: Event) => apply((e as CustomEvent)
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
 }, [chapterId, currentStep, maxStep, goToStep, selectByVariables]);
}

```



```

 hasNumbers: boolean;
}

const MAX_MATRIX_SIDE = 64;

const toCellNumber = (value: DebugValue | undefined): number | null {
 if (typeof value === "number" && Number.isFinite(value)) return value;
 // float("inf") снапшотится строкой "inf" – показываем NaN
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "+inf" || t === "infinity") return Number.POSITIVE_INFINITY;
 if (t === "-inf" || t === "-infinity") return Number.NEGATIVE_INFINITY;
 const n = Number(t);
 return Number.isFinite(n) ? n : null;
 }
 return null;
};

/** Ключ-пара вида "{0, 3}" или "0, 3" → [0, 3]. */
const parsePairKey = (key: string): [number, number] | null {
 const m = key.match(/^\(?\s*(-?\d+)\s*,\s*(-?\d+)\s*\)?$/);
 if (!m) return null;
 return [Number(m[1]), Number(m[2])];
};

export function liveMatrix(value: DebugValue | undefined): Matrix {
 if (value === undefined || value === null) return null;

 const cells = new Map<string, number>();
 let rows = 0;
 let cols = 0;
 const put = (i: number, j: number, v: DebugValue | undefined) => {
 if (!Number.isInteger(i) || !Number.isInteger(j) ||
 i < 0 || j < 0 || i >= MAX_MATRIX_SIDE || j >= MAX_MATRIX_SIDE) return;
 const n = toCellNumber(v);
 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 };

```

```

 const j = Number(k2);
 if (Number.isInteger(j)) put(i, j, v2);
 }
}
}

if (rows === 0 || cols === 0) return null;
return {
 rows,
 cols,
 at: (i, j) => (i >= 0 && j >= 0 && i < rows && j < cols),
 sizeLabel: `${rows}x${cols}`,
 hasNumbers: cells.size > 0,
};
}

/** Первое имя из списка, которое реально есть в снимке
export function pickLiveVar(
 vars: Record<string, DebugValue> | undefined,
 names: string[]
): { name: string; value: DebugValue } | null {
 if (!vars) return null;
 for (const name of names) {
 if (Object.prototype.hasOwnProperty.call(vars, name)) {
 return { name, value: vars[name] };
 }
 }
 return null;
}

/** Индекс, ограниченный реальной размерностью данных
export const clampIndex = (value: number | null, size: number): number {
 if (value === null || !Number.isFinite(value) || size <= 0) {
 return null;
 }
 return Math.max(0, Math.min(size - 1, Math.trunc(value)));
}

/* — Выбор демонстрации внутри страницы (вкладки ID / ID)
const DEMO_CHANNEL = "algo:viz-demo";

```

```

/**
 * Захардкоженная «карта синхронизации» компилятора с ви
 *
 * Для каждой страницы (главы) здесь вручную перечислены
 * которые понимает её визуализация: i, j, k, v, dist, l
 * Компилятор выполняет любой Python-код и передаёт реал
 * имён напрямую; привязки к номерам или тексту строк э
 */

export interface SyncVariable {
 /** Имя переменной, как в коде визуализации: i, j, k,
 name: string;
 /** Роль в алгоритме (показывается во всплывающей под
 role: string;
 /** Характерный диапазон/значение на демонстрации. */
 range?: string;
}

export interface PageSync {
 /** Заголовок демонстрации на странице (если интеракт
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тул
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет
 variables: SyncVariable[];
 /** Полный референсный Python-код, который кнопка-гла
 code?: string;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков",
 stepNote: "Шаг = спуск по вершине v: либо читаем tr
 code: `A = [5, 8, 3, 12, 7, 2, 0, 0] # исходный мас
a = A # псевдоним a
n = len(A) # длина массива n=8

```

```

"""Декартово дерево из отсортированных по x точек:
pts = sorted(pts, key=lambda p: p[0])
stack = []
for x, y in pts:
 cur = Node(x, y)
 last = None
 while stack and stack[-1].y < y: # поднимаемс
 last = stack.pop()
 cur.l = last
 if stack:
 stack[-1].r = cur
 stack.append(cur)
return stack[0] if stack else None

```

```

def split(t, m):
 """(L, R): в L всё с x <= m, в R — с x > m."""
 global step
 if t is None:
 return (None, None)
 step += 1
 if t.x <= m:
 a, b = split(t.r, m)
 t.r = a
 return (t, b)
 a, b = split(t.l, m)
 t.l = b
 return (a, t)

```

```

def merge(a, b):
 """Слияние: все x в a меньше всех x в b."""
 global step
 if a is None: return b
 if b is None: return a
 step += 1
 if a.y > b.y:
 a.r = merge(a.r, b)
 return a
 b.l = merge(a, b.l)

```

```

 { name: "height", role: "высота дерева: в среднем
 { name: "root", role: "корень – узел с максимальн
],
},
"splay-tree": {
 vizTitle: "Splay: Zig / Zig-Zig / Zig-Zag на узлах
 stepNote: "Шаг = одно применение случая: переменная
 rota.",
 code: `# Splay-дерево: три случая поворота на узлах
x, p, g = None, None, None # узел, родитель, дед (о
xk, pk, gk = 0, 0, 0 # их ключи-числа: 20 / 4
case = "zig" # какой случай сработал:
step = 0 # номер поворота
root = None # корень текущего дерева

--- тело алгоритма ---
class N:
 def __init__(s, k):
 s.k = k; s.l = None; s.r = None; s.p = None

def rot(x): # один поворот: поднять
 p = x.p
 g = p.p if p else None
 if p and p.l is x:
 p.l = x.r
 if x.r: x.r.p = p
 x.r = p
 elif p:
 p.r = x.l
 if x.l: x.l.p = p
 x.l = p
 if p: p.p = x
 x.p = g
 if g:
 if g.l is p: g.l = x
 else: g.r = x

def inorder(n, acc): # обход слева направо –

```

```

 { name: "step", role: "номер применения случая — 1",
 { name: "xk / x", role: "ключ узла, который поднимается",
 { name: "pk / p", role: "ключ родителя x перед пометкой",
 { name: "gk / g", role: "ключ деда — нужен в Zig-zag",
 { name: "root", role: "корень дерева после расшейвления",
],
},
"sparse-table": {
 vizTitle: "Разреженная таблица (1D и 2D)",
 stepNote: "Шаг = одна ячейка $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j - 1])$ ",
 code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):
 st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])

for row in st:
 print(row) # готовая таблица: None там, где не надо

запрос min на отрезке за O(1): отрезок покрывают ДВА блока
L, R = 1, 6
K = (R - L + 1).bit_length() - 1
print("min на [" + str(L) + ", " + str(R) + "] =",
 min(st[L][K], st[R - (1 << K) + 1][K]),
 "| блоки длины", 1 << K)`
variables: [
 { name: "a", role: "исходный массив, из которого берем элементы",
 { name: "n", role: "длина массива a", range: "n = len(a)",
 { name: "LOG", role: "число уровней таблицы: log2(n)",
 { name: "st", role: "разреженная таблица: st[i][j] — минимальный элемент в отрезке [i, i + 2^j)",
 { name: "i", role: "начало блока, за который отвечает таблица",
 { name: "j", role: "уровень таблицы: ячейка хранит минимальный элемент в отрезке [i, i + 2^j)",
 { name: "row", role: "очередной элемент a, из которого берем элементы"

```

```

print("итог:", fib(n))`,
 variables: [
 { name: "n", role: "целевой номер числа Фибоначчи"},
 { name: "k", role: "аргумент текущего рекурсивного вызова"},
 { name: "memo", role: "кэш уже посчитанных значений"},
 { name: "r1, r2", role: "ответы двух рекурсивных вызовов"},
],
},
"heap-beam-search": {
 vizTitle: "Куча (priority queue)",
 stepNote: "Шаг = просеивание: вставленный элемент в кучу",
 code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас просеиваем"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой вершины на стек или pop",
 code: `st = [] # стек = рекурсивный вызов
st.append("A") # push — зашли в вершину
st.append("B")
v = st[-1] # вершина на вершине стека:
top = st.pop() # pop — сняли её со стека
print("v =", v, "| pop →", top, "| стек:", st)`,
 variables: [
 { name: "top", role: "вершина стека — то, откуда мы пришли"},
],
},

```

```

"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к непосещенной",
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
visited, order = set(), []

stack = ["A"] # DFS: свой стек вместо рекурсии
while stack:
 v = stack.pop() # шаг: достали вершину с вершины
 if v in visited:
 continue
 visited.add(v); order.append(v)
 for u in adj[v]: # соседи v → на стек
 stack.append(u)
print("DFS:", order)

visited, order = set(), []
q = deque(["A"]) # BFS: очередь, обход по слоям
while q:
 v = q.popleft() # шаг: достали из начала очереди
 if v in visited:
 continue
 visited.add(v); order.append(v)
 for u in adj[v]: # соседи v → в конец очереди
 q.append(u)
print("BFS:", order)`

variables: [
 { name: "v", role: "текущая вершина обхода", gang: 1 },
 { name: "to / u", role: "очередной сосед вершины v", gang: 1 },
 { name: "visited", role: "множество уже посещённых вершин", gang: 1 },
 { name: "order", role: "порядок, в котором обходился граф", gang: 1 }
],
},
"graph-components": {
 vizTitle: "Компоненты связности: DFS-покраска",
 stepNote: "Шаг = вершина v достаётся из стека/очереди и окрашивается в цвет",
 code: `

```



```
{ name: "v", role: "вершина, где обход находится"},
{ name: "stack", role: "фронт обхода: стек (DFS)"},
{ name: "i / step", role: "номер шага – перематывае"},
],
},
"top-sort": {
 vizTitle: "Топологическая сортировка",
 stepNote: "Шаг = выход рекурсии из вершины v: она д",
 code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2]},
used, order = set(), []

def dfs(v):
 used.add(v)
 for to in adj[v]: # шаг: ребро v → to
 if to not in used:
 dfs(to)
 order.append(v) # выход из v!

for v in adj:
 if v not in used:
 dfs(v)

answer = order[::-1] # разворот – топологический
for i, v in enumerate(answer):
 print(f"позиция {i}: вершина {v}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "вершина, куда ведёт ребро из
 { name: "i", role: "перебор стартовых вершин во в
 { name: "order", role: "список выхода из рекурсии
],
},
"scc-kosaraju": {
 vizTitle: "Компоненты сильной связности (Косарайю)",
 stepNote: "Шаг = строка алгоритма: в фазе 1 DFS кла
 рованному графу и красит свою SCC.",
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [4], 4:
n = 5
```

```

 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
 c += 1

print("порядок выхода:", order, "| компонент:", c, "| р
variables: [
 { name: "adj / adjT", role: "граф и его транспози
 { name: "v", role: "текущая вершина DFS (в фазе 1
 { name: "to", role: "сосед по ребру: в фазе 1 пря
 { name: "used", role: "помеченные первым проходом
 { name: "order", role: "порядок ВЫХОДА из DFS пер
 { name: "i", role: "индекс прохода по order на вто
 { name: "stack", role: "стек DFS второго прохода:
 { name: "u", role: "вершина, снятая со стека во в
 { name: "comp", role: "номер компоненты сильной с
 { name: "c", role: "счётчик компонент: каждая нов
 { name: "n", role: "число вершин", range: "5" },
],
},
"graph-articulation": {
 vizTitle: "Точки сочленения",
 stepNote: "Шаг = обновление low[v] по ребру; вершин
 code: `tin, low, timer = {}, {}, 0

i = 0 # номер шага DFS
v, to = 0, 1 # текущее ребро DFS: v → to
tin[v] = tin.get(v, timer); low[v] = tin[v]

шаг: возврат из to в v
i += 1
low[v] = min(low[v], low.get(to, tin[v]))
critical = low.get(to, 0) >= tin[v] # → v — точка соч.
print(f"i={i} v={v} to={to} critical={critical}")`,
 variables: [

```

```

path = [start] # маршрут: пос.
last = start # где мы сейчас
stack = [start] # стек для отката
v = start
step = 0

--- тело алгоритма ---
while stack:
 v = stack[-1]
 moved = False
 for to, i in adj[v]:
 if not used[i]:
 used[i] = True
 stack.append(to)
 path.append(to)
 last = to
 step += 1
 moved = True
 break
 if not moved:
 stack.pop() # из v все рёбра
print("степени:", deg)
print("нечётных вершин:", len(odd), odd, "->", "цикл" if len(odd) > 1 else "нет")
print("маршрут:", path, "| рёбер пройдено:", sum(used), "из", len(adj))
variables: [
 { name: "path", role: "маршрут — последовательность вершин"},
 { name: "last / v", role: "вершина, где обход находится"},
 { name: "used", role: "какие рёбра уже пройдены (True/False)"},
 { name: "deg", role: "степени вершин: чётные всюду, нечётные в цикле"},
 { name: "odd", role: "список нечётных вершин — по порядку"},
 { name: "stack", role: "стек отката алгоритма Иерархического обхода"},
 { name: "step / i", role: "номер шага обхода", range: 1, len(adj)},
],
},
"planarity-euler-formula": {
 vizTitle: "Покраска вершин: $\chi \leq \Delta + 1$ ",
 stepNote: "Шаг = одна вершина v: смотрим цвета её соседей",
 code: `n = 8 # вершин в графе

```

```

 },
 dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

g = {"A": [{"B", 4}, {"C", 2}], "B": [{"G", 4}, {"D", 5}],
 "E": [{"F", 1}], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
 du, u = heapq.heappop(pq)
 if u in done:
 continue
 done.add(u)
 print(f"обрабатываем {u} (dist={du})")
 for v, w in g[u]:
 if v in done:
 continue
 print(f"смотрим ребро {u}→{v} (w={w})")
 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)`
 variables: [
 { name: "u", role: "вершина с минимальным dist среди не обработанных", range: "поиск"},
 { name: "v", role: "сосед вершины u, до которого есть ребро", range: "поиск"},
 { name: "w", role: "вес ребра (u, v)", range: "поиск"},
 { name: "dist[v]", role: "лучшее известное расстояние от A до v", range: "поиск"}
],
 },
 "bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
 code: `INF = float("inf")

```

```

 { name: "n", role: "число вершин графа – ровно n-1",
 { name: "m", role: "число рёбер, по которым проходим",
 { name: "i", role: "номер итерации (после i итераций)",
 { name: "u, v, w", role: "текущее ребро: откуда, куда, вес",
 { name: "dist[v]", role: "расстояние; если обновилось, то обновить",
 },
 },
 },
 },
 },
 },
 floyd: {
 vizTitle: "Флойд–Уоршелл",
 stepNote: "Шаг = инициализация матрицы, смена промежуточного узла",
 code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
 [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, INF, 0],
 [INF, INF, INF, INF, INF, INF, 0], [INF, INF, INF, INF, INF, INF, 0]]
n = 7
for k in range(n):
 print(f"посредник k={k}")
 for i in range(n):
 for j in range(n):
 if i != j and i != k and j != k and d[i][k] < INF and d[k][j] < INF:
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
 variables: [
 { name: "n", role: "число вершин – размер квадратной матрицы",
 { name: "k", role: "промежуточная вершина – ВНЕШНИЙ цикл",
 { name: "i", role: "начало пути – строка матрицы",
 { name: "j", role: "конец пути – столбец матрицы",
 { name: "dist[i][j]", role: "кратчайший путь i → j",
 },
 },
 },
 },
 },
 },
],
 },
 "johnson-algo": {
 vizTitle: "Алгоритм Джонсона (перевзвешивание)",
 stepNote: "Шаг = Беллман-Форд из фиктивной вершины",
 code: `# потенциалы из Беллмана-Форда от фиктивной вершины
h = {"A": 0, "B": -3, "C": -1, "D": 1, "E": 2, "F": 4}

```

```

 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)\,
 variables: [
 { name: "n", role: "число вершин (A...I); в остов в"},
 { name: "edges", role: "список рёбер (вес, u, v) -"},
 { name: "i", role: "номер текущего ребра в отсорти"},
 { name: "u, v, w", role: "концы и вес ребра, котор"},
 { name: "parent", role: "DSU: представитель множе"},
 { name: "rank", role: "высота дерева DSU – union f"},
 { name: "mst", role: "взятые рёбра остова – рисуем"},
 { name: "total", role: "вес остова; на этом графе"}
],
},
"string-kmp": {
 vizTitle: "Префикс-функция (КМП)",
 stepNote: "Шаг = обработка символа s[i]: откаты по",
 code: `s = "aabaabaaba"
n = len(s)
i, j = 1, 0
pi = [None] * n # пустые клетки: значения п
pi[0] = 0

for i in range(1, n): # шаг: считаем pi[i]
 j = pi[i - 1] # длина совпадения
 while j > 0 and s[i] != s[j]:
 j = pi[j - 1] # откат назад
 if s[i] == s[j]:
 j += 1
 pi[i] = j
print(pi)\,
 variables: [
 { name: "n", role: "длина строки s, для которой с"},
 { name: "i", role: "индекс текущего символа – счи"}

```

```

state = 0 # текущее состояние
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин
queue = [] # очередь BFS, к которой добавляются новые вершины
fail = {} # fail[v] = вершина, с которой нужно начинать поиск
matches = [] # найденные вхождения
```

```
--- тело алгоритма ---
```

```
nxt = {} # nxt[(v, ch)] = u — переходы бора
term = {} # term[v] = слово, которое заканчивается в вершине v
```

```
def add_word(w):
 """Добавить образец в бор: недостающие вершины будут добавлены
 global nodes
 v = 0
 for ch in w:
 if (v, ch) not in nxt:
 nxt[(v, ch)] = nodes
 nodes += 1
 v = nxt[(v, ch)]
 term[v] = w
```

```
for w in words:
 add_word(w)
```

```
BFS по уровням: fail-ссылка ребенка = куда приведёт переход по символу
fail[0] = 0
queue = sorted(v for (p, ch), v in nxt.items() if p == 0)
for v in queue:
 fail[v] = 0
qi = 0
while qi < len(queue):
 v = queue[qi]
 qi += 1
 for (p, ch), u in sorted(nxt.items()):
 if p != v:
 continue
```

```

],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа
ops = 0 # сколько операций насчитал
cls = "P" # класс, к которому относим
step = 0 # кадр схемы: 0 вход → 1 алг
k = 0 # длина сертификата: сколько
cert = [] # сам сертификат – предполага
ok = True # проверился ли сертификат э

--- тело алгоритма ---
Полиномиальный алгоритм: перебор всех пар – $\Theta(n^2)$.
Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
 step = 1 # запустили алгоритм на вход
 ops = 0
 for i in range(n):
 for j in range(i, n):
 ops += 1
 step = 2 # ответ готов
 cls = "P" # $n^2/2$ операций – полином, э
 print("n =", n, "| ops =", ops, "|", cls)

Сертификат NP-задачи (3-SAT): решение угадано, провер
step = 3
cert = [1, 0, 1, 0] # значения переменных x1...x4
k = len(cert) # проверяем все k бит
ok = all(bit in (0, 1) for bit in cert)
cls = "NP" # угадать + быстро проверить
print("сертификат:", cert, "| k =", k, "| принят:", ok,
 variables: [
 { name: "n", role: "размер входа – горизонтальная
 { name: "ops", role: "сколько операций выполнил а
 " },
 { name: "cls", role: "имя класса (P / NP / PSPACE

```



```

print("chi =", chi, "| delta =", delta, "| chi <= delta
 variables: [
 { name: "color", role: "цвета вершин (0 = не покра",
 { name: "v", role: "текущая вершина", range: "0 ...",
 { name: "delta / chi", role: "максимальная степень",
 { name: "i / step", role: "номер кадра", range: "0 ...",
],
},
"planarity-euler-formula#bipartite": {
 vizTitle: "Двудольность: BFS в 2 цвета",
 stepNote: "Шаг = сосед u получает цвет 3 - color[v]",
 code: `n = 8
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4], [3, 4, 5]]
color = [0] * n # 0 = не покрашена, иначе 1 или 2
queue = [] # очередь BFS
v = -1 # вершина, из которой смотрим
u = -1 # сосед, которого красим сейчас
ok = True # двудольный ли граф
i = 0

--- тело алгоритма ---
for start in range(n):
 if color[start] != 0:
 continue
 color[start] = 1
 queue = [start]
 while queue and ok:
 v = queue.pop(0)
 for u in adj[v]:
 if color[u] == 0:
 color[u] = 3 - color[v] # противоположный цвет
 queue.append(u)
 i += 1
 elif color[u] == color[v]:
 ok = False # нечётный цикл
 print("конфликт на ребре", (v, u))
print("двудольный:", ok, "| разметка:", color)`,
 variables: [

```

```

 { name: "inc", role: "для каждой вершины: какие ц
 { name: "chi_e / delta", role: "χ' — сколько цвет
],
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",
 stepNote: "Шаг = вершина v достаётся из стека; сосе
 code: `n = 10 # вершин в графе
adj = [[1, 3], [0, 2, 3], [1, 3], [0, 1, 2], [5, 6], [4
comp = [-1] * n # -1 = вершина ещё не покраш
used = [False] * n # то же самое булевым массив
comps = 0 # счётчик найденных компонен
c = 0 # номер текущей компоненты
v = -1 # вершина, в которой обход н
stack = [] # фронт обхода (стек для DFS
i = 0 # номер шага — двигает покард

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => H
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`,
 variables: [

```

```

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)
print("корни:", [find(x) for x in range(n)])`,
 variables: [
 { name: "parent", role: "DSU-массив: parent[v] = 0 ... n-1" },
 { name: "rank", role: "высота дерева DSU – union" },
 { name: "e / step", role: "номер обрабатываемого ребра" },
 { name: "u, v", role: "концы текущего ребра – подмножества" },
 { name: "comps", role: "сколько множеств осталось" },
],
},
"complexity-classes#hierarchy": {
 vizTitle: "Иерархия классов $P \subseteq NP \subseteq PSPACE \subseteq EXP$ ",
 stepNote: "Шаг не нужен: схема подсвечивает тот овал, который является подмножеством",
 code: `cls = "NP" # имя класса: P, NP, PSPACE, EXP
n = 10 # размер входа
ops = 0 # счётчик операций (нужен вк. в переменную)

--- тело алгоритма ---
P: решается за полином – сортировка
a = [5, 2, 8, 1, 9, 3]
for i in range(len(a)):
 for j in range(len(a) - 1):
 ops += 1
 if a[j] > a[j + 1]:
 a[j], a[j + 1] = a[j + 1], a[j]
print(cls, "отсортировано за", ops, "операций:", a)`,
 variables: [
 { name: "cls", role: "имя класса – P, NP, PSPACE, EXP" },
 { name: "n", role: "размер входа задачи, про который идёт речь" },
 { name: "ops", role: "сколько операций потребовалось" },
],
},

```

```

 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение пер
 { name: "k", role: "номер дизъюнкта, который пров
 { name: "ok", role: "итоговый вердикт: сертификат
 { name: "clauses", role: "сама формула 3-SAT – 4
],
},
"complexity-classes#growth": {
 vizTitle: "Рост числа операций: полином или экспонен
 stepNote: "Шаг = одна точка (n, ops). Чем больше то
 code: `n = 1 # размер входа
ops = 0 # сколько операций насчитал

--- тело алгоритма ---
Полиномиальный алгоритм: перебор всех пар – $O(n^2)$.
Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
 ops = 0
 for i in range(n):
 for j in range(i, n):
 ops += 1
 print("n =", n, "ops =", ops)

Экспоненциальный (полный перебор подмножеств) – раскол
for n in range(1, 15):
ops = 2 ** n
print("n =", n, "ops =", ops)`,
 variables: [
 { name: "n", role: "размер входа – по нему строит
 { name: "ops", role: "число операций алгоритма на
 { name: "i", role: "внутренний счётчик цикла – то
],
},
"mst#kruskal": {

```

```

],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

n = 9 # вершин: A..I
adj = {"A": [("B", 2), ("D", 5)], "B": [("A", 2), ("C", 3), ("E", 4)],
 "C": [("B", 4), ("F", 9)], "D": [("A", 5), ("E", 3), ("G", 8)],
 "E": [("D", 3), ("B", 6), ("F", 7), ("H", 11)], "F": [("C", 9), ("E", 7), ("I", 13)],
 "G": [("D", 8), ("H", 10)], "H": [("G", 10), ("E", 11), ("I", 12)],
 "I": [("H", 12), ("F", 13)]}}
start = "E" # из какой вершины растим дерево
heap = [(0, start, start)] # (вес, вершина, откуда пришло ребро)
taken = [] # вершины уже в остовае — краем
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
u, v, w = None, None, None # текущее ребро

--- тело алгоритма ---
while heap:
 w, x, frm = heapq.heappop(heap) # самое лёгкое ребро
 if x in taken: # устаревшая запись
 continue
 taken.append(x)
 u, v = frm, x
 if frm != x:
 mst.append((w, frm, x))
 total += w
 for to, wt in adj[x]:
 if to not in taken:
 heapq.heappush(heap, (wt, to, x))
print("вес остова:", total, "| вершин захвачено:", len(taken))
print(mst)`
variables: [
 { name: "start", role: "вершина, из которой растим дерево",
 value: "E",
 },
 { name: "heap", role: "куча рёбер-кандидатов (вес, вершина, откуда пришло ребро)",
 value: [(0, "E", "E")],
 },
 { name: "taken", role: "вершины уже в остовае — краем",
 value: ["E"],
 },
 { name: "mst", role: "рёбра остова",
 value: [],
 },
 { name: "total", role: "вес остова (= 51)",
 value: 0,
 },
 { name: "u, v, w", role: "текущее ребро",
 value: [None, None, None],
 },
]

```

```

if not cheapest:
 break # граф несвязен
for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты
 comp[ra] = rb
 mst.append((w, a, b))
 total += w
print("фаз:", phase, "| вес остова:", total)
print(mst),
variables: [
 { name: "comp", role: "DSU: корень компоненты вер",
 { name: "cheapest", role: "корень компоненты -> с",
 { name: "phase", role: "номер фазы; на этом графе",
 { name: "u, v, w", role: "ребро-кандидат, которое",
 { name: "mst / total", role: "взятые рёбра и вес "
],
},
"sparse-table#2d-build": {
 vizTitle: "Разреженная таблица 2D: построение",
 stepNote: "Шаг = новый уровень k: блоки 2^k×2^k доп",
 code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
 88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
 1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

--- тело алгоритма ---
База не отдельный уровень вычислений: блок 1x1 – это
Записываем её одним шагом, а не проигрываем 64 одноти
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(((r, c, 0, 0), A[r][c]) for r in range(
for k in range(1, 4):
 print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
 size = 9 - (1 << k)
 half = 1 << (k - 1)

```

```

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`,
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (0 ≤ i, j ≤ 1000)", },
 { name: "S[i][j]", role: "сумма прямоугольника (0 ≤ i, j ≤ 1000)", },
 { name: "r1, c1, r2, c2", role: "углы запроса; от 1 до 1000", },
],
},
};

```

```

/** Безопасный доступ: страница без записи получает пустую страницу.
export const getPageSync = (chapterId?: string | null, demoId?: string) => {
 (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demoId}`]) ||
 (chapterId && PAGE_SYNC[chapterId]) ||
 { variables: [] };

```

```

/**
 * Стартовое содержимое редактора для страницы.
 *
 * Формат: короткий комментарий-шапка + исполняемый референт.
 * Визуализация читает из его трассы знакомые имена (i, j, S[i][j]),
 * но точно так же примет пользовательский код с теми же именами.
 */

```

```

export function buildSyncTemplate(chapterId: string, chapterTitle: string) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .`
 : `# «${chapterTitle}»\n# демо на странице нет — сводка`;

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}

```

```

/**

```

ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРӨК: 218 | РАЗМ

```
import { Chapter } from '../../../types';
```

```
/**
```

```
 * Билет 1 – «Дерево отрезков с операциями на отрезках»
```

```
 *
```

```
 * Страница написана по канону остальных билетов (эталон
```

```
 * одна главная секция, строгое правило по центру, анализ
```

```
 * явная строка сложности, весь код спрятан в <details>
```

```
 *
```

```
 * Визуализация – src/components/SegmentTreeVisualizer.ts
```

```
 * построение / запросы / теория), синхронизация с комп
```

```
 * (переменные n , i , v , l , r , m) и её подрежимы #build
```

```
 */
```

```
export const ticket1Segtree: Chapter[] = [
```

```
 {
```

```
 id: "segment-trees",
```

```
 title: "1. Дерево отрезков с операциями на отрезках",
```

```
 type: "html",
```

```
 description:
```

```
 "Структура для запросов на отрезке и обновлений з
```

```
 category: "Продвинутые структуры",
```

```
 content: `
```

```
<section id="segment-trees" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
```

```
 <h3 class="text-xl font-bold text-blue-400">
```

```
 </div>
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-6">
```



## Аналогия 1: Корпоративная пре

```

</div>
<p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]) и
свой баланс. Это и есть отложенное обнов
</div>

```

```

<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">

```

## Аналогия 2: Матрёшки-коробки

```

</div>
<p class="text-slate-300 text-sm">М
Нужно покрасить половину массива в синий? Н
ь спускаемся только тогда, когда нас спроси
(push).</p>
</div>
</div>

```

```

<div class="bg-slate-800/70 p-5 rounded-lg
 <p class="text-sm font-bold text-white
 <p class="text-slate-300 text-sm mb-3">
на [L, R]» – всё это делается одним спуском
ss="font-mono text-emerald-300">tree[v] +=
: перед любым спуском в детей вызываем
<table class="w-full text-xs text-left
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300 font-m
 <tr class="border-b border-slate
x · len</td><td class="py-1.5">lazy[v] +=
 <tr class="border-b border-slate

```

в редакторах и базах, сжатие координат + «оить», дерево отрезков по дереву (HLD), пер

```
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
```

Код: построение и запрос суммы (C

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```

```
nt MAXN = 100005;
```

```
int tree[4 * MAXN]; // 4n – обязательно, 2n не х
```

```
int a[MAXN];
```

```
void build(int v, int l, int r) {
```

```
 if (l == r) { tree[v] = a[l]; return; } // лист
```

```
 int m = (l + r) / 2;
```

```
 build(2 * v, l, m); // левая
```

```
 build(2 * v + 1, m + 1, r); // права
```

```
 tree[v] = tree[2 * v] + tree[2 * v + 1]; // merge
```

```
}
```

```
int sum(int v, int l, int r, int L, int R) { // сумма
```

```
 if (r < L || R < l) return 0; //
```

```
 if (L <= l && r <= R) return tree[v]; //
```

```
 int m = (l + r) / 2;
```

```
 return sum(2 * v, l, m, L, R) // части
```

```
 + sum(2 * v + 1, m + 1, r, L, R);
```

```
}
```

```
void update(int v, int l, int r, int i, int x) { // a[i
```

```
 if (l == r) { tree[v] += x; return; }
```

```
 int m = (l + r) / 2;
```

```
 if (i <= m) update(2 * v, l, m, i, x);
```

```
 else update(2 * v + 1, m + 1, r, i, x);
```

```
 tree[v] = tree[2 * v] + tree[2 * v + 1]; // перес
```

```
}</pre>
```

</details>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">  
 <summary class="p-4 font-bold text-slate-300">  
 -b border-slate-700/50">

Код: итеративное дерево ( $2n$ , снизу)

</summary>

<div class="p-5 text-sm text-slate-300">  
 <pre class="bg-slate-950 p-4 rounded">  
 # для удобства

```
tree = [0] * (2 * n)
for i in range(n): # листья: tree[n + i] = a[i]
 tree[n + i] = a[i]
for v in range(n - 1, 0, -1): # предки снизу вверх
 tree[v] = tree[2 * v] + tree[2 * v + 1]
```

```
def query(l, r): # сумма на полуинтервале [l, r)
 res = 0
 l += n; r += n
 while l < r:
 if l & 1: res += tree[l]; l += 1 # l – правая граница
 if r & 1: r -= 1; res += tree[r] # r – правая граница
 l //= 2; r //= 2
 return res
```

<p class="text-slate-400 text-xs">Иногда это решение кажется вам не самым лучшим, но на самом деле оно действительно заметно хитрее – на экзамене безопаснее использовать проверенные шаблоны.  
 </div>

</details>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">  
 <summary class="p-4 font-bold text-slate-300">  
 -b border-slate-700/50">

Вопросы, которые любят задавать (и отвечать на которые не всегда просто)

</summary>

<div class="p-5 text-sm text-slate-300">

<p>• Почему  $4n$ , а не  $2n$ ? – при  $n$ , нечетном, количество листьев равно  $n + 1$ , а не  $n$ .  
 пример:  $n = 6$ ).</p>

<p>• Сколько вершин посещает запрос

```
<h3 class="text-xl font-bold text-emerald-400">Действие
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Действие — это требование к графу — неотрицательные веса
 <p class="text-xl font-mono text-white">д[u]
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Действие — это требование к графу — неотрицательные веса, и на каждом шаге забирает в него ту самую вершину.
n class="font-mono text-emerald-300">д[u]</p>
```

```
<p class="text-slate-300 text-sm mb-4">Почему? Потому что <u>у</u> обязан выйти из этой вершины, а у неё u
х, а у неё u означает неотрицательные веса, значит корректная механика: извлечь минимум → зафиксировать
ниже.</p>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700 p-4">
 <p class="text-[10px] uppercase tracking-wider">Действие
 <p class="font-mono text-emerald-300">д[u]
 </div>
 <div class="bg-slate-900 border border-slate-700 p-4">
 <p class="text-[10px] uppercase tracking-wider">Действие
 <p class="font-mono text-emerald-300">д[u]
 </div>
 <div class="bg-slate-900 border border-slate-700 p-4">
 <p class="text-[10px] uppercase tracking-wider">Действие
 <p class="font-mono text-emerald-300">д[u]
 </div>
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">
```

$(u \rightarrow v)$

: переменные `<span class="font-monospace text-emerald-300">u</span>`, `<span class="font-monospace text-emerald-300">v</span>` ают картинку напрямую.

```
<ol class="list-decimal pl-5 space-y-2 text-emerald-300">
 Инициализация. ∞; в кучу кладём prev пуст.
 Извлечь минимум. Достаём пару du. Если done – пара устарела.
 Зафиксировать. du меняется.
 Релаксировать рёбра. Для каж. u: если dist[u] < dist[v]:
 dist[u] = dist[v] + weight(u, v)
 prev[u] = v
 Повторять, пока куча не пуста. ∞.

```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">Таблица маршрутов
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">Пункт
 <th class="py-1.5 pr-2 font-normal">Время
 <th class="py-1.5 pr-2 font-normal">Длина
 <th class="py-1.5 pr-2 font-normal">Путь
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-700">
 <td class="py-2 font-normal">Пиццерия А (0)</td>
 <td class="py-2 font-normal">10 мин</td>
 <td class="py-2 font-normal">1 км</td>
 <td class="py-2 font-normal">А → Б → В</td>
 </tr>
 <tr class="border-b border-slate-700">
 <td class="py-2 font-normal">Парк С (2)</td>
 <td class="py-2 font-normal">20 мин</td>
 <td class="py-2 font-normal">2 км</td>
 <td class="py-2 font-normal">А → С → В</td>
 </tr>
 <tr class="border-b border-slate-700">
 <td class="py-2 font-normal">Дом 1 В (3)</td>
 <td class="py-2 font-normal">30 мин</td>
 <td class="py-2 font-normal">3 км</td>
 <td class="py-2 font-normal">А → В → С → В</td>
 </tr>
 </tbody>
 </table>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-indigo-400
```

**Инициализация**  $\text{dist}[v]$  извлекается из кучи,  $s \rightarrow u$ . Отсюда  $\text{dist}[v] = \delta(s, v)$  для всех достижимых  $v$ . Для недостижимых  $v$   $\text{dist}[v] = \infty$ .

```
<details class="bg-slate-900/40 rounded-lg p-4">
 <summary class="p-4 font-bold text-slate-100">Доказательство ин
```

Пусть `di` и `s` не извлечена, а `y` не первая

<p>Так как <span class="font-mono text-emerald-300">(x, y) </span>  
 <span class="font-mono text-emerald-300"> $\delta(s, x) + w(x, y) = \delta(s, y)$ </span>. Путь д</p>
 <span class="font-mono text-emerald-300">u</span>, веса </p>
 <span class="font-mono text-emerald-300"><math>\leq \text{dist}[u]</math></span>.</p>

Итого – противоречие с нет. ■

«Г, u)». С отрицательным ребром хвост п. о контрпример из карточки «Ограничения».

&lt;/details&gt;

```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">» Откуда берётся сл
 <div class="p-5 text-sm text-slate-300
```

```

 </tr>
 <tr class="border-b border-slate-200">
 ><td class="py-2 pr-3 font-mono">0(V + E)</td><td class="py-2 pr-3 font-mono">0(E + V · C)</td><td class="py-2">ц
 </tbody>
 </table>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">» Код: Дейкстра на
 <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
INF = float("inf")

def dijkstra(graph, start):
 """graph[v] = [(сосед, вес), ...]; веса >= 0."""
 dist = {v: INF for v in graph}
 prev = {v: None for v in graph}
 dist[start] = 0
 done = set()
 pq = [(0, start)] # (расстояние, v)

 while pq:
 du, u = heapq.heappop(pq)
 if u in done: # устаревшая информация
 continue
 done.add(u) # dist[u] окончено
 for v, w in graph[u]:
 if du + w < dist[v]: # релаксация
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
 return dist, prev

def restore(prev, start, target):

```

```

vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>,
 vector<pair<long long,int>&,
 greater<pair<long long,int>>>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {
 if (du + w < d[v]) {
 d[v] = du + w;
 p[v] = u;
 pq.push({d[v], v});
 }
 }
 }
}

vector<int> path(int s, int t) { // восст
 vector<int> res;
 if (d[t] == INF) return res; // недостижим
 for (int v = t; v != -1; v = p[v]) res.push_back(v)
 reverse(res.begin(), res.end());
 return res;
}
}

```

<p class="text-slate-400 text-xs">0  
 уть из  $10^5$  рёбер веса  $10^9$  даёт  $10^{14}$  – <span  
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">» Код:  $O(V^2)$  без куд  
 <div class="p-5 text-sm text-slate-300



```
 Ориентированность. Демо-
ом графе каждое ребро добавляется в оба спи
 Дейкстра $\neq$ BFS. BFS – ча
 С разными весами FIFO ломается, нужна куча
 «Одна пара». Алгоритм сч
ий выход при извлечении цели (билет 17), но

</div>
```

```
<p class="text-slate-300 text-sm mt-4 mb-4">
ов ценой скорости $O(V^3)$; б
после перевзвешивания потенциалов). Предст
```

```
 </div>
 </div>
</section>`
,
{
 id: "bellman-ford",
 title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="bellman-ford-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы. Пути
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">15. Графы. Поиск кратчайшего пути. Форд-Беллман</h2>
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-rose-400">15.1. Алгоритм Форда-Беллмана</h3>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана работает на графах с отрицательными весами, но не может найти кратчайший путь, если в графе есть отрицательный цикл.</p>
 </div>
 </div>
 </div>
```





```

 </tbody>
 </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-emerald-400">
 <h3 class="text-xl font-bold text-emerald-400">Алгоритм Форда-Беллмана
 </h3>
 <div class="bg-slate-900/60 p-4 rounded-lg border border-slate-700/50">
 <p class="text-xs font-bold text-amber-400">Описание:
 </p>
<pre class="bg-slate-950 p-3 rounded text-xs font-mono">
 d = [float("inf")] * n
 d[start] = 0
 # Бэтмен патрулирует n - 1 ночей
 for _ in range(n - 1):
 for u, v, w in edges:
 if d[u] + w < d[v]:
 d[v] = d[u] + w

 # На n-ю ночь проверяем отрицательный цикл ("бесконечная ночь")
 for u, v, w in edges:
 if d[u] + w < d[v]:
 return True # Бэтмен нашел бесконечную кормушку
 return False # Всё тихо, отрицательных циклов нет</pre>
 </div>
 <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 <b border="1" border-slate-700/50">> Код: Форд–Беллман
 </summary>
 <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-sm">
def bellman_ford(n, edges, s):
 """edges = [(u, v, w), ...] – ориентированные рёбра
 dist = [INF] * n
 prev = [-1] * n

```

```
dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D", i], i, 0) for i in range(1, len(nodes))])
print([dist[i] for i in range(len(nodes))]) # [0, 4, 1, 2, 3]
print(cyc) # None
```

```
dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D", i], i, 0) for i in range(1, len(nodes))])
print([nodes[i] for i in cyc2]) # ['D', 'A', 'B', 'C', 'D']
print([dist2[i] for i in range(len(nodes))]) # все вершины имеют расстояние 0
```

```
<p class="text-slate-400 text-xs">Примечание: если в графе есть отрицательный цикл, то функция вернёт None. Если же цикл не найден, то вернётся float('inf').</p>
</div>
</details>
```

```
<details class="bg-slate-900/40 rounded-lg p-4">
 <summary class="p-4 font-bold text-slate-400">Код: C++ (структура)</summary>
 <div class="p-5 text-sm text-slate-300">
```

```
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-400">
 #include <iostream>
 using namespace std;
 const long long INF = 4e18;
```

```

 struct Edge { int u, v; long long w; };

 int n; // число вершин
 vector<Edge> edges; // плоский список рёбер
 vector<long long> d;
 vector<int> p;
```

```

 struct Edge { int u, v; long long w; };

 int n; // число вершин
 vector<Edge> edges; // плоский список рёбер
 vector<long long> d;
 vector<int> p;
```

```

 // возвращает вершину, улучшенную на «лишней» итерации,
 int bellman_ford(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 int x = -1;
 for (int it = 0; it < n; ++it) { // n-1 итераций
 x = -1;
 for (const auto &e : edges) {
 if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
 d[e.v] = d[e.u] + e.w; p[e.v] = e.u;
 if (x == -1) x = e.v;
 }
 }
 }
 return x;
 }
```

```

 // возвращает вершину, улучшенную на «лишней» итерации,
 int bellman_ford(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 int x = -1;
 for (int it = 0; it < n; ++it) { // n-1 итераций
 x = -1;
 for (const auto &e : edges) {
 if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
 d[e.v] = d[e.u] + e.w; p[e.v] = e.u;
 if (x == -1) x = e.v;
 }
 }
 }
 return x;
 }
```

```

<div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 rt deque

```

```

def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые»
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина в очереди
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True
 while q:
 u = q.popleft()
 in_queue[u] = False
 for v, w in adj[u]:
 if dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 if not in_queue[v]:
 q.append(v)
 in_queue[v] = True
 counter[v] += 1
 if counter[v] >= n:
 return None # отрицательный цикл
 return dist

```

- Ранний выход. Если з
- рафах экономит кратно.
- SPFA (Shortest Path
- , у которых только что улучшился an, но **худший** случай остаётся cas из рёбер) SPFA ловит TL. Критерий цикла
- Порядок рёбер. Формал
- итерации, против течения – за иция даёт финальные 0(V + E), и
- DAG. Если граф ацикл



```

 <p class="font-mono text-emerald-300">
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300"
 </div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
rder border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в
 </div>
 <p class="text-slate-300 text-sm mb
це только соседние станции. Затем говорите:
 class="font-mono">через 1 там, где
плакат окончательный. Порядок «разрешений»
an>.</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 b
border-rose-500 shadow-md">
 Аналогия 2: Почему k обязан быть
 </div>
 <p class="text-slate-300 text-sm mb
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
 </div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400

```



```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 font-bold"
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">конечное</td><td class=
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">INF</td><td class="py-2"
 <tr class="border-b border-slate-
y-2 pr-2 font-mono">0</td><td class="py-2">
 <tr class="align-top"><td class:
se-300">< 0</td><td class="py-2 text-rose-400"
делены</td></tr>
 </tbody>
 </table>
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">
```

```
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">» Формальная динамика
 <div class="p-5 text-sm text-slate-300"
 <p>Обозначим <span class="font-mono"
o"> $i \rightarrow j$, все промежуточные вершины
font-mono"> i и <span class="font-mono"
 <p>База:
```

```

 "ikj": (c, a, b)}[order]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]

 return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good[i][j] != bad[i][j]))
4 клетки отличаются; при порядке (i,k,j) их 6</code><
<p class="text-slate-400 text-xs">В
угих «почти работают» (большинство клеток с
</div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border
<h3 class="text-xl font-bold text-emerald-400
 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-400
 -b border-slate-700/50">> Код: Флойд–Уоршелл
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

def floyd_warshall(n, edges, directed=True):
 d = [[INF] * n for _ in range(n)]
 nxt = [[-1] * n for _ in range(n)]
 for i in range(n):
 d[i][i] = 0 # диагональ об

 for u, v, w in edges:
 if w < d[u][v]: # кратчайше
 d[u][v] = w
 nxt[u][v] = v
 if not directed and w < d[v][u]:
 d[v][u] = w
 nxt[v][u] = u

```

```

-b border-slate-700/50">> Код: C++ (матрица
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <algorithm>;
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d;

void init_and_floyd() {
 d.assign(n, vector<long long>(n, INF));
 for (int i = 0; i < n; ++i) d[i][i] = 0;
 // занесение рёбер: d[u][v] = min(d[u][v], w);
 for (int k = 0; k < n; ++k) // k
 for (int i = 0; i < n; ++i) {
 if (d[i][k] == INF) continue; // защи
 for (int j = 0; j < n; ++j) {
 if (d[k][j] == INF) continue;
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]
 }
 }
}

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return false;
}

long long diameter() { // сама
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (d[i][j] < INF) res = max(res, d[i][j]
 return res;
}
```

```

pr-2 font-mono">при d[i][k]+d[k][j] == d[i]
nt[k][j] двойно считает пути (multi-countin
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt
> при r[i][k]</span
V³).</p>
 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-amber-400

```

```

<div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Порядок циклов. <span cl
s="font-mono">i и <span class="font-m
м порядке (k, i, j<
 Диагональ. <span class="
«починит» её только при наличии цикла, а п
 Переполнение INF. <span <
уходит в минус и «улучшает» всё подряд. Пр
no">long long.
 Параллельные рёбра. Во в
no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Если
>k принадлежит отрицательному циклу.
, достижимых ИЗ цикла в ПРЯМОМ графе
 Недостижимость ≠ 0. В ор
таких пар из 49). Выводить 999 или 0 как «р
 Память. <span class="font
ачений (≈100 МБ на
источка берите Дейкстру.
 Сам путь. Матрица <span <
pan class="font-mono">nxt[i][j] (как


```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Такси с навигатором
 </div>
 <p class="text-slate-300 text-sm mb
ы одинаково. А* — тот же таксист, но с нави
/span> прибавляется оценка остатка пути по
т туда, где сумма меньше. Навигатор обязан
/span>): занижать можно, завышать нельзя —
0">h = 0 это снова Дейкстра.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 2: Две бригады туннеля
 </div>
 <p class="text-slate-300 text-sm mb
пускают две: от А и от В. Каждая зона роста
я не в момент встречи, а когда сумма
— иначе можно пропустить более дешёвый обх
ирует.</p>
 </div>
</div>

<div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-2 pr-3 font-b
 <th class="py-2 pr-3 font-b
 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">

```

```
er border-amber-500 shadow-md">
```

```
 ☞ Суть перевзвешивания
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-3">
 Дейкстра (билет 14) быстрая, но не универсаль-
 ленный. Алгоритм Джонсона делает финт: пере-
 ь теми же.
```

```
</p>
```

```
<ol class="text-sm text-slate-300 space-y-2">
 Добавляем фиктивную вершину s
 Запускаем от s Форда-Беллмана. Важ-
 ажный факт: Джонсон НЕ «удаляет» отрица-
 кой и прекращает работу, так как при отрица-
 Перевзвешиваем: $w(u, v) \leftarrow w(u, v) + h(u, s) - h(v, s)$
 тво следует из того, что $h(v) \leq h(u) + w(u, v)$
 Магия: для любого пути P из s в t
) $= h(t) - h(s)$, то есть все пути между фик-
 еняется.
```

```
 Запускаем быструю Дейкстру от t .
 lass="font-mono text-emerald-300"> $d(u, v) = \min_{w \in P(u, v)} \sum_{(u, w) \in P} w(u, w)$


```

```
<p class="text-slate-400 text-xs mt-3">
 еженном графе ($E \approx V^2$) это $O(V^2 \log V)$ против
 ойд — для маленьких или очень плотных.</p>
```

```
</div>
```

```
</div>
```

```
<h3 class="text-lg font-bold text-white mt-3">Анализ
```

```
<p class="text-slate-300 text-sm mb-3">Если t — это
 азад из t (по перевёрнутым рёбрам). Волны р-
 : в «широком» графе (дорожная сеть) это эконо-
 огда волны «встретились» в одной вершине; на-
 огда $w(u, v) \leftarrow w(u, v) + h(u, s) - h(v, s)$
 более короткий путь.</p>
```

```
<h3 class="text-lg font-bold text-white mt-3">Анализ
```

```
<p class="text-slate-300 text-sm mb-3">Дейк-
 по $g(v)$ (уже посчитано) и $w(u, v)$ (уже посчитано)
 $g(v) + w(u, v)$ — это
```



```

while qf and qb:
 if qf[0][0] + qb[0][0] >= mu: # вот прав
 return mu
 dist, v = heapq.heappop(qf)
 if dist > df.get(v, INF): continue
 for to, w in g[v]:
 nd = dist + w
 if nd < df.get(to, INF):
 df[to] = nd; heapq.heappush(qf, (nd, to))
 if to in db:
 mu = min(mu, nd + db[to]) # встреча:
 dist, v = heapq.heappop(qb)
 if dist > db.get(v, INF): continue
 for to, w in rg[v]:
 nd = dist + w
 if nd < db.get(to, INF):
 db[to] = nd; heapq.heappush(qb, (nd, to))
 if to in df:
 mu = min(mu, nd + df[to])
return mu

--- A*: приоритет f = g + h ---
def astar(g, s, t, h): # h(v) — допустим
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход:
 if v in closed:
 continue
 closed.add(v)
 for to, w in g[v]:
 nd = gv + w
 if nd < gdist.get(to, INF):
 gdist[to] = nd

```



```
 </details>
 </div>
 </div>
</section>`
 },
];
```

---

ФАЙЛ 28/111: src/data/tickets/ticket18\_20\_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 231 | РАЗМ

---

```
import { Chapter } from '../..types';
```

```
/**
```

```
 * Билеты 18 · 19 · 20 — одна страница на три алгоритма
 * минимального остовного дерева (MST). Так же, как в с
 * KruskalSimulator, три алгоритма живут на одном графе
 * переключаются вкладками «Краскал / Прима / Борувка».
 *
```

```
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + под
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
 * через emitVizDemo, код в панели следует за ней).
 */
```

```
export const tickets18to20Mst: Chapter[] = [
 {
```

```
 id: "mst",
 title: "18–20. Графы. Остовное дерево: Краскал, При
 type: "html",
 description:
 "Три способа построить минимальное остовное дерев
 .",
```

```
 category: "Графы. Остовные",
 content: `
```

```
<section id="mst" class="mb-12 scroll-mt-10">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
 <span class="bg-blue-600 text-white px-4 py-1 r
```

```

 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
</thead>
<tbody class="text-slate-300">
 <tr class="border-b border-slate
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3">Рёбра
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 <td class="py-2">рёбра даны
 </tr>
 <tr class="border-b border-slate
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3">Пастёр
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
/td>
 <td class="py-2">плотный гр
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3">Все к
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 <td class="py-2">параллельны
 </tr>
</tbody>
</table>
</div>

<div class="grid grid-cols-1 xl:grid-cols-3
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
der border-indigo-500 shadow-md">
 Аналогия к билету 18 • Краск
 </div>
 <p class="text-slate-300 text-sm">С

```

ин раз, DSU спасает от дубля (find(u) == find(v))

**Код 18: Краскал + DSU (Скрыто)**

```

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код 18: Краскал + DSU (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
 #include <bits/stdc++.h>
 using namespace std;
 typedef long long ll;
 vector<pair<int,int>> edges;
 vector<int> p, rnk;

 int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
 bool unite(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false;
 if (rnk[a] < rnk[b]) swap(a, b);
 p[b] = a;
 if (rnk[a] == rnk[b]) rnk[a]++;
 return true;
 }

 int main() {
 int n, m;
 cin >> n >> m;
 edges.resize(m);
 for (int i = 0; i < m; i++) {
 int u, v, w;
 cin >> u >> v >> w;
 edges[i] = {u, v};
 }
 p.resize(n);
 rnk.resize(n, 0);
 for (int i = 0; i < n; i++) p[i] = i;

 sort(edges.begin(), edges.end()); // O(E log E)
 long long cost = 0; int taken = 0;
 for (auto [u, v] : edges) {
 if (unite(u, v)) { // не замыкает цикл
 cost += edges[u].second;
 if (++taken == n - 1) break; // V - 1 ребро
 }
 }
 cout << cost << endl;
 }
 </pre>
 </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код 20: Борувка (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
 = list(range(n))
def find(v):
 while parent[v] != v:
 parent[v] = parent[parent[v]]
 v = parent[v]
 return v

comps = n
total = 0
while comps > 1:
 best = {} # компонента
 for w, u, v in edges:
 cu, cv = find(u), find(v)
 if cu == cv:
 continue # уже внутри
 if cu not in best or w < best[cu][0]:
 best[cu] = (w, u, v)
 if cv not in best or w < best[cv][0]:
 best[cv] = (w, u, v)
 if not best:
 break # граф несвя.
 for w, u, v in best.values():
 if find(u) == find(v):
 continue # дубль: два
 parent[find(u)] = find(v)
 comps -= 1
 total += w
print(total)</pre>
 <p class="text-slate-400 text-xs">3.
 ну), значит фаз не больше $\log_2 V$.</p>
 </div>

```

```

id: "string-kmp",
title: "21. Алгоритм Кнута-Морриса-Пратта (KMP)",
type: "html",
category: "Строки",
content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i .</p>
 </div>

 <div class="bg-slate-900 border border-indigo">
 <h4 class="text-indigo-300 font-bold">
 Префикс-функция $\pi[i]$ – смотрим $S[0..i]$
 </h4>
 <p class="text-slate-300 text-sm mb-4">
 Чтобы узнать значение $\pi[i]$ –
 </p>
 <p>Шаг 1</p>
 <p>Шаг 2</p>
 <p>Шаг 3</p>
 <p>Шаг 4</p>
 <p>Шаг 5</p>
 <p>Шаг 6</p>
 <p>Шаг 7</p>
 <p>Шаг 8</p>
 <p>Шаг 9</p>
 <p>Шаг 10</p>
 <p>Шаг 11</p>
 <p>Шаг 12</p>
 <p>Шаг 13</p>
 <p>Шаг 14</p>
 <p>Шаг 15</p>
 <p>Шаг 16</p>
 <p>Шаг 17</p>
 <p>Шаг 18</p>
 <p>Шаг 19</p>
 <p>Шаг 20</p>
 <p>Шаг 21</p>
 <p>Шаг 22</p>
 <p>Шаг 23</p>
 <p>Шаг 24</p>
 <p>Шаг 25</p>
 <p>Шаг 26</p>
 <p>Шаг 27</p>
 <p>Шаг 28</p>
 <p>Шаг 29</p>
 <p>Шаг 30</p>
 <p>Шаг 31</p>
 <p>Шаг 32</p>
 <p>Шаг 33</p>
 <p>Шаг 34</p>
 <p>Шаг 35</p>
 <p>Шаг 36</p>
 <p>Шаг 37</p>
 <p>Шаг 38</p>
 <p>Шаг 39</p>
 <p>Шаг 40</p>
 <p>Шаг 41</p>
 <p>Шаг 42</p>
 <p>Шаг 43</p>
 <p>Шаг 44</p>
 <p>Шаг 45</p>
 <p>Шаг 46</p>
 <p>Шаг 47</p>
 <p>Шаг 48</p>
 <p>Шаг 49</p>
 <p>Шаг 50</p>
 <p>Шаг 51</p>
 <p>Шаг 52</p>
 <p>Шаг 53</p>
 <p>Шаг 54</p>
 <p>Шаг 55</p>
 <p>Шаг 56</p>
 <p>Шаг 57</p>
 <p>Шаг 58</p>
 <p>Шаг 59</p>
 <p>Шаг 60</p>
 <p>Шаг 61</p>
 <p>Шаг 62</p>
 <p>Шаг 63</p>
 <p>Шаг 64</p>
 <p>Шаг 65</p>
 <p>Шаг 66</p>
 <p>Шаг 67</p>
 <p>Шаг 68</p>
 <p>Шаг 69</p>
 <p>Шаг 70</p>
 <p>Шаг 71</p>
 <p>Шаг 72</p>
 <p>Шаг 73</p>
 <p>Шаг 74</p>
 <p>Шаг 75</p>
 <p>Шаг 76</p>
 <p>Шаг 77</p>
 <p>Шаг 78</p>
 <p>Шаг 79</p>
 <p>Шаг 80</p>
 <p>Шаг 81</p>
 <p>Шаг 82</p>
 <p>Шаг 83</p>
 <p>Шаг 84</p>
 <p>Шаг 85</p>
 <p>Шаг 86</p>
 <p>Шаг 87</p>
 <p>Шаг 88</p>
 <p>Шаг 89</p>
 <p>Шаг 90</p>
 <p>Шаг 91</p>
 <p>Шаг 92</p>
 <p>Шаг 93</p>
 <p>Шаг 94</p>
 <p>Шаг 95</p>
 <p>Шаг 96</p>
 <p>Шаг 97</p>
 <p>Шаг 98</p>
 <p>Шаг 99</p>
 <p>Шаг 100</p>
 <p>Шаг 101</p>
 <p>Шаг 102</p>
 <p>Шаг 103</p>
 <p>Шаг 104</p>
 <p>Шаг 105</p>
 <p>Шаг 106</p>
 <p>Шаг 107</p>
 <p>Шаг 108</p>
 <p>Шаг 109</p>
 <p>Шаг 110</p>
 <p>Шаг 111</p>
 <p>Шаг 112</p>
 <p>Шаг 113</p>
 <p>Шаг 114</p>
 <p>Шаг 115</p>
 <p>Шаг 116</p>
 <p>Шаг 117</p>
 <p>Шаг 118</p>
 <p>Шаг 119</p>
 <p>Шаг 120</p>
 <p>Шаг 121</p>
 <p>Шаг 122</p>
 <p>Шаг 123</p>
 <p>Шаг 124</p>
 <p>Шаг 125</p>
 <p>Шаг 126</p>
 <p>Шаг 127</p>
 <p>Шаг 128</p>
 <p>Шаг 129</p>
 <p>Шаг 130</p>
 <p>Шаг 131</p>
 <p>Шаг 132</p>
 <p>Шаг 133</p>
 <p>Шаг 134</p>
 <p>Шаг 135</p>
 <p>Шаг 136</p>
 <p>Шаг 137</p>
 <p>Шаг 138</p>
 <p>Шаг 139</p>
 <p>
```

```
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}
```

```
</div>
</details>
```

```
<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
p>
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Префикс
для шаблона (pattern) один раз, затем идём
пало – $j += 1$. Не совпало – откатываемся на
у заново сравнивать его не нужно. Когда j до
ждения), делаем $j = \pi[j - 1]$.
```

```
<p class="text-slate-300 text-sm mb-4">П
ждый откат j уменьшает. Значит суммарно отк
классическая амортизационная оценка, её люб
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
льшего собственного префикса, равного суффи
 <p class="text-slate-400 text-sm mb-1">
ый алгоритм.
```

```
<p class="text-slate-400 text-sm mb-1">
т онлайн (символы могут приходить потоком),
 <p class="text-slate-400 text-sm">•
5] = 4, период 6 – 4 = 2 («ab»).
```

```
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```



```
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}
```

</div>

</details>

</div>

</div>

</section>

},

{

id: "aho-corasick",

title: "23. Алгоритм Ахо-Корасик",

type: "html",

category: "Строки",

content: `

<section id="aho-corasick" class="mb-12 scroll-mt-10">

<div class="flex items-center mb-6 flex-wrap gap-3">

<span class="bg-indigo-600 text-white px-4 py-1 rounded">

<h2 class="text-2xl sm:text-3xl font-bold text-white">

</div>

<div class="space-y-8">

<div class="bg-slate-700/50 p-6 rounded-xl border-l">

<h3 class="text-xl font-bold text-blue-400 mb-4">

<div class="bg-slate-900 p-4 rounded-lg mb-6 text">

<p class="text-sm text-blue-300 italic mb-2">Осн

<p class="text-lg font-mono text-white">Бор (Tr

</div>

<p class="text-slate-300 text-sm">Позволяет найти

го один проход.</p>

</div>

<!-- Аналогии -->

<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">



```

 }
 return t[v].go[c];
}

`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description:
 "P, NP, co-NP, NP-трудные и NP-полные, PSPACE и E. Если ли задача NP-полна.",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 24
 <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-purple-400">Введение
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">Вопрос: Существует ли алгоритм, который за полиномиальное время решает задачу?
 <p class="text-xl font-mono text-white">Алгоритм, который решает задачу за время $O(n^k)$ для некоторого k называется полиномиальным. Сведение A $\leq_p$ B – преобразовать вход A в вход B за полиномиальное время.
 </p>
 </div>
 <p class="text-slate-300 text-sm mb-4">Класс сложности P – это класс задач, которые можно решить за полиномиальное время на машине Тьюринга) с выбранным бюджетом ресурсов. Вопрос: Существует ли алгоритм, который решает задачу за полиномиальное время?
 худший случай, а не про «в среднем»
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">

```



```

<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код: верификатор сертификата и про
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
 ьтонов цикл NP-полон: сертификат = порядок
def verify_hamilton(n, edges, cert):
 if sorted(cert) != list(range(n)): # каждая
 return False
 allowed = {(u, v) for u, v in edges} | {(v, u) for
 for i in range(n):
 a, b = cert[i], cert[(i + 1) % n] # замык
 if (a, b) not in allowed:
 return False
 return True

3-SAT ≤p КЛИКА: формула с k дизъюнктов → граф из 3k в
def reduce_3sat_to_clique(clauses):
 # вершина = {номер дизъюнкта, литерал}; ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)
 edges = []
 for i in range(len(verts)):
 for j in range(i + 1, len(verts)):
 ci, li = verts[i]; cj, lj = verts[j]
 if ci == cj: # из од
 continue
 if li.lstrip("-") == lj.lstrip("-") and (li
 continue # x и n
 edges.append((i, j))
 return verts, edges, len(clauses) # клика

Проверка задачи из P для контраста: двудольность = 2-р
def is_bipartite(n, graph):
 from collections import deque
 col = [-1] * n

```

```

 </div>
 </div>
</section>`
 }
};

```

ФАЙЛ 30/111: src/data/tickets/ticket6 13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 576 | РАЗМ:

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление графов, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 /* База представлений */
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">

```

```

 Детектирование циклов (через DFS)

</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; box-shadow: 5px 5px 0px #008080;">
 <div class="absolute -top-3 left-4 right-4 bottom-4" style="border: 1px solid #008080; border-radius: 5px; padding: 10px;">
 BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-0">
 Что делает: Распространение по всем соседям. Работает через Очередь (FIFO). Гарантирует минимальное расстояние от источника.
 </p>
 <ul class="text-xs text-emerald-300" style="list-style-type: none; padding-left: 0;">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Проверка двудольности графа
 Алгоритм Ахо-Корасик (построение автомата)
 Двунаправленный BFS для поиска кратчайшего пути (в неориентированном графе).

 </div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #008080; border-radius: 10px; margin-top: 10px;">
 <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #008080; border-radius: 5px; padding: 5px;">
 Код обходов C++ (Основа)
 </summary>
 <div class="grid grid-cols-1 md:grid-cols-2" style="padding: 10px;">
 <pre class="bg-slate-950 p-3 rounded" style="font-family: monospace; font-size: 0.9em; color: #e0e0e0;">
// DFS (Рекурсия с возвратом)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
 vis[v] = true;

```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-500">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-2xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
включая внешнюю)</p>
</div>
```

```
<div class="space-y-4 text-slate-300 text-sm">
 <p>Главные неравенства на число рёбер</p>
 <ul class="list-disc list-inside space-y-2">
 Для простого планарного графа
 <i>Доказательство:</i> Каждая грань
$2E \geq 4F \Rightarrow F \leq E/2$. Подставляя в формулу Эйлера: $2V - 4$

 Для простого двудольного планарного графа
$2E \geq 4F \Rightarrow F \leq E/2$
 <i>Доказательство:</i> В двудольном графе
 $2E \geq 4F \Rightarrow F \leq E/2$


```

```
 <div class="bg-slate-950 p-4 rounded-lg">
 <p class="text-amber-300 font-bold">
 <p class="text-slate-300 text-xs leading-5">
 1. K_5 (полный граф на 5 вершинах)
 $10 \leq 3(5) - 6 = 9$ – формально противоречие
 2. $K_{3,3}$ (полный двудольный 3×3 граф)
 $9 \leq 12$ – формально противоречие
 $2V - 4$: $9 \leq 12$
 </p>
 </div>
```

```
 <p>Теорема Куратовского (1930) / Вагнера
 фа, являющегося подразделением (subdivision)
 </p>
</div>
```

```

 "Компоненты связности: обходами DFS/BFS за $O(V + E)$
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 рой вершин есть путь. Внешний цикл «для каж
 » даёт и разметку, и количество компонент.</p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Два р
 ной запускаем DFS или BFS и присваиваем все
 mp[v] = c. Счётчик c увеличивается р
 Время $O(V + E)$, память $O(V)$. Чере
 каждое ребро делаем union(u, v), в конце чи
 ту и после каждого добавления спрашиваю

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm">Т
 ались серые (неисследованные) зоны? Прыгаеш
 и компонент. Краска — это comp[v], прыжок ч
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 border border-emerald-500 shadow-md">

```

```

 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Топологическая сортировка ДАГ и детект",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
екая сортировка – это расписание, которое
р>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
получить опыт, нужна работа". Алгоритм выяв

```



```

 return;
 }
}
color[v] = 2; // Вышли – вершина чёрная
ans.push_back(v); // Сохраняем в момент ВЫХОДА (pos
}

bool topological_sort(int n) {
 fill(color, color + n + 1, 0);
 has_cycle = false;
 ans.clear();
 for (int i = 1; i <= n; ++i) {
 if (color[i] == 0) dfs(i);
 }
 if (has_cycle) return false; // Граф содержит цикл!
 reverse(ans.begin(), ans.end()); // Разворачиваем р
 return true;
}

```

```

 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая-Жару",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 10. Графы. Компоненты сильной связности (SCC)
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Графы. Продвинутое</h2>
 </div>

 <div class="space-y-8">

```

```

{
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых увеличивает ч",
 category: "Графы. Продвинутое",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="bg-slate-950 p-4 rounded-lg border">
 <p class="text-amber-300 font-bold text">
 <p class="text-slate-300 text-xs leading">
 Проверка <span class="font-mono text-
тиграфе между u и v может быть 2 параллельны
азуют цикл длиной 2.

 Решение: передавать в DFS не v
rald-300">p_edge_id (индекс ребра, по
 </p>
 </div>

 <details class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">
 Код поиска мостов C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-rose-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.
 <div class="p-3 bg-slate-950 rounded">

 <p class="text-xl font-bold tex">
 <p class="text-xs text-slate-400">
 </p>
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Особый случай: Корень дерева DFS
мого ребёнка в дереве DFS.
 </p>

 <details class="bg-slate-900/40 rounded-lg border"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код поиска точек сочленения (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
void dfs_cutpoints(int v, int p = -1) {
 tin[v] = low[v] = ++timer;
 int children = 0;
 for (int to : adj[v]) {
 if (to == p) continue;
 if (tin[to]) {
 low[v] = min(low[v], tin[to]);
 } else {

```

```

<div class="grid grid-cols-1 md:grid-cols-2"
 <!-- Неориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg"
 <p class="text-indigo-300 font-bold"
 <ul class="list-disc list-inside sp
 Эйлеров цикл: связан
-emerald-300">deg[v] % 2 == 0.
 Эйлеров путь: связан
o 2 – путь начинается в одной нечётной и за

 </div>

 <!-- Ориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg"
 <p class="text-indigo-300 font-bold"
 <ul class="list-disc list-inside sp
 Эйлеров цикл: слабос
ald-300">in[v] == out[v].
 Эйлеров путь: слабос
s] = 1 (старт), у одной <span class=
an class="font-mono text-emerald-300">in[v]

 </div>
 </div>

<h3 class="text-lg font-bold text-white mt-3"
<p class="text-slate-300 text-sm mb-3">Исполн
 → рекурсивный переход. При возврате (на вых
ты: <span class="font-mono text-emerald-300"

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код построения Эйлерова цикла C++
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded

```

```

 label: string;
}

interface Edge {
 u: number;
 v: number;
}

const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

```

recordStep("Начало DFS (Тарьян) для поиска точек со

```
const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], current
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], current

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}
 v,
);

 if (currentLow[to] >= currentTin[v] && p !==
 ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}
 v,
);
 }
 }
 }
}
```

```

 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}

```

```

const isCurrent = currentStep.currentNode
const isVisited = currentStep.visited.has
const isAp = currentStep.ap.has(node.id);

const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
const r = isAp ? 24 : 20;

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 />
 </g>
)

```



```
</div>
```

```
<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-400">
 Статус алгоритма
 </h4>
```

```
<div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-16">
 {currentStep.desc}
 </p>
</div>
```

```
<div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 font-medium">
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Точка сочленения
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Посещена
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500">
 Текущая
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border">
 Мост
 </div>
 </div>
 </div>
 </div>
 </div>
```

```
<div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
```

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from './VizRuntime';
import { LiveDistTable } from './LiveDistTable';
```

```
interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← → и листать вручную.";
```

```
export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];
```

```
const getEdges = (cycle: boolean): Edge[] => {
```

```
];
```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999',
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v} ${
 updated ? 'Релаксация успешна! dist[${edge.v}] = ' +
 dist[edge.v] : ''
 }`,
 activeCodeLine: updated ? 6 : 5,
 });
 }
 }
}
```

```

 setIsPlaying(false);
 }, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарплата...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node.id, liveDist[node.id]])),
 log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} -> ${liveV}, вес ${liveW}, расстояние ${liveDist[liveU]} -> ${liveDist[liveV]}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white rounded-md">Шаг {currentStepIndex + 1}</div>

 </div>
 </div>
 </div>
);

```

```

<div className="absolute top-3 left-3 bg-blue
 ld shadow-sm">
 Итерация {currentStep.iteration} (Шаг {currentStep.step})
</div>

```

```

{currentStep.hasNegativeCycle && (
 <div className="absolute top-3 right-3 bg-red
 animate-pulse shadow-lg shadow-rose-600/50">
 ⚠ Отрицательный цикл!
 </div>
)}

```

```

<svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

```

```

{edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking = currentStep.checkingEdges === idx;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

```

```

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; }
 else if (currentStep.hasNegativeCycle && isNeg) { strokeColor = '#e11d48'; }

```

```

 return (
 <g key={idx}>

```

```

<h4 className="text-lg font-bold text-emerald-500">
<div className="space-y-2 text-sm font-mono text-emerald-500">
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2">
 <div className="flex flex-wrap gap-2">
 {u}
 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === edge.id
 return (
 <span key={idx} className={`px-2 py-1 rounded-sm ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 }`}>{edge.v} ({edge.w})
)
 })}
 </div>
 </div>
 })}
</div>
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-300">Таблица расстояний
 <div className="grid grid-cols-4 gap-2 text-sm font-mono text-rose-300">
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge === n.id
 return (
 <div key={n.id} className={`p-2 rounded-sm ${
 isTgt ? 'border-slate-700/60 bg-slate-800/40' : 'bg-rose-900/40'
 }`}>
 <div className="text-xs text-slate-400">{n.id}</div>
 <div className={`font-mono font-bold text-rose-300`}>{d}</div>
 </div>
)
 })}
 </div>
 </div>
 </div>
 <div className="w-full lg:w-1/2">
 <div>
 <h4 className="text-lg font-bold text-rose-300">График
 <div className="flex flex-wrap gap-2">
 {adjList[currentStep.start].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === edge.id
 return (
 <div key={idx} className="flex flex-wrap gap-2">
 <div className="flex flex-wrap gap-2">
 {currentStep.start}
 {adjList[currentStep.start].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === edge.id
 return (
 <span key={idx} className={`px-2 py-1 rounded-sm ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 }`}>{edge.v} ({edge.w})
)
 })}
 </div>
 </div>
)
 })}
 </div>
 </div>
 </div>
</div>

```

```

 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];

const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0).",
 });
};

```

```

 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.`
 });
 }
}
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

```





```

 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration-300"
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-sm"
 >
 {n.label}
 </text>
 </g>
);
}}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4">
 bfs(graph, start)
 War {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visited.add(start);' },
 { line: 3, code: '}' },
 { line: 4, code: 'while not queue.isEmpty():' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node]:' },
 { line: 7, code: ' if neighbor not in visited:' },
 { line: 8, code: ' visited.add(neighbor)' },
 { line: 9, code: ' queue.push(neighbor)' },

```

```
 });
}
```

---

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-freakazoid.
import bellmanImg from '../assets/bellman-ford-batman.jp
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string;
 dijkstra: {
 src: dijkstraImg,
 alt: 'Декстер Дуглас и Фриказоид',
 caption: '⚡ Декстер Дуглас и Фриказоид – волна виру
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Бэтмен на машине Генри Форда',
 caption: ' Бэтмен на машине Форда – n-1 ночей патр
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Fortnite Все против Всех (Флойд–Уоршелл)',
 caption: ' Fortnite: Все против Всех – Флойд-Уорше
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```
export default function ChapterImage({ vizType }: { vizType
```

```

return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
}

```

```

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2">
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-slate-100
 transition-all"
 >
 <div className="flex items-center gap-1.5 text-sm">

```

```
}

/**
 * Блиц по прочитанной теме: 3–5 коротких вопросов с не
 *
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
 * компонент-викторина существовали, но не были подклю
 * контент был, а добраться до него было нельзя. Теперь
 * демонстрации: прочитал → посмотрел, как работает → п
 *
 * В текстовой версии (`?lite=1`) и в PDF те же вопросы
 * с ответами и разбором – там интерактив не нужен.
 */
export const ChapterQuiz: React.FC<Props> = ({ chapterId
 const list = quizzes[chapterId];
 const [idx, setIdx] = useState(0);
 const [picked, setPicked] = useState<number | null>(n
 const [score, setScore] = useState(0);
 const [finished, setFinished] = useState(false);

 // другая тема – другой блиц с нуля
 useEffect(() => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);

 if (!list || list.length === 0) return null;

 const q = list[idx];
 const answered = picked !== null;
 const correct = answered && picked === q.correctIndex

 const pick = (i: number) => {
 if (answered) return;
 setPicked(i);
 if (i === q.correctIndex) setScore((s) => s + 1);
```

```

 : "Стоит вернуться к тексту страницы:
 </p>
 <p className="text-2xl font-bold text-white
 {score} / {list.length}
 </p>
 <button
 type="button"
 onClick={restart}
 className="inline-flex items-center gap-1
 font-bold text-indigo-200 hover:bg-indigo-500/20
 >
 <RotateCcw className="w-3.5 h-3.5" /> Проп
 </button>
 </div>
) : (
 <>
 <p className="text-sm font-semibold text-wh
 <div className="space-y-2">
 {q.options.map((option, i) => {
 const isRight = i === q.correctIndex;
 const chosen = picked === i;
 const tone = !answered
 ? "border-slate-600 bg-slate-800/70 t
 : isRight
 ? "border-emerald-500 bg-emerald-500
 : chosen
 ? "border-rose-500 bg-rose-500/15
 : "border-slate-700 bg-slate-800/
 return (
 <button
 key={i}
 type="button"
 onClick={() => pick(i)}
 disabled={answered}
 className={`w-full text-left flex i
 colors disabled:cursor-default ${tone}`}
 >
 {answered && isRight && <CheckCirc

```

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, Minimize2 } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
import ChapterImage from "../ChapterImage";
import MnemonicCards from "../MnemonicCards";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлено название */
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter, prev, next, index, total, onGo, onOpenSimulator, onOpenCompiler }) => {
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
```

```

 });
 };
 const restore = () => {
 each().forEach((d) => {
 if (d.dataset.wasOpen === "0") d.open = false;
 delete d.dataset.wasOpen;
 });
 };
 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);
 return () => {
 window.removeEventListener("beforeprint", openAll);
 window.removeEventListener("afterprint", restore);
 };
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {

```



```

<button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small border-1px
 300 hover:text-white hover:border-emerald-500"
>
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 {saving === "done" ? "Файл сохранён" : "Скачать"}
</button>
<button
 onClick={() => window.print()}
 title="Распечатать или сохранить эту тему в PDF-формате. Без шапки,
 содержания и панели комментариев"
 className="flex items-center gap-1.5 text-x-small border-1px
 300 hover:text-white hover:border-indigo-500"
>
 <Printer className="w-3.5 h-3.5" /> PDF этой темы
</button>
<a
 href={`?lite=1&topic=${encodeURIComponent(topic)}`}
 target="_blank"
 rel="noreferrer"
 title="Текстовая версия темы без навигации, комментариев и
 панели"
 className="flex items-center gap-1.5 text-x-small border-1px
 300 hover:text-white hover:border-sky-500"
>
 <FileText className="w-3.5 h-3.5" /> Текстовая версия

 </div>

```

```

 className="flex items-center gap-1.5
 slate-300 hover:text-white hover:border-ind
 >
 <Maximize2 className="w-3.5 h-3.5" />
 </button>
 {onOpenCompiler && {
 <button
 onClick={onOpenCompiler}
 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5
 t-emerald-400 hover:text-white hover:border
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
 }}
 </div>
</div>
{viz.hint && <p className="text-xs text-sla
<VizChapterContext.Provider value={chapter.
 <VizLiveStatus demoTitle={viz.title} clas
 <viz.Component />
</VizChapterContext.Provider>
</section>
)}

{/* прочитал → посмотрел, как работает → провер
<ChapterQuiz chapterId={chapter.id} />

 <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 />

```

```

 { id: "hierarchy", label: "1 · Иерархия классов", icon: <
 { id: "reduce", label: "2 · Сведение A ≤p B", icon: <
 { id: "verify", label: "3 · Верификатор (NP)", icon: <
 { id: "growth", label: "4 · Рост по n и ops", icon: <
];

```

```

/* ----- Иерархия -----

```

```

const CLASSES = [
 { id: "EXP", label: "EXP", sub: "2^{n^k}", color: "#0000FF",
 (кванторные формулы)" } },
 { id: "PSPACE", label: "PSPACE", sub: "полиномиальная
 с полной информацией" } },
 { id: "NP", label: "NP", sub: "сертификат проверяется
 AT", "гамильтонов цикл", "раскраска в 3 цвета", "кли
 { id: "P", label: "P", sub: "решается за O(n^k)", color: "#00FF00",
 ид", "MST", "2-SAT, двудольность" } },
];

```

```

const NPHARD_NOTE = "NP-трудные — всё, к чему сводится
 амильтонов цикл. Проблема остановки NP-трудна, но в NP

```

```

/* ----- Сведение -----

```

```

const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма
 — то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x — конкретный
 "x"], active: 1 },
 { title: "Строим f(x) = y за полиномиальное время", note: "
 шинами из разных дизъюнктов, если литералы не противоречивы",
 ve: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм
 — но допустим, что он есть.", boxes: ["A", "x", "f(x)"], active: 3 },
 { title: "Переводим ответ обратно: A(x) = B(f(x))", note: "
 ⇒ формула выполнима. Полиномиальное время сохранилось",
 5 },
 { title: "Вывод: B не проще A", note: "Раз A ≤p B и A не ≤p B,

```

```

 { id: "n", label: "n", fn: (n: number) => n, color: "#000000" },
 { id: "n log n", label: "n · log2 n", fn: (n: number) => n * Math.log2(n), color: "#000000" },
 { id: "n²", label: "n²", fn: (n: number) => n * n, color: "#000000" },
 { id: "n³", label: "n³", fn: (n: number) => n * n * n, color: "#000000" },
 { id: "2n", label: "2n", fn: (n: number) => Math.pow(2, n), color: "#000000" },
];

```

```

export const ComplexityViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("hierarchy");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [samples, setSamples] = useState<Sample[]>([]);

```

```

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

```

```

 const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 0;
 useEffect(() => {
 setStep(0);
 setPlaying(false);
 }, [tab]);

```

```

 useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 return () => window.clearInterval(t);
 }, [maxStep]);

```

```

 for (let i = 0; i < xs.length; i++) {
 num += (xs[i] - mx) * (ys[i] - my);
 den += (xs[i] - mx) ** 2;
 }
 const slope = den === 0 ? 0 : num / den;
 // экспонента на лог-лог растёт неограниченно: сравн
 const half = Math.floor(pts.length / 2);
 const slopeTail = (() => {
 const q = pts.slice(half);
 if (q.length < 2) return slope;
 const lx = q.map((p) => Math.log(p.n));
 const ly = q.map((p) => Math.log(p.ops));
 const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
 const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
 let nn = 0;
 let dd = 0;
 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 return dd === 0 ? slope : nn / dd;
 })();
 const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e0b",
 : slope <= 1.25
 ? { name: "≈ линейный, O(n)", color: "#10b981",
 : slope <= 1.8
 ? { name: "≈ O(n log n)", color: "#06b6d4", hint: "log",
 : slope <= 2.4
 ? { name: "≈ квадратичный, O(n²)", color: "#f59e0b",
 : slope <= 3.4
 ? { name: "≈ кубический, O(n³)", color: "#fb9238",
 : { name: "полином высокой степени", color: "#fb9238"
 return { slope, slopeTail, ...cls };
 }, [samples]);

```

```

 /></button>
 <button onClick={() => setPlaying((p) => !p)}
 text-xs font-bold hover:bg-purple-500">
 {playing ? <Pause className="w-4 h-4" /> : <Play />}
 </button>
 <button onClick={() => { setPlaying(false); setVolume(0.2); }}
 text-sm rounded-lg bg-slate-900 border border-slate-800
 e="w-4 h-4" /></button>
 {volume}
 </div>
)}
 }

```

```

 { /* ----- ИЕРАРХИЯ ----- */
 {tab === "hierarchy" && (
 <div className="space-y-4">
 <div className="bg-slate-950 rounded-xl p-3 border border-slate-800">
 <svg viewBox="0 0 500 380" className="w-full h-380">
 {CLASSES.map((c) => {
 const on = liveCls ? liveCls.toUpperCase().startsWith(c.id) : false;
 return (
 <g key={c.id}>
 <ellipse cx={c.x} cy={c.y} rx={c.rx} ry={c.ry}
 {on ? 3 : 1.5} strokeDasharray={on ? undefined : [4, 4]}
 stroke="white" />
 <text x={c.x} y={c.y - c.ry + 18} textAnchor="middle">{c.name}</text>
 <text x={c.x} y={c.y - c.ry + 32} textAnchor="middle">{c.tasks}</text>
 </g>
);
 })}
 </svg>
 { /* примеры задач – внутри своих овалов */
 CLASSES.map((c) =>
 c.tasks.map((t, i) => (
 <text key={t} x={c.x} y={c.y + (c.id === 'root' ? 40 : 10) + i * 20}
 textAnchor="middle" fill="#e2e8f0" fontSize="8">
 {t}
 </text>
))
)
 }
 </div>
 </div>
)}
 { /* NP-полные = пересечение */

```

```

 </div>
 </div>
 <div className="bg-slate-900/70 border border-
 <p className="text-sm font-bold text-white l
 text-purple-400" />{reduceFrame.title}</p>
 <p className="text-xs text-slate-400 leadin
 </div>
 </div>
 })
}

{/* ----- ВЕРИФИКАТОР -----
{tab === "verify" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <p className="text-[10px] uppercase tracking
 <div className="flex flex-wrap gap-2 mb-3">
 {CERT.map((v, i) => {
 <span key={i} className={"font-mono tex
 ld-600/50 text-emerald-300" : "bg-slate-900
 x{i + 1} = {liveCert && typeof liveCe

)}}
 {liveCert && <span className="font-mono t
 le-950/40">cert из кода}
 </div>
 <div className="space-y-2">
 {CLAUSES.map((cl, i) => {
 const checked = liveK !== null ? i < li
 const current = liveK !== null ? i ===
 const st = VERIFY_STEPS[i];
 return (
 <div
 key={i}
 className={"flex items-center gap-3
 amber-950/20" : checked ? "border-
 slate-800 bg-slate-900/40"}>
 >
 <span className="text-[10px] font-b

```

```

<button onClick={() => setSamples([])} className="text-slate-500 text-
0 rounded-lg px-2 py-1">очистить график</button>
</div>
<svg viewBox="0 0 500 240" className="w-full h-40">
 {/* оси в лог-лог */}
 <line x1={44} y1={200} x2={486} y2={200} />
 <line x1={44} y1={14} x2={44} y2={200} strokeDash={[]} />
 <text x={265} y={222} textAnchor="middle">log</text>
 <text x={16} y={110} fill="#64748b" fontStyle="italic">log</text>
 {GROWTH_LINES.map((g) => {
 const pts: string[] = [];
 for (let i = 0; i <= 20; i++) {
 const n = 1 + i * 0.8;
 const v = g.fn(n);
 const x = 44 + (Math.log(n) / Math.log(2)) * 442;
 const y = 200 - (Math.log(Math.max(1, v)) / Math.log(2)) * 186;
 pts.push(`${x.toFixed(1)},${Math.max(1, v).toFixed(1)}`);
 }
 return <polyline key={g.id} points={pts} />;
 })}
 {samples.map((s) => {
 const x = 44 + (Math.log(Math.max(1, s.n)) / Math.log(2)) * 442;
 const y = 200 - (Math.log(Math.max(1, s.v)) / Math.log(2)) * 186;
 return <circle key={`_${s.n}_`} cx={x} cy={y} r={10} />;
 })}
 {GROWTH_LINES.map((g, i) => (
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + (i % 3) * 18} width={50} height={10} />
 <text x={343 + (i % 3) * 58} y={24 + (i % 3) * 18} text="Growth" />
 </g>
))}
</svg>
{samples.length === 0 && (
 <p className="text-xs text-slate-500 text-center">Пусто. Запусти в компиляторе код с переменной ops</p>
 <code>ops</code> — например,

```



```

 </div>
 </div>
);
};

export default ComplexityViz;

```

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-";

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },

```

];

```
const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:4,3:5}, stack: [0,1,6,2,3],
 activeEdge: [2,3], backEdge: null,
 log: "Из C идём в D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:4,3:5}, stack: [0,1,6,2,3],
 activeEdge: null, backEdge: null,
 log: "У D нет соседей (кроме C). Возврат. low[D]=5 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:4,3:5}, stack: [0,1,6,2,3],
 activeEdge: null, backEdge: null,
 log: "У C нет соседей (кроме B). Возврат. low[C]=4 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:4,3:5}, stack: [0,1,6,2,3],
 activeEdge: null, backEdge: null,
 log: "У B нет соседей (кроме A). Возврат. low[B]=2 : ",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:4,3:5}, stack: [0,1,6,2,3],
 activeEdge: null, backEdge: null,
 log: "У A нет соседей (кроме B). Возврат. low[A]=1 : ",
 activeCodeLine: [13,14,15]
 },
];
```

```

 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост",
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм закончен.",
 activeCodeLine: [11,13,16,17,18]
},
{
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм закончен.",
 activeCodeLine: []
}

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <div className="flex items-center gap-1 bg-slate-900 p-1.5">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
 disabled={dfsIdx === 0}
 aria-label="Шаг назад" title="Шаг назад"
 className="p-1.5 hover:bg-slate-700 rounded">
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${
 dfsAuto ? "bg-rose-600 text-white" : "bg-slate-900 text-white"
 }`}
 title={dfsAuto ? "Пауза" : "Включить авто DFS"}>
 {dfsAuto ? "<>" : "▶"} <Pause className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1); }}
 disabled={dfsIdx === DFS_STEPS.length-1}
 aria-label="Шаг вперёд" title="Шаг вперёд"
 className="p-1.5 hover:bg-slate-700 rounded">
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
 aria-label="Сбросить DFS" title="Сбросить DFS"
 className="p-1.5 hover:bg-slate-700 rounded">
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
 </div>

 <div className="grid grid-cols-1 md:grid-cols-5 gap-4">
 <div className="md:col-span-3 bg-slate-950 rounded p-4">
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]"
 style={{ backgroundSize: '4px 4px' }}>
 </div>
 <svg className="w-full h-[280px] z-10 relative"
 style={{ background: 'linear-gradient(to right, transparent 49%, #000000 49% 51%, #000000 51% 53%, transparent 53%)' }}>
 {GRAPH_EDGES.map(e => {

```

```

 {vis && (
 <
 <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor=
} l:{step.low[n.id]||"?"}</text>
 </>
)}
 </g>;
 }}}
</svg>
<div className="flex items-center justify-bet
 ||ar {dfsIdx+1}/{DFS_STEPS.length}</sp
 <div className="flex-1 mx-2 bg-slate-950 h-
 <div className="bg-indigo-600 h-full tran
 </div>
 </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg borde
 <div className="bg-slate-900 px-3 py-1.5 te
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono lea
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includ
 return <div key={i}
 className={`px-1 transition-all dur
 hl ? "bg-indigo-600/20 text-indig
 }`}>
 <span className="text-slate-600 mr-1
 {cl.line || ' '}
 </div>;
 })}
 </div>

```

```
 </div>
);
}
```

---

ФАЙЛ 40/111: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';
```

```
interface Node { id: string; x: number; y: number; name: string; }
```

```
interface Edge { u: string; v: string; w: number; }
```

```
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
и листать вручную.";
```

```
export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
];
```

```
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);
```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu
```

```
simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пи
 activeCodeLine: 2,
});
```

```
for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}
```

```
if (!u) break;
```

```
visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited }
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
});
```

```
const neighbors = edges.filter(e => e.u === u);
```

```

 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(var
const compilerLinked = liveU !== null || liveV !== nu
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: l
 distances: Object.fromEntries(nodes.map((node)
 previous: Object.fromEntries(nodes.map((node)
 visited: Object.fromEntries(nodes.map((node) =>
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl bor
 /* Шапка симулятора */
 <div className="flex flex-wrap items-center just
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-whit


```





```

 const dist = currentStep.distances[node.id]

 return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVis
 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-
 />
 <text x={node.x} y={node.y + 5} fill=
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill=
"middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill=
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }
 }
 </svg>
 <div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
 </div>
</div>

 /* Список смежности */
 <div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.current

```

```

<h4 className="text-lg font-bold text-rose-900">
<div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-900">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред.)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-b border-rose-900 ${
 isCurr ? 'bg-indigo-950/60 font-medium' : ''
 }`}>
 <td className="py-2.5 px-3 flex items-center">
 <span className={`w-2 h-2 rounded-full ${
 vis ? 'bg-emerald-500' : 'bg-gray-300'
 }`} style={n.isLeaf ? {display: 'block', width: 10px, height: 10px} : {}}>
 {n.name}

 </td>
 <td className="py-2.5 px-3 font-mono">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev || '-'}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
</div>
</div>

```

```
interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в`
```

```

 const res2 = simulateFib(n - 2);
 memoMap[n] = res1 + res2;

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
 }

 const finalRes = simulateFib(targetN);

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {

```

```

 </p>
 </div>

 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center gap-2 bg-slate-100 p-2" >
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
 </div>

 <div className="flex items-center bg-slate-950 p-2" >
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-2`}
 >

```

```

 }` }
 >

 <span className={`text-xl md:text-2xl font-medium`} {
 {isCached ? val : '0'}
 }>

 {num <= 2 &&
 </div>
);
 }}}
</div>

{currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-200">
 <div className="text-sm font-medium text-slate-700">

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-100 p-2">
 Текущий N: <strong className="text-emerald-500">{currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-200">
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3 rounded-md ${
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-white'
 : 'border-transparent text-slate-400 hover:text-slate-500'
 }`}
 >
 <Eye className="w-4 h-4" />
 Степ-бай-степ состояние
 </button>

```

```

 <pre className="p-6 font-mono text-sm space-x-4" style={{
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2 ${
 isActive
 ? 'bg-emerald-600/20 border-l-4 border-emerald-600'
 : 'text-slate-300 hover:bg-slate-800'
 }`}
 >
 {lineNum}
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-600" style={{
 {
 {currentStep?.desc}
 }
 </div>
 </div>
)}
</div>

```

```

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800" style={{
 <div className="flex items-center justify-between" style={{
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 }>
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden" style={{
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }
 ></div>
 }>
 </div>

```



```

* Две части:
* 1. игра «пройди все рёбра» кликами (Эйлер не прощает
* 2. критерий: степени вершин и число нечётных (0 → ц
*
* Синхронизация с Python-компилятором (PAGE_SYNC["eule
* path – маршрут (массив вершин) рисуется поверх граф
* пройдены, deg/odd – степени и нечётные вершины, las
* step – номер шага.
*/
export function EulerSimulator() {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Клики на вершину
 const [c1Done, setC1Done] = useState(false);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "walk");
 }, [chapterId]);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери

```

```

 const num = typeof val === "number" ? val : v
 return num ?? DEG[n.id];
 })
 : null;
const oddRow = vizArray(vars.odd);
const odd = oddRow ? oddRow.map((x) => (typeof x ===
const last = vizNumber(vars.last) ?? vizNumber(vars
const step = vizNumber(vars.step) ?? vizNumber(vars
if (path.length === 0 && usedFlags.length === 0 &&
// рёбра маршрута: пары соседних вершин
const keys: string[] = [];
for (let i = 0; i + 1 < path.length; i++) keys.push
usedFlags.forEach((f, i) => {
 if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i].
});
return {
 path,
 keys: Array.from(new Set(keys)),
 deg,
 odd,
 last: last !== null && last >= 0 && last < C1_NODES
 step,
 complete: keys.length >= C1_EDGES.length,
};
}, [vars]));

const path = live && live.path.length ? live.path : c
const visitedKeys = live ? live.keys : c1VisitedEdges
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[
const done = live ? live.complete : c1Done && visited
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёб
 шин ${odd.length} → это ${odd.length === 0 ? "ц
 : `Маршрут из кода: ${live.path.length ? live.pat
 ys.length}/${C1_EDGES.length}`

```

```

 <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={isLastEdge ? "#a5b4fc" : used ?
 strokeWidth={isLastEdge ? 5 : used ? 4
 className="transition-all duration-300"
 {order !== null && (
 <text x={({u.x + v.x} / 2} y={({u.y + v.y
 }</text>
)}
 </g>;
)))
 {C1_NODES.map(n => {
 const isLast = lastId === n.id;
 const visited = path.includes(n.id);
 const isOdd = odd.includes(n.id);
 return <g key={n.id} className="cursor-poin
 {isLast && <circle cx={n.x} cy={n.y} r={1
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y + 4} textAnchor="mid
 label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="mi
 one">deg {deg[n.id]}{isOdd ? " • нечёт" : "
 </g>;
)))
</svg>
<div className="absolute top-2 left-2 bg-slate-
 Peбëp: {visitedKeys.length}/{C1_EDGES.length}
</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 te
 : c1Done && !live
 ? "bg-rose-950/40 border-rose-500/30 text-ros

```

-----  
}

-----  
ФАЙЛ 43/111: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

-----  
import { useState, useEffect } from 'react';  
import { liveMatrix, pickLiveVar, useVizRuntime, vizNums

```
interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
ва листать вручную.";
```

```
export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольн
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
```

```
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];
```

```

log: ' Инициализация матрицы прямых путей, ∞ (999)
activeCodeLine: 2,
});

```

```

const N = 7;
for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром. узлы
activeCodeLine: 3,
 });

```

```

for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 && viaVal < oldVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]),
 log: ` Улучшение пути ${i}→${j} через пром. узлы
 ∞ `,
 activeCodeLine: 7,
 });
 }
 }
}
}

```

```

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ' Все пары отработаны. Алгоритм Флойда завершён
 activeCodeLine: 8,
});

```

```

setSteps(simulationSteps);

```

```

 ? `Демо-схема графа нарисована для ${demoSize} вершин и
 ерности; схема графа остаётся демонстрационной.`
 : null;
const labelOf = (idx: number) => nodeNames[idx]?.split(' ')?.[0]
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: shownMatrix,
 updated: !!runtime?.changed?.some(({name}) => name !== nodeNames[k][i][j]),
 log: `Компилятор: k=${liveK ?? "-"}, i=${liveI ?? "-"}, j=${liveJ ?? "-"}: Подсветка взята из реальных переносов
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Полная демонстрация
 </p>
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false); TEXT : "Вернуться к первому шагу"} className="text-sm font-medium cursor-not-allowed">Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying)} className="text-sm font-medium cursor-not-allowed ${isPlaying ? 'bg-amber-600' : 'bg-emerald-600'}">
 Полная демонстрацию
 </button>
 </div>
 </div>
)

```

```

 <circle cx={({uNode.x + vNode.x) / 2}
isVia ? '#818cf8' : '#64748b'} strokeWidth=1
 <text x={({uNode.x + vNode.x) / 2} y={
8fafc'} fontSize="10" fontWeight="bold" text="
 </g>
);
 });
 }}}
 {nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a8a2a2'; }
 else if (isI) { fill = '#b45309'; stroke = '#a8a2a2'; }
 else if (isJ) { fill = '#047857'; stroke = '#a8a2a2'; }
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isK ? 4 : 2} className="transition-colors duration-200" />
 <text x={node.x} y={node.y + 5} fill="white" />
 <text x={node.x} y={node.y + 32} fill="white" />
 </text>
 </g>
);
 })}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2 text-sm font-mono">

```

```

 </p>
) : null}
 <div className="overflow-x-auto">
 <table className="w-full text-center border">
 <thead><tr className="bg-rose-950/40 text">
 <th className="p-2 border-r border-rose">
 {Array.from({ length: cols }, (_, idx) => {
 <th key={idx} className={`p-2 border-
labelOf(idx)} {idx}</th>
 })}
 </tr></thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => {
 <tr key={i} className="border-b borde">
 <td className={`p-2 bg-rose-950/40
tStep.k === i ? 'text-amber-400' : ''}`>
 {labelOf(i)} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i
 const isVia = currentStep.k !== -
tep.j === j));
 let bg = 'bg-slate-900/40 text-sl
 if (isUpd) bg = 'bg-emerald-900/8
 else if (isVia) bg = 'bg-indigo-9
 else if (currentStep.i === -1 &&
 bg = 'bg-indigo-950/60 text-ind
 else if (i === j) bg = 'bg-slate-
 const shown = val === null ? '.'
 return (<td key={j} className={`p
 }}}
 </tr>
 }}}
 </tbody>
 </table>
 </div>
 </div>

```



```

*
* Синхронизация с Python-компилятором (PAGE_SYNC["graph"])
* - comp / color / used — массив разметки: вершины к
* - parent / p — массив DSU: рисуем множества
* - v — текущая вершина (пульсирует)
* - stack / dq / q — фронт обхода (подсвечивает)
* - comps / c — счётчик компонент (большая)
* - i / e / step — номер шага: двигает по кадру
*/

```

```

type Mode = "dfs" | "dsu";

```

```

interface Node {
 id: number;
 label: string;
 x: number;
 y: number;
}

```

/\*\* Граф из 10 вершин и трёх компонент — ровно как в те

```

const NODES: Node[] = [
 { id: 0, label: "0", x: 90, y: 80 },
 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

```

```

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

```

```

 цвет ${c} и кладём в стек.`);
 while (stack.length) {
 const v = stack[stack.length - 1];
 const next = ADJ[v].find((to) => comp[to] === 0);
 if (next === undefined) {
 stack.pop();
 push(v, stack, c, `Из ${v} идти некуда – возвра
 ;
 continue;
 }
 comp[next] = c;
 stack.push(next);
 push(next, stack, c, `${v} → ${next}: красим в цв
 }, кладём ${next} в стек.` , [v, next]);
 }
 push(null, [], c, `Компонента ${c} готова`, `Стек п
 } вершин. Ищем следующую серую вершину.`);
}
push(null, [], c, "Готово: компонент = " + c, `Внешни
 вязности.`);
return frames;
}

```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v
 let comps = NODES.length;
 const push = (title: string, note: string, edge: [num

```

```

const [mode, setMode] = useState<Mode>("dfs");
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);
const timer = useRef<number | null>(null);

const frames = mode === "dfs" ? DFS_FRAMES : DSU_FRAMES;
const maxStep = frames.length - 1;
const frame = frames[Math.min(step, maxStep)] as DfsFrame;

/* — вкладка сообщает компилятору, какой код подстав. */
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode);
}, [chapterId, mode]);

/* — автопроигрывание — */
useEffect(() => {
 if (!playing) return;
 timer.current = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 900);
 return () => {
 if (timer.current) window.clearInterval(timer.current);
 };
}, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.min(n, maxStep)));

/* — переменные компилятора, которые рисуются НАПРЯМУЮ */
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р. */
const liveComp = useMemo(() => {
 const arr = vars ? (vizArray(vars.comp) ?? vizArray(vars.used)) : [];
 return arr;
}, [vars]);

```

```

 if (liveComp && liveComp[id] !== undefined && liveC
 if (parentNow) return setOf(parentNow, id);
 return frame.comp?.[id] ?? 0;
 };

```

```

const frontier = liveFrontier ?? (mode === "dfs" ? (f
const cur = liveCur ?? frame.cur;
const comps = liveComps ?? frame.comps;
const edge = frame.edge;
const title = runtime && vars ? `${frame.title} · K

```

```

const switchMode = (m: Mode) => {
 setMode(m);
 setStep(0);
 setPlaying(false);
};

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-1
 <div className="flex flex-wrap items-center justifi
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-whi
 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold
 <p className="text-xs text-emerald-300">10
 </div>
 </div>
 </div>
 <div className="flex items-center bg-slate-950 p
 <button
 onClick={() => switchMode("dfs")}
 className={"flex items-center gap-1.5 px-3 p
 600 text-white shadow" : "text-slate-400 ho
 >
 <Layers className="w-3.5 h-3.5" /> 06ход (D
 </button>
 <button

```

```

<div className="bg-slate-950 rounded-xl p-3 border
 <div className="absolute inset-0 bg-[radial-gra
 <svg viewBox="0 0 500 370" className="w-full h-
 {EDGES.map(([u, v], i) => {
 const a = NODES[u];
 const b = NODES[v];
 const active = edge && ((edge[0] === u && e
 const sameComp = compOf(u) > 0 && compOf(u)
 return (
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={active ? "#fbbf24" : sameComp ?
 strokeWidth={active ? 4 : sameComp ? 2.
 strokeDasharray={sameComp ? undefined :
 className="transition-all duration-300"
 opacity={sameComp || active ? 1 : 0.55}
 />
);
 })}
 {NODES.map((n) => {
 const c = compOf(n.id);
 const color = c > 0 ? COLORS[(c - 1) % COLOR
 const isCur = cur === n.id;
 const inFrontier = frontier.includes(n.id);
 return (
 <g key={n.id} onClick={() => goTo(step)} .
 {isCur && <circle cx={n.x} cy={n.y} r={
 {inFrontier && !isCur && <circle cx={n.
 3" />}
 <circle cx={n.x} cy={n.y} r={13} fill={
 3 : 1.5} className="transition-all duration
 <text x={n.x} y={n.y + 4} textAnchor="m
 {n.label}</text>
 {c > 0 && <text x={n.x} y={n.y - 19} te
 ents-none">c{c}</text>}
 </g>
);
 })}

```

```

 {vars ? {
 ["v", "comps", "c", "i", "e", "comp", "parent"] : (
 vars[name] === undefined ? null : (

 {name} = {JSON.stringify(vars[name]).slice(0, 100)}

)
)
 } : (

 открой панель Python – переменные <code cla
 erald-400">parent</code>, <code className="
 comps</code> покрасят этот граф сами

)}
 </div>
</div>
);
};

```

```
export default GraphComponentsViz;
```

---

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';

```

```

type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };

```

```

const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },

```

```

 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зашли в ${v}`,
 visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс в ${v}`,
 .stack], visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Вышли из ${v}`,
 ueOrStack: [...stack], visitedNodes: Array.from(vis) });
 }

 dfs(start);
 return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начали с ${start}`,
 ray.from(vis)] });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Действие в ${v}`,
 q], visitedNodes: [...Array.from(vis)] });
 q.shift();
 }
}

```

```

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium rounded-md
 text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>
 </div>
);

```



```

 strokeColor = "stroke-amber-500"
 textColor = "fill-amber-500"
 radius = 22;
 }

 return (
 <g key={n.id} className="graph-node">
 <circle cx={n.x} cy={n.y}
 className={`stroke-${strokeColor} fill-${fillColor} stroke-width=2`}
 />
 <text x={n.x} y={n.y}
 className={`text-${textColor} font-size=14 font-weight=bold`}
 {n.label}
 />
 />

 {
 /* Ripple for BFS mode */
 {current && mode === 'bfs' && n === current ? (
 <circle cx={n.x} cy={n.y}
 className="stroke-amber-500 stroke-width=2"
 />
) : null}
 />
)
)
}
}
}
</svg>
</div>

/* Structure View (Stack / Queue) */
<div className="bg-slate-900 border border-emerald-500 p-10" >
 <div className={`absolute -top-3 left-3 right-3 bottom-3
 ${mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'}
 border border-emerald-500`} >
 {mode === 'dfs' ? 'Стек вызовов' : 'Очередь'}
 </div>

 <div className="flex flex-col gap-2 mt-10" >
 {step.queueOrStack && step.queueOrStack.length > 0 ? (
 <div key={idx} className={`w-40 p-2 ${
 mode === 'dfs' ? 'bg-slate-800' : 'bg-slate-900'
 }`} >
 {step.queueOrStack[idx]}
 </div>
) : null}
)
 </div>
</div>

```

```
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 p-4 overflow-hidden">
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>
 </div>
);
}
```

---

ФАЙЛ 46/111: src/components/HeapViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../hooks';

interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}

// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
```

```

 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Болиит сердце', emoji: '🫀' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '🦴' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '🌡️' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '🩸' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '🤧' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();
 const liveHeap = vizArray(runtime?.variables?.h) ?? v

```

```

const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x => x);
// Mark specifically this new patient as 'in' in the heap
const target = nextHeap.find(x => x.priority === priority);
const marked = nextHeap.map(x => x.id === (target?.id));

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
}, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};

```

```

 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))), 1000);
};

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
 x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
 x2={` ${rp.x}%` } y2={` ${rp.y - 4}%` }
 stroke="#475569" strokeWidth="2"
 />
);
 }
});

```

```

 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
 />
 <div className="flex flex-col items-center min-w-[300px]">
 Приоритет
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-700 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center"
 >

```

```

 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isCompilerNode ? 'ring-4 ring-amb
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 roun
 <div className="absolute top-4 left-4 text-[1
 Операционная койка
 </div>

```

```
 <div className="text-[10px] font-mono text-slate-500">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`}
 >
 {idx === 0 ?
 {entry}
 </div>
)}
 </div>
 </div>
);
 };
};
```

---

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { createContext, useContext, useEffect, useReducer } from 'react';
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```



```

 {caption}
 </div>
 })
 <div className="text-[9px] text-slate-600 text-center"
 {paused ? "пауза – курсор на карточке" : "наведи"}
 </div>
 </div>
);
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

```

```

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDash={dashed ? [4, 4]} : []
 />
);

```

```

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (

```

```

 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
}}
{Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
})}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотные индексы"
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
 <text x={startX + i * (boxW + 4) + boxW / 2} y={30} />
 </g>
 })}
 </Svg>
);
};

```

```

const f = frames[step];
const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f87193" : "#f08080";

return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: p поворачивается к x"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x"}] },
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "p"}] }
]}
 />
);

```

```

};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "E"], ["D", "F"],
];

export const BfsDemo: React.FC = () => {
 const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order[step - 1] : null;
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`} >
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
 })}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
 stroke={wave?.includes(k) ? "#a5b4fc" : undefined} />
 })}
 </Svg>
);
};

```

```

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join(" → ")}` : ""} >
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
 fill={onPath ? C.onPath : visited.has(u) || visited.has(v) ? C.visited : C.inactive}
 stroke={onPath ? "#a5b4fc" : undefined} />
 })}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={visited.has(k) ? C.visited : C.inactive}
 stroke={visited.has(k) ? "#a5b4fc" : undefined} />
 })}
 </Svg>
);
};

```

```

 </Svg>
);
};

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь

 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
 <text x={130} y={54} textAnchor="middle" fontSize=14 />
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize=14 />
 <text x={200} y={106} textAnchor="middle" fontSize=14 />
 <text x={130} y={22} textAnchor="middle" fontSize=14 />
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C–D → граф развалился на 2 компонента" : ""} />
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (

```

```

const taken = new Set(chosen.slice(0, step));
return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={(pos[e.u][0] + pos[e.v][0]) / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step :
 })}
 </Svg>
);
};

```

```

 <text x={130} y={111 - i * 26} textAnchor="middle"
 </g>
)}
 <text x={130} y={16} textAnchor="middle" fontSize=16>
 </Svg>
);
};

```





```

<text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
{a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim} />
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
)}}
<text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
{pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "stroke 0.5s" }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
)}}
</Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`}>
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={12}
 fill={C.dim} />
 <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
 </g>
))}
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} />
))}
 </Svg>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={44}
 fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.dim}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">π</text>
 <rect x={18 + idx * 33} y={70} width={28} height={44}
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] — длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово ${step}`
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (

```

```

| "suffix-link" | "toposort" | "relax" | "prefix-sum"
| "segment-tree" | "bridge" | "articulation" | "mst"
| "prefix-function" | "dp" | "floyd" | "components" |
| "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};

/** Список работающих мини-демо — его проверяет `npm run
```

```

export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

 const viz = getViz(vizId ?? undefined);
 if (!vizId || !viz) return null;
 const Component = viz.Component;

 return createPortal(
 <div className="fixed inset-0 z-[110] flex items-end
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 <VizLiveStatus demoTitle={viz.title} classN
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-cont

```

```
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor,
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({
```

```
 useLayoutEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - GAP)), left: left });
 }, [anchor]);
```

```
 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
 }, [anchor, onClose]);
```

```

 <p className="text-[11.5px] leading-relaxed
 {entry.formal}
 </p>
)}

 {entry.complexity && (
 <div className="text-[11px] font-mono text-
 Сложность: {entry.complexity}
 </div>
)}

 {viz && simulatorId && (
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-
 unded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
)}
 </div>
 </div>,
 document.body
);
};

```

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {

```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-dasharray-4";
 if (step === 2) return "stroke-pink-500 stroke-dasharray-4";
 return "stroke-slate-700 stroke-dasharray-4";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-dasharray-4";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда и далее
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

```



```

 return false;
 };

 const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соеди
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.

```

```

<div className="bg-[#0f172a] flex-1 min-h-screen p-4">
 <svg width="400" height="300" class="w-full h-full">
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5" orient="start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" />
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5" orient="start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" />
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5" orient="start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" />
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5" orient="start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" />
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5" orient="start-reverse">
 <path d="M 0 0 L 10 5 L 0 10" />
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e, step);
 }
 })}
 </svg>

```

```

 : 'fill-slate-900'
 />
 <text x={n.x} y={n.y}
- white"> {n.label}
 </text>

 {pot !== null && {
 <g transform={`
fade-in zoom-in duration-500"> <rect x="-100" y="0"
k-500 stroke-1" /> <text x="0" y="0"
font-bold fill-pink-400"> {pot}
 </text>
 </g>
 }}
 </g>
)
 }}}
 </svg>
 </div>

 <div className="w-full md:w-[320px] flex flex-col
 { /* Log Box */
 <div className="bg-[#0f172a] p-5 rounded-md
l justify-center leading-relaxed text-sm text-white"> {getDesc()}
 </div>

 { /* Controls */
 <div className="flex bg-slate-900 border border-slate-800
 <button onClick={() => setStep(1)}
 className="p-3 bg-slate-800 border border-slate-400 transition-colors" title="Сброс
 <RotateCcw strokeWidth={3}
 </button>

```

```

 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;

```

```

 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещенной`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {

```

```

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в ${sccMap[u]}`,
 u,
 true,
);
 }
}

```

```

 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button

```

```

 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" />
 </marker>
 </defs>

 {/* Edges */}
 {initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u)
 const v = initialNodes.find((n) => n.id === edge.v)

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||

```



```

 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "done") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-300`}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >

```

```

) : (
 [...step.order].reverse().map((val, idx) => {
 <div
 key={idx}
 className="bg-indigo-950 border border-white
ex items-center gap-1"
 >
 {val}
 </div>
))
 })
 </div>
</div>

{/* List of actions log */}
<div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map((s, idx) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded
 >
 {s.desc}
 </div>
))}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">


```

```

 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'm
}

```

```

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

```

```

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

```

```

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

```

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending'

```

```

 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v.id === 1));
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 weight: 1,
 type: 'success'
 }, ...prev));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 weight: 3,
 type: 'info'
 }, ...prev));
 },
 // Step 4: Include D-E
 () => {
 setVertices(prev => prev.map(v => v.id === 3 || v.id === 4));
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 weight: 3,
 type: 'success'
 }, ...prev));
 },
 // Step 5: Inspect B-C (4)
 () => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 weight: 4,
 type: 'info'
 }, ...prev));
 }
]
}

```

```

 description: 'H0: это ребро соединяет вершины B
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе п
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину

```

```

 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовый',
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. Соединяем!',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю I',
 type: 'info'
 }, ...prev]));
},

```

```

() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D:
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону верш
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 eap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в куч
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
 description: 'Плесень стремительно бежит к верши

```

```

 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (Е) тянет
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]));
},

```



```

 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем E-H',

```

```

// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
description: 'Каждая из 9 деревень одновременно
 А выбирает А-В(2), D выбирает D-Е(3), а G вы
 !',
 type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/blue)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' };
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' };
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
},

```

```

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const introLog = (algo: 'kruskal' | 'prim' | 'boruvka') => {
 if (algo === 'kruskal') {
 return [{
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа',
 type: 'info',
 }];
 }
 if (algo === 'prim') {
 return [{
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем старто
 ь (Нар) и захватываем соседей!',
 type: 'info',
 }];
 }
 return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 дерев
 ы объединиться в колхоз.',
 type: 'info',
 }];
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);

```

```
}, []));
```

```
useVizStepSync(stepIndex, goToStep, currentSteps.length);
```

```
/** A..I ↔ id 0..8 */
```

```
const idOf = (name: string | number | null | undefined) => {
 if (name === null || name === undefined) return null;
 if (typeof name === 'number') return Number.isInteger(name) ? name : null;
 const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
 return c ? c.charCodeAt(0) - 65 : null;
};
```

```
/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
```

```
const parseEdge = (item: unknown): { u: number; v: number; w: string | null } | null => {
 if (item === null || item === undefined) return null;
 if (typeof item === 'string') {
 const nums = item.match(/-?\d+(?:\.\d+)?/g);
 const letters = item.match(/[A-Ia-i]/g);
 let u: number | null = null;
 let v: number | null = null;
 if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
 else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
 if (u === null || v === null) return null;
 return { u, v, w: nums && nums.length ? nums[0] : null };
 }
 if (Array.isArray(item)) {
 const letters = item.filter((x) => typeof x === 'string');
 const nums = item.filter((x) => typeof x === 'number');
 if (letters.length >= 2) {
 const u = idOf(letters[0]); const v = idOf(letters[1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length ? nums[0] : null };
 }
 if (nums.length >= 2) {
 const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length > 2 ? nums[0] : null };
 }
 }
 return null;
};
```

```

 feed(id, val);
 });
}
/**
 * Ключ множества вершины: если код дал DSU-массив
 * идём по цепочке до корня – иначе вершины одной к
 * разные цвета. Для произвольных меток («comp1») б
 */
const compOfVertex = new Map<number, string>();
rawComp.forEach((label, id) => {
 if (parentIdx.size === 0) {
 compOfVertex.set(id, label);
 return;
 }
 let cur = id;
 const seen = new Set<number>([id]);
 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentIdx.get(cur);
 if (nxt === undefined || nxt === cur || seen.has(nxt)) {
 seen.add(nxt);
 cur = nxt;
 }
 compOfVertex.set(id, String.fromCharCode(65 + cur));
 });
const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
const takenEdges = (takenRaw ?? []).map(parseEdge);
const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken))
 .map((x) => idOf(typeof x === 'string' ? x : vizNumber(x)))
 .filter((x): x is number => x !== null);
const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
const curW = vizNumber(vars.w);
const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
const heapEdges = (heapRaw ?? []).map(parseEdge);
const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw)
 .filter(e => e !== null) : [];
const total = vizNumber(vars.total) ?? vizNumber(vars.total);

```

```

switch (color) {
 case 'red': return 'bg-red-500 border-red-300 text-red-500';
 case 'blue': return 'bg-blue-500 border-blue-300 text-blue-500';
 case 'purple': return 'bg-purple-600 border-purple-300 text-purple-500';
 case 'mold': return 'bg-emerald-500 border-emerald-300 text-emerald-500';
 case 'gold': return 'bg-amber-400 border-amber-200 text-amber-500';
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIndex === 0)
 return 'bg-slate-700 border-emerald-500 text-emerald-500';
 return 'bg-slate-700 border-slate-500 text-slate-500';
}
};

```

```

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800 p-4">
 <div className="flex flex-col lg:flex-row justify-between">
 <div>
 <div className="flex items-center gap-2 mb-2">


```

```

 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-400
)
 }
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
</div>

<button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2
-200 font-semibold text-xs rounded-xl border
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

<button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 >

```

```

 })
 {live.cheapestEdges.length > 0 && {

 cheapest: {live.cheapestEdges.length}

 })
 {live.total !== null && {

 вес остова = {live.total}

 })
 </>
) : (

 открой панель Python и запусти код вкладки
 'ка'>»: переменные
 {' '}<code className="font-mono text-purple-300">
 e className="font-mono text-purple-300">u</code>
 mono text-purple-300">w</code> перекрасят э

)}
</div>

<div className="grid grid-cols-1 lg:grid-cols-3">
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 rounded-full ro
 ative min-h-[480px] overflow-hidden">

 {/* Legend */}
 <div className="absolute top-4 left-4 bg-slate-950 p-4
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">
 </div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-950">

```



```

<svg className="absolute inset-0 w-full h-full"
 aspectRatio="none">
 {effEdges.map(edge => {
 const u = effVertices[edge.u];
 const v = effVertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-200"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' : isRejected ? '#94a3b8'}
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 })}

```

```

<div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
</div>

```

```

{activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5"
 {heapQueue.length === 0 ? {
 <span className="text-xs text-slate
 } : {
 heapQueue.map(({item, idx}) => {
 <span key={idx} className={"px-2 p
0 shadow-md animate-pulse" : "bg-slate-800
 {item}

 })
 })
 </div>
</div>
}}

```

```

<div className="flex-1 p-4 overflow-y-auto
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500

```

```

 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka')}
 className={"px-3 py-1.5 rounded text-xs font-medium"}
 style={activeCheatTab === 'boruvka' ? "bg-rose-600 text-white" : "bg-white text-rose-600"}
 >
 Боровка (Билет 20)
 </button>
</div>
</div>

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols-2 gap-4">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Clock className="w-3.5 h-3.5 text-inherit" />
 </p>
 <p className="text-2xl font-black text-white">Крускал</p>
 <p className="text-slate-400 text-xs font-medium">Метод нахождения минимального остовного дерева</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-inherit" />
 </p>
 <p className="text-2xl font-black text-white">Авард</p>
 <p className="text-slate-400 text-xs font-medium">Метод нахождения минимального остовного дерева</p>
 </div>
 <div className="bg-slate-950 p-4 rounded-lg">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-inherit" />
 </p>
 <p className="text-2xl font-black text-white">Авард</p>
 <p className="text-slate-400 text-xs font-medium">Метод нахождения минимального остовного дерева</p>
 </div>
 </div>
 <div className="lg:col-span-2">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-inherit" />
 </p>
 <p className="text-2xl font-black text-white">Авард</p>
 <p className="text-slate-400 text-xs font-medium">Метод нахождения минимального остовного дерева</p>
 </div>
 </div>
)

```

```

 <p className="text-2xl font-black text-slate-900">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <p className="text-2xl font-black text-slate-900">
 <p className="text-slate-400 text-xs mt-1">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 в предложении:</p>
 <p className="text-xs text-slate-300 leading-5">
 ие чистые вершины через самое дешевое доступное ребро
 </p>
 </div>
</div>
<div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
80 flex-1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next w, v in graph[u]:

```

```

 </p>
 <pre className="text-xs text-emerald-400" style="font-family: monospace; font-size: 0.8em; margin: 10px 0;">
 80 flex-1">
{`def boruvka(n, edges):
 parent = list(range(n))
 mst = []
 while len(mst) < n - 1:
 # Для каждого колхоза ищем самый дешёвый мост
 cheapest = [-1] * n
 for u, v, w in edges:
 ru, rv = find(u), find(v)
 if ru != rv:
 if cheapest[ru] == -1 or cheapest[ru][2] > w:
 cheapest[ru] = (u, v, w)
 if cheapest[rv] == -1 or cheapest[rv][2] > w:
 cheapest[rv] = (u, v, w)
 # Объединяем все колхозы по выбранным мостам
 for bridge in cheapest:
 if bridge != -1 and union(bridge[0], bridge[1]):
 mst.append(bridge)`}
 </pre>
 </div>
</div>
}}
</div>
</div>
);
};
```

---

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
```

```

 .lite-wrap pre { background: #f3f4f6; border: 1px solid #d1d3d4; padding: 1.45rem; }
 .lite-wrap table { border-collapse: collapse; width: 100%; }
 .lite-wrap th, .lite-wrap td { border: 1px solid #d1d3d4; padding: .5rem; }
 .lite-wrap th { background: #f3f4f6; }
 .lite-wrap details { margin: .6rem 0; border: 1px solid #d1d3d4; padding: .5rem; }
 .lite-wrap summary { cursor: default; font-weight: 700; }
 .lite-toc { font-size: .92rem; }
 .lite-toc ol { margin: .4rem 0 0 1.2rem; }
 @media print {
 .lite-wrap { max-width: none; padding: 0; }
 h2 { page-break-before: always; }
 h2:first-of-type { page-break-before: avoid; }
 pre, table, details { page-break-inside: avoid; }
 }
 `;

```

/\*\* Раскрываем все спойлеры и вычищаем служебную разметку \*/

```

function toPlainTextHtml(html: string): string {
 return html
 .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
 // кнопки/иконки внутри текста не нужны в текстовой версии
 .replace(/<button[\s\S]*?<\s*\/button>/gi, "")
 .replace(/<svg[\s\S]*?<\s*\/svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/<sdata-term-id="[^"]*">/g, "")
 .replace(/<class="term-hint"[^"]*">/g, "")
 // ссылки на другие билеты должны вести в текстовую версию
 .replace(/<href="\?topic=/>/g, 'href="?lite=1&topic=')
 .replace(/<class="ticket-link"/>/g, "");
}

```

```

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

```

```

export const LiteGuide: React.FC<Props> = ({ topicId }) => {

```

```

<h2>{chapter.title}</h2>
<p className="lite-topics">
 {topics.length > 0 && <>Билет{topics.leng
 {chapter.category ? <>Раздел: {chapter.ca
 id страницы: <code>{chapter.id}</code>
</p>

<div
 dangerouslySetInnerHTML={{
 __html: linkifyTicketsHtml(toPlainTextH
 }}
/>

{viz && (
 <div className="lite-viz">
 <h4>Визуализация: {viz.title}</h4>
 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {
 const sync = PAGE_SYNC[k];
 const demo = k.includes("#") ? k.spli
 return {
 <div key={k}>
 <p>
 {demo ? `Вкладка «${demo}»`
 {sync.vizTitle ? ` – ${sync.viz
 {sync.stepNote ? <> Шаг демонстр
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => {
 <li key={v.name}>
 <code>{v.name}</code> – {
 {v.range ? ` (${v.range})

 }}}

)}
 </div>

```

```

export default LiteGuide;

```

ФАЙЛ 58/111: src/components/LiveDistTable.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
 /** dist из кода пользователя: имя вершины → расстояние */
 dist: Record<string, DebugValue> | null;
 /** prev/p – предок на кратчайшем пути (необязательно) */
 prev?: Record<string, DebugValue> | null;
 /** done/visited – зафиксированные вершины (необязательно) */
 done?: DebugValue[] | null;
 /** Идентификаторы вершин, нарисованных на схеме демо */
 demoIds: string[];
 /** Подпись таблицы (по умолчанию «Ваши расстояния»). */
 title?: string;
}

const fmt = (value: DebugValue | undefined): string =>
 if (value === undefined || value === null) return "—";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "−∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "−∞";
 return value;
 }
 return String(value);
};
```



```

 {shown.map(({key}) => {
 const parent = prev ? fmt(prev[key]) : null;
 const fixed = doneSet.has(key);
 return (
 <span
 key={key}
 className={`inline-flex items-baseline gap-2
 ${fixed
 ? "border-emerald-500/40 bg-emerald-500/10"
 : "border-slate-700 bg-slate-900/70 text-slate-500"}
 `}
 title={fixed ? "вершина зафиксирована (done)" : ""}
 >
 <b className={fixed ? "text-emerald-300" : ""}>
 =
 {fmt(dist[key])}
 {parent && parent !== "-" ?
 {fixed ?

 } : ""}

);
 })}
 {keys.length > shown.length ? (

 +{keys.length - shown.length} ещё

) : null}
 </div>
 </div>
);
}

```

```
 rest: '}'. Все против всех – ищет кратчайшие пути от
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-500/40',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4">
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-105"/>
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">
 {c.desc} {c.complexity}
 </p>
 <div className={`mt-3 text-xs font-mono px-2 ${c.titleColor}`}>{c.time}</div>
 </div>
 </div>
))}
 </div>
);
}
```

---

ФАЙЛ 60/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
```

```

 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[100px] h-[100px] bg-white text-slate-900 flex items-center justify-center transition-colors duration-180s ease-out">
 <div className="flex items-center justify-center">

 Закрыть
 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
)}

```

```

return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 h-10 w-10 rounded-sm">
 <Sparkles className="w-5 h-5 lg:w-6 lg:h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-lg lg:text-xl font-extrabold">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры данных
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center flex-wrap sm:flex-wrap md:flex-nowrap">

 <div className="bg-indigo-600/30 shadow-indigo-600/30 whitespace-nowrap">
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </div>

 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (настройка)"
 aria-label={compilerOpen ? "Скрыть панель Python-компилятора" : "Показать панель Python-компилятора"}
 className={`shrink-0 flex items-center justify-center gap-2 duration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg'
 : 'text-slate-400 hover:text-white bg-slate-800'
 }`}
 >
 <div className="w-4 h-4"></div>
 Python-компилятор
 </button>
 </div>
 </div>
)

```

```

 >
 <FileDown className="w-3.5 h-3.5" />

 <a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}exp
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center gap-2 px
/20 border border-sky-500/30 transition-all
 title="Скачать полный TXT файл со все
 >
 <FileText className="w-3.5 h-3.5" />

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center gap-2 w-
te hover:bg-amber-600/20 border border-amber
ointer"
 title="Скачать всё пособие одним авто
 >
 {busy ? <Loader2 className="w-3.5 h-3
 </button>
 </div>
 </details>
</div>
</div>
</div>
</div>
</header>
);
};

```

```

 adj: number[][];
 pos: { x: number; y: number }[];
 names: string[];
}

const PALETTE = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];
const colorOf = (c: number) => (c >= 1 && c <= PALETTE.length) ? PALETTE[c - 1] : "#ccc";

function makeGraph(key: GraphKey): Graph {
 if (key === "bip") {
 const edges: [number, number][] = [[0, 3], [0, 4], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]];
 const pos = [
 { x: 80, y: 60 }, { x: 80, y: 150 }, { x: 80, y: 240 },
 { x: 380, y: 60 }, { x: 380, y: 150 }, { x: 380, y: 240 }
];
 const g: Graph = { key, label: "Двудольный (6 вершин, 6 ребер)", n: 6, edges };
 g.adj = buildAdj(g);
 return g;
 }
 const edges: [number, number][] = [[0, 1], [0, 2], [1, 2], [3, 4], [3, 5], [4, 5], [0, 3], [1, 4], [2, 5]];
 const pos = [
 { x: 70, y: 60 }, { x: 175, y: 40 }, { x: 150, y: 140 },
 { x: 250, y: 185 }, { x: 365, y: 110 }, { x: 350, y: 240 }
];
 const g: Graph = { key, label: "Основной (8 вершин, 9 ребер)", n: 8, edges };
 g.adj = buildAdj(g);
 return g;
}

function buildAdj(g: Graph): number[][] {
 const adj: number[][] = g.edges.reduce((acc: number[][], [u, v]) => {
 acc[u].push(v);
 acc[v].push(u);
 return acc;
 }, Array.from({ length: g.n }, () => [] as number[]));
 adj.forEach((a) => a.sort((x, y) => x - y));
 return adj;
}

```

```

 });
 }
 const chi = Math.max(...color);
 frames.push({
 title: `Готово: жадная покраска использовала ${chi}
 note: ` $\chi_{\text{жадн}} = ${chi}$. Истинное хроматическое число
 , а теорема Брукса ($\chi \leq \Delta$, если граф не полный и
 color: [...color], v: null, u: null, e: null,
 });
 return frames;
}

```

```

/** Проверка двудольности BFS-ом: 2 краски нет нечётных
function bipartiteFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const queue: number[] = [];
 let conflict: [number, number] | null = null;
 frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",
 note: "BFS красит вершину в цвет, противоположный р
 двудольный ($\chi > 2$).",
 color: [...color], v: null, u: null, e: null,
 });
 for (let start = 0; start < g.n && !conflict; start++)
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.`
 color: [...color], v: start, u: null, e: null,
 });
 while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {
 if (color[u] === 0) {

```

```

frames.push({
 title: "Старт: рёбра не покрашены",
 note: `Правильная рёберная покраска: у каждой верши
 бер одной вершины различны), а по теореме Визинга
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
g.edges.forEach(([a, b], i) => {
 const usedA = Array.from(inc[a].values());
 const usedB = Array.from(inc[b].values());
 const used = Array.from(new Set([...usedA, ...usedB]));
 let c = 1;
 while (used.includes(c)) c += 1;
 ecolord[i] = c;
 inc[a].set(i, c);
 inc[b].set(i, c);
 frames.push({
 title: `Рёбро (${g.names[a]}, ${g.names[b]}): цве
 note: `В вершине ${g.names[a]} заняты цвета ${us
 y) => x - y).join(", ") || "-"}$. Наименьший св
 да.`
 color: new Array(g.n).fill(0), ecolord: [...ecolor]
 });
});
const chiE = Math.max(...ecolor);
frames.push({
 title: `Готово:  $\chi' = \chiE$  при  $\Delta = \delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф: $\chi' = \Delta = \delta$ – это теор
 сс 1 по Визингу.`
 : ` $\chi' = \chiE$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$
 $\chi' = \Delta + 1$ – классом 2 (например, КЗ и любой
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
 });
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React

```



```

 setPlaying(false);
 return s;
 }
 return s + 1;
 });
}, 1300);
return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizA
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vi
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumb
const linked = Boolean(liveColor || liveEcolor || (li

const frame = frames[Math.min(step, maxStep)] ?? ({ t
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number")
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV)
const curU = liveU !== null && Number.isFinite(liveU)
const curE = liveE !== null && Number.isFinite(liveE)
const delta = Math.max(...graph.adj.map((a) => a.leng
const usedColors = Array.from(new Set(graph.pos.map((

```

```

<div className="flex flex-wrap items-center gap-2"
 <button onClick={() => { setStep(0); setPlaying
 rounded-lg bg-slate-900 border border-slate-
 <button onClick={() => go(-1)} disabled={step =
 00 border border-slate-700 text-slate-300 hove
 <button onClick={() => setPlaying((p) => !p)} c
 ext-xs font-bold hover:bg-indigo-500">
 {playing ? <Pause className="w-4 h-4" /> : <P
 </button>
 <button onClick={() => go(1)} disabled={step >=
 late-900 border border-slate-700 text-slate-3
 on>
 <span className="text-[11px] text-slate-500 ml-
 <span className="ml-auto flex flex-wrap items-c
 <span className="bg-slate-950 border border-s
 {tab === "edges"
 ? <span className="bg-slate-950 border bord
 rs}
 : <span className="bg-slate-950 border bord
 }}

</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-5 g
 <div className="lg:col-span-3 bg-slate-950 roun
 <svg viewBox="0 0 480 300" className="w-full l
 {graph.edges.map(([a, b], i) => {
 const ec = edgeColor(i);
 const isCur = i === curE;
 const isBad = Boolean(frame.bad && ((fram
)));
 return (
 <g key={i}>
 <line
 x1={graph.pos[a].x} y1={graph.pos[a
 stroke={isBad ? "#f43f5e" : ec} > 0
 strokeWidth={isCur ? 6 : isBad ? 5

```

```

 <div className="flex flex-wrap gap-1.5 font
 {(tab === "edges" ? graph.edges.map(({_, i
 <span key={i} className="px-1.5 py-0.5
 ? colorOf(c) : "#475569", background: c > 0
 {tab === "edges" ? `e${i}` : graph.na

 }}}
 </div>
</div>
<div className="bg-slate-900/70 border border
 {tab === "greedy" && <>Жадная покраска: <b
 : <b className="text-slate-200"> $\chi \leq \Delta$,
 в $\chi \leq 5$ (доказывается через $E \leq 3V - 6$ и це
 ром).</>}</div>
 {tab === "bipartite" && <>Двудольный граф
 /DFS за $O(V + E)$. Распознавание 3-раскрашив
 {tab === "edges" && <>Рёберная покраска: <b
 Δ (Кёниг). Задача « $\chi' = \Delta$?» NP-полна даже
 </div>
</div>
</div>

<div className="mt-3 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python: массив <code classNam
 igo-300">ecolor</code> для рёбер) и вершина

```

---

ФАЙЛ 63/111: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотри
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900 text-white p-2">
 00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0" >
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f[id]; }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 const o1=ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y);
 const o2=ccw(p1.x,p1.y,p2.x,p2.y,p4.x,p4.y);
 const o3=ccw(p3.x,p3.y,p4.x,p4.y,p2.x,p2.y);
 const o4=ccw(p4.x,p4.y,p3.x,p3.y,p1.x,p1.y);
 return (o1>0&&o2>0&&o3>0&&o4>0);
 }
 })()}
 </svg>
 </div>
 </div>
)
 }
```

```

<div className="absolute top-2 left-2 bg-slate-
 00/30 w-auto z-20">К3,3 – НЕПЛАНАРЕН</div>
<svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:160,y:220},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:60,y:60}
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} y1={p1.y} x2={p2.x} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 if (crosses.includes(i)) {
 circles.push(<circle key={`c${i}`} cx={p1.x} cy={p1.y} r={20}/>);
 circles.push(<circle key={`c${i+1}`} cx={p2.x} cy={p2.y} r={20}/>);
 }
 }
 })}

```

```

 ChevronRight,
 ChevronUp,
 CircleStop,
 Copy,
 Eye,
 EyeOff,
 HelpCircle,
 Link2,
 Loader2,
 Pause,
 Pencil,
 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

```

```

interface Window {
 __algv0PythonRuntime?: PythonRuntimeProvider;
}

function getPythonRuntime(): Promise<PyodideRuntime> {
 const injected = typeof window !== "undefined" ? window : null;
 if (injected) return typeof injected === "function" ? injected : null;
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_PATH)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_PATH }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но не по
 * напечатанной строки: print("x" * 10_000_000) или print("x" * 10_000_000)
 * – это один шаг, после которого весь текст уезжал в <div>.
 * Обрезаем по границе строки и честно пишем, сколько э
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
 if (text.length <= OUTPUT_CHAR_CAP) return text;
 const cut = text.slice(0, OUTPUT_CHAR_CAP);
 const lastBreak = cut.lastIndexOf("\n");
 const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.slice(0, lastBreak) : cut;
 return `${head}\n... вывод обрезан: показаны первые ${head.length} символов`;
}

```

```
{
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
},
{
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"Итерация {i}\")\ni += 1",
},
{
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
},
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = take",
 code: "st = []\nst.append('A') # push\nst.pop() # take",
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
 code: "from collections import deque\nq = deque()\nq.append(1)\nq.popleft()",
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: "import heapq\nh = []\nheapq.heappush(h, (1, 'a'))\nheapq.heappop(h)",
 },
],
},
{
 group: "Графы",
 items: [
 {
 label: "Поиск в ширину (BFS)",
 tip: "Используется очередь (deque).",
 code: "from collections import deque\ngraph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}\nq = deque(['A'])\nvisited = set()",
 },
 {
 label: "Поиск в глубину (DFS)",
 tip: "Используется рекурсия или стек (list).",
 code: "graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}\ndef dfs(node):\n visited.add(node)\n for neighbor in graph[node]:\n if neighbor not in visited:\n dfs(neighbor)",
 },
],
},
{
 group: "Динамическое программирование",
 items: [
 {
 label: "Таблица",
 tip: "Хранит результаты подзадач.",
 code: "dp = [0] * (n + 1)",
 },
 {
 label: "Память",
 tip: "Ограничение на использование памяти.",
 code: "def fib(n):\n if n < 2:\n return n\n return fib(n - 1) + fib(n - 2)",
 },
],
},
{
 group: "Сортировки",
 items: [
 {
 label: "Быстрая сортировка (QuickSort)",
 tip: "Среднее время O(n log n).",
 code: "def quicksort(arr):\n if len(arr) < 2:\n return arr\n pivot = arr[len(arr) // 2]\n left = [x for x in arr if x < pivot]\n right = [x for x in arr if x > pivot]\n return quicksort(left) + [pivot] + quicksort(right)",
 },
 {
 label: "Сортировка слиянием (MergeSort)",
 tip: "Всегда O(n log n).",
 code: "def mergesort(arr):\n if len(arr) < 2:\n return arr\n mid = len(arr) // 2\n left = mergesort(arr[:mid])\n right = mergesort(arr[mid:])\n return merge(left, right)",
 },
],
},
{
 group: "Алгоритмы",
 items: [
 {
 label: "Нахождение кратчайшего пути (Dijkstra)",
 tip: "Используется куча (heapq).",
 code: "import heapq\ngraph = {'A': {'B': 1, 'C': 4}, 'B': {'A': 1, 'D': 2}, 'C': {'A': 4, 'D': 3}, 'D': {'B': 2, 'C': 3}}\ndist = {node: float('inf') for node in graph}\ndist['A'] = 0\npq = [(0, 'A')]\nwhile pq:\n d, u = heapq.heappop(pq)\n if d > dist[u]:\n continue\n for v, w in graph[u].items():\n nd = d + w\n if nd < dist[v]:\n dist[v] = nd\n heapq.heappush(pq, (nd, v))",
 },
 {
 label: "Нахождение минимального остовного дерева (Kruskal)",
 tip: "Используется сортировка по весу.",
 code: "graph = {'A': {'B': 1, 'C': 4}, 'B': {'A': 1, 'D': 2}, 'C': {'A': 4, 'D': 3}, 'D': {'B': 2, 'C': 3}}\nedges = [(u, v, w) for u, v, w in graph.items() for neighbor, weight in graph[u].items() if u < v]\nedges.sort()\nparent = {node: node for node in graph}\ndef find(u):\n while parent[u] != u:\n u = parent[u]\n return u\ndef union(u, v):\n pu, pv = find(u), find(v)\n if pu != pv:\n parent[pu] = pv",
 },
],
},
{
 group: "Матрицы",
 items: [
 {
 label: "Умножение матриц",
 tip: "O(n^3) для наивного алгоритма.",
 code: "def multiply(A, B):\n n = len(A)\n m = len(B)\n C = [[0] * m for _ in range(n)]\n for i in range(n):\n for j in range(m):\n for k in range(len(B[0])):\n C[i][j] += A[i][k] * B[k][j]",
 },
 {
 label: "Нахождение минимального пути (DP)",
 tip: "Используется динамическое программирование.",
 code: "def min_path_cost(grid):\n n, m = len(grid), len(grid[0])\n dp = [0] * m\n for i in range(1, n):\n for j in range(1, m):\n dp[j] = min(dp[j-1] + grid[i][j-1], dp[j] + grid[i][j], dp[j+1] + grid[i][j+1])",
 },
],
},
{
 group: "Графы",
 items: [
 {
 label: "Поиск в ширину (BFS)",
 tip: "Используется очередь (deque).",
 code: "from collections import deque\ngraph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}\nq = deque(['A'])\nvisited = set()",
 },
 {
 label: "Поиск в глубину (DFS)",
 tip: "Используется рекурсия или стек (list).",
 code: "graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}\ndef dfs(node):\n visited.add(node)\n for neighbor in graph[node]:\n if neighbor not in visited:\n dfs(neighbor)",
 },
],
},
{
 group: "Динамическое программирование",
 items: [
 {
 label: "Таблица",
 tip: "Хранит результаты подзадач.",
 code: "dp = [0] * (n + 1)",
 },
 {
 label: "Память",
 tip: "Ограничение на использование памяти.",
 code: "def fib(n):\n if n < 2:\n return n\n return fib(n - 1) + fib(n - 2)",
 },
],
},
{
 group: "Сортировки",
 items: [
 {
 label: "Быстрая сортировка (QuickSort)",
 tip: "Среднее время O(n log n).",
 code: "def quicksort(arr):\n if len(arr) < 2:\n return arr\n pivot = arr[len(arr) // 2]\n left = [x for x in arr if x < pivot]\n right = [x for x in arr if x > pivot]\n return quicksort(left) + [pivot] + quicksort(right)",
 },
 {
 label: "Сортировка слиянием (MergeSort)",
 tip: "Всегда O(n log n).",
 code: "def mergesort(arr):\n if len(arr) < 2:\n return arr\n mid = len(arr) // 2\n left = mergesort(arr[:mid])\n right = mergesort(arr[mid:])\n return merge(left, right)",
 },
],
},
{
 group: "Алгоритмы",
 items: [
 {
 label: "Нахождение кратчайшего пути (Dijkstra)",
 tip: "Используется куча (heapq).",
 code: "import heapq\ngraph = {'A': {'B': 1, 'C': 4}, 'B': {'A': 1, 'D': 2}, 'C': {'A': 4, 'D': 3}, 'D': {'B': 2, 'C': 3}}\ndist = {node: float('inf') for node in graph}\ndist['A'] = 0\npq = [(0, 'A')]\nwhile pq:\n d, u = heapq.heappop(pq)\n if d > dist[u]:\n continue\n for v, w in graph[u].items():\n nd = d + w\n if nd < dist[v]:\n dist[v] = nd\n heapq.heappush(pq, (nd, v))",
 },
 {
 label: "Нахождение минимального остовного дерева (Kruskal)",
 tip: "Используется сортировка по весу.",
 code: "graph = {'A': {'B': 1, 'C': 4}, 'B': {'A': 1, 'D': 2}, 'C': {'A': 4, 'D': 3}, 'D': {'B': 2, 'C': 3}}\nedges = [(u, v, w) for u, v, w in graph.items() for neighbor, weight in graph[u].items() if u < v]\nedges.sort()\nparent = {node: node for node in graph}\ndef find(u):\n while parent[u] != u:\n u = parent[u]\n return u\ndef union(u, v):\n pu, pv = find(u), find(v)\n if pu != pv:\n parent[pu] = pv",
 },
],
},
{
 group: "Матрицы",
 items: [
 {
 label: "Умножение матриц",
 tip: "O(n^3) для наивного алгоритма.",
 code: "def multiply(A, B):\n n = len(A)\n m = len(B)\n C = [[0] * m for _ in range(n)]\n for i in range(n):\n for j in range(m):\n for k in range(len(B[0])):\n C[i][j] += A[i][k] * B[k][j]",
 },
 {
 label: "Нахождение минимального пути (DP)",
 tip: "Используется динамическое программирование.",
 code: "def min_path_cost(grid):\n n, m = len(grid), len(grid[0])\n dp = [0] * m\n for i in range(1, n):\n for j in range(1, m):\n dp[j] = min(dp[j-1] + grid[i][j-1], dp[j] + grid[i][j], dp[j+1] + grid[i][j+1])",
 },
],
},
{
 group: "Графы",
 items: [
 {
 label: "Поиск в ширину (BFS)",
 tip: "Используется очередь (deque).",
 code: "from collections import deque\ngraph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}\nq = deque(['A'])\nvisited = set()",
 },
 {
 label: "Поиск в глубину (DFS)",
 tip: "Используется рекурсия или стек (list).",
 code: "graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}\ndef dfs(node):\n visited.add(node)\n for neighbor in graph[node]:\n if neighbor not in visited:\n dfs(neighbor)",
 },
],
},
{
 group: "Динамическое программирование",
 items: [
 {
 label: "Таблица",
 tip: "Хранит результаты подзадач.",
 code: "dp = [0] * (n + 1)",
 },
 {
 label: "Память",
 tip: "Ограничение на использование памяти.",
 code: "def fib(n):\n if n < 2:\n return n\n return fib(n - 1) + fib(n - 2)",
 },
],
},
{
 group: "Сортировки",
 items: [
 {
 label: "Быстрая сортировка (QuickSort)",
 tip: "Среднее время O(n log n).",
 code: "def quicksort(arr):\n if len(arr) < 2:\n return arr\n pivot = arr[len(arr) // 2]\n left = [x for x in arr if x < pivot]\n right = [x for x in arr if x > pivot]\n return quicksort(left) + [pivot] + quicksort(right)",
 },
 {
 label: "Сортировка слиянием (MergeSort)",
 tip: "Всегда O(n log n).",
 code: "def mergesort(arr):\n if len(arr) < 2:\n return arr\n mid = len(arr) // 2\n left = mergesort(arr[:mid])\n right = mergesort(arr[mid:])\n return merge(left, right)",
 },
],
},
{
 group: "Алгоритмы",
 items: [
 {
 label: "Нахождение кратчайшего пути (Dijkstra)",
 tip: "Используется куча (heapq).",
 code: "import heapq\ngraph = {'A': {'B': 1, 'C': 4}, 'B': {'A': 1, 'D': 2}, 'C': {'A': 4, 'D': 3}, 'D': {'B': 2, 'C': 3}}\ndist = {node: float('inf') for node in graph}\ndist['A'] = 0\npq = [(0, 'A')]\nwhile pq:\n d, u = heapq.heappop(pq)\n if d > dist[u]:\n continue\n for v, w in graph[u].items():\n nd = d + w\n if nd < dist[v]:\n dist[v] = nd\n heapq.heappush(pq, (nd, v))",
 },
 {
 label: "Нахождение минимального остовного дерева (Kruskal)",
 tip: "Используется сортировка по весу.",
 code: "graph = {'A': {'B': 1, 'C': 4}, 'B': {'A': 1, 'D': 2}, 'C': {'A': 4, 'D': 3}, 'D': {'B': 2, 'C': 3}}\nedges = [(u, v, w) for
```



```

 () =>
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\s*(?:,|\s\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/))
)],
 [sync]
);
const syncVarSet = useMemo(() => new Set(syncVarBases));
/** Полный эталон: глаз сразу вставляет его в обычный код */
const template = useMemo(() => buildSyncTemplate(chapterId));
/** Дефолт редактора: только инициализация демо; она не работает */
const initTemplate = useMemo(() => buildInitTemplate(chapterId));

const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 if (saved.code === saved.base || saved.code.trim() === "")
 return saved.code;
});
/** «Глаз» – переключатель: эталонный код демо ↔ обычный код */
const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved.code !== saved.base;
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится");
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState("");
/** Принудительный повтор (глаз / Ctrl+Enter), даже если код не готов */
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "highlight" | "none">("vars");
const [outputOpen, setOutputOpen] = useState(true);

```

```

useEffect(() => {
 savedEditors.set(chapterId, { code, base: initTempl
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId
useEffect(() => () => clearVizState(chapterId), [chap

// Смена страницы (или вкладки демо): нетронутый реда
// инициализацию; свой код не затираем – предлагаем с
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev
 prevTplRef.current = { base: initTemplate, full: te
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старо
 setRefOn(false);
 if (code === prev.base || code === prev.full || cod
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
}, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: te
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);

```

```

const locals = { ...afterLocals };
for (const [name, value] of Object.entries(beforeVars)) {
 const scalar = value === null || typeof value !== 'object';
 if (scalar && !assigned.has(name) && name in afterVars) {
 variables[name] = value;
 if (step?.locals[name] !== undefined) locals[name] = step?.locals[name];
 }
}
const changed = changedVars(step?.locals, locals);
return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => changed[name]),
};
}, []));

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!aliveRef.current) return; // панель закрыта
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение трекера
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, debug.source);
}, [debug, publishDebug]);

```

```

 }
 const t = window.setTimeout(() => stepBy(1), 650 /
 return () => window.clearTimeout(t);
}, [playing, debug, stepBy, playbackSpeed]);

const debugRun = useCallback(async (autoPlay = false)
 if (running) return;

 const source = code;
 const input = stdin;
 const request = traceRequest;
 setRunning(true);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setOutputOpen(false);

 // Пустой редактор: Pyodide не гоняем, панель в реж
 // связь с визуализацией снимаем — демо показывает
 if (source.trim() === "") {
 setDebug({ result: { steps: [], error: "", trunca
 setOutput("В редакторе пусто — выполнять нечего.\n
 «глаз».");
 setRuntimeMessage("Пустой код: связь с визуализац
 setOutputOpen(true);
 setRunning(false);
 return;
 }

 setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = input.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();

```

```

 if (result.error) setOutputOpen(true);
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STEPS} шагов. ← → — шаги,
 : "Трасса готова автоматически. ← → — шаги,
);
 }
} catch (error) {
 if (!aliveRef.current) return;
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, truncated: true },
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код — произошла ошибка");
} finally {
 setRunning(false);
}
}, [chapterId, code, stdin, traceRequest, running, runtimeReady]);

// Код выполняется сам: debounce даёт спокойно допечать
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady, runtimeReady]);

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки)
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName))
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 }
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [debug]);

```

```

 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
 }, [curDebugLine]);

 // производные данные шага: что изменилось, в каком п
 // Строка, которую назначает текущий шаг, подсвечивае
 // локали берём со следующего шага трассы (для послед
 // трейсером на возврате из фрейма). Значит на строке
 const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.sr
 }, [debug, curDebug, snapshotAt]);
 const debugChanged = curDebug?.step ? changedVars(cur
 const debugOrdered = curDebug?.step ? orderVars(shown

 /** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
 const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) ste
 setPlaying(true);
 }, [debug, debugRun, playing, running, stepTo]);

 /** Вставка сниппета из шпаргалки в позицию курсора.
 const insertSnippet = useCallback(({snippet: string}) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")
 const start = ta.selectionStart ?? current.length

```

```

 onHeightChange(Math.max(minHeight, Math.min(maxHeight, height)));
 };
 const onUp = () => {
 el.removeEventListener("pointermove", onMove as EventListener);
 el.removeEventListener("pointerup", onUp as EventListener);
 el.removeEventListener("pointercancel", onUp as EventListener);
 };
 el.addEventListener("pointermove", onMove as EventListener);
 el.addEventListener("pointerup", onUp as EventListener);
 el.addEventListener("pointercancel", onUp as EventListener);
 }}
 >
 <div className="absolute left-1/2 top-1/2 h-10 w-10 rounded-400 transition-colors" />
</div>
))

```

```

{/* Левый край меняет ширину на десктопе. */}
{onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину панели"
 title="Потяните, чтобы изменить ширину панели"
 className="hidden lg:block absolute top-0 bottom-0 left-0 right-0"
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX - ev.clientX) * 2);
 onWidthChange(Math.max(380, Math.min(next, 1000)));
 };
 const onUp = () => {
 el.removeEventListener("pointermove", onMove as EventListener);
 el.removeEventListener("pointerup", onUp as EventListener);
 };
 }}
 >
 <div className="absolute left-1/2 top-1/2 h-10 w-10 rounded-400 transition-colors" />
 </div>
)}

```





```

 {running
 <select
 aria-label="Скорость автопрохода"
 value={playbackSpeed}
 onChange={(event) => setPlaybackSpeed(Num
 className="cursor-pointer rounded bg-slate
 >
 {PLAYBACK_SPEEDS.map((speed) => (
 <option key={speed} value={speed}>{speed
)}}
 </select>
 </label>
</div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргал
<div className="shrink-0 border-b border-slate-800
 <div className="flex items-center gap-1 px-2 pt
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-
 }`}
 >
 <Variable className="w-3.5 h-3.5" /> Перемен
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text
 }`}
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подс
 </button>
 <div className="ml-auto pr-1">
 <Tooltip

```

```

 {chapterTitle}
 </div>
)}
 </div>
 {desyncedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница
т заменён). "
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
 >
 <Unlink className="w-3.5 h-3.5 shrink
 Страница сменилась – подставить её пер
 </button>
 </Tooltip>
)}
 {hasViz && (
 <div className="flex flex-wrap items-centr
 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >
 <code

```

```

 <Plus className="w-3 h-3 text-slate-600"
 {s.label}
 </button>
 </Tooltip>
)})
</div>
</div>
)})
 <p className="text-[10px] text-slate-600">Начало
</div>
)}
</div>

```

```

{/* — Редактор (всегда на месте) + трасса + ввод
<div className="flex-1 min-h-0 flex flex-col">
 {/* Заголовок редактора: сразу видно, что выполняем
 <div className="shrink-0 flex items-center gap-1">
 <Pencil className="w-3.5 h-3.5 text-emerald-400" />

 Редактор • выполняется этот текст

 {stale && (
 <span
 className="inline-flex items-center gap-1"
 font-weight="bold" text="amber-300 shrink-0"
 title="Трасса ниже построена по предыдущему коду"
 >
 <Loader2 className="w-2.5 h-2.5 animate-spin" />

)}

 {code.trim() === "" ? "пусто — ничего не выполнено" : ""}

 </div>

 <textarea
 ref={editorRef}
 value={code}
 />

```

```

 <button type="button" aria-label="В нач
 }} disabled={!curDebug?.step} className="p-
 transition-colors">
 <SkipBack className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="Шаг назад (←) – автопро
 <button type="button" aria-label="Шаг на
 bled={!curDebug?.prev} className="p-1 round
 tion-colors">
 <ChevronLeft className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="Шаг вперёд (→) – автопро
 <button type="button" aria-label="Шаг в
 abled={!curDebug?.step || debug.idx >= debug
 e hover:bg-slate-800 disabled:opacity-40 tra
 <ChevronRight className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="В конец трассы">
 <button type="button" aria-label="В кон
 result.steps.length - 1)); }} disabled={!cur
 d text-slate-400 hover:text-white hover:bg-
 <SkipForward className="w-4 h-4" />
 </button>
</Tooltip>
<span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
) : (
 "шагов нет – исправьте ошибку и начини
)}

{debug.result.truncated && (

```

```

 onClick={stopDebug}
 className="inline-flex shrink-0 items-center gap-2"
 style={fontStyle}
 >
 <CircleStop className="w-3.5 h-3.5" />
 стоп
 <kbd className="rounded border border-gray-200 px-2 py-1" />
 </button>
</Tooltip>
</div>

{traceOpen && (
 <>
 /* листинг с подсветкой текущей строки */
 <div ref={listRef} className="flex-1 min-h-100px" style={fontStyle}>
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-2 py-1
 ${active ? "bg-emerald-500/15 text-emerald-300" : ""}
 `}
 >
 <span
 className={`w-7 shrink-0 select-none text-right
 ${active ? "text-emerald-300" : "text-gray-400"}
 `}
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
 })}
 </div>
)}
</div>

```

```

 </div>
 </div>
 </>
 })
</div>
})
{debug && !showTrace && debug.result.error ? {
 <pre className="shrink-0 max-h-28 overflow-auto py-2 font-mono text-[11px] leading-snug text-[#c0c0c0]">
 {debug.result.error}
 </pre>
} : null}
<div className="shrink-0 border-t border-slate-800" >
 <label htmlFor="python-stdin" className="text-slate-400">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono text-slate-400"
 />
</div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800" >
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between gap-2 text-slate-400"
 >

```

```

 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
 }

const PRESETS = [
 { name: 'Студент Вася', emoji: '🍷', color: 'from-blue', text: 'Я голода после матана!' },
 { name: 'Кодер Семён', emoji: '🍲', color: 'from-violet', text: 'Я поеду за порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber', text: 'Я попрошу сметаны, плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose', text: 'Я попрошу вареников!' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald', text: 'Я попрошу пива для нашего НЛО!' },
 { name: 'Бизнесмен', emoji: '💼', color: 'from-slate', text: 'Я попрошу тартаны.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'from-blue', text: 'Я голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'from-violet', text: 'Я поеду за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber', text: 'Я попрошу сметаны, плиз.', animState: 'idle' },
]);

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 }))
 : [];

```

```

 }, 450);
 }, [queue, isServing]));

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар скучает');
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsServing(true);
 const headGuest = queue[0];

 addLog(` DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c,
 isEating: true,
 }) : c));

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue shake
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c,
 isEating: false,
 }) : c));

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(` ${headGuest.name} поел и ушел сытым. 🍴`);
 }, 400);
 }, 900);
}, [queue, isServing]);

const reset = () => {
 counter = 3;
 setQueue([

```



```

 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={enqueue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 opacity-40 disabled:cursor-not-allowed text-
 :scale-95 transition-all cursor-pointer flex
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНАЯ КУХНЯ */}
 <div className="grid grid-cols-1 md:grid-cols-12 gap-4">

 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`>
 <div className="absolute top-4 left-4 text-[1.2em] font-medium">
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && {liveQ.length}}
 </div>

 <div className="flex-1 flex flex-col justify-between p-4">
 <div className="flex items-end justify-between">
 {/* ПОВАРИХА И КОТЕЛ С СУПОМ */}

```

```

 className={`
 flex flex-col items-center gap-4
 ${customer.animState === 'in' ? 'animate-in' : ''}
 ${customer.animState === 'eat' ? 'animate-eat' : ''}
 ${customer.animState === 'out' ? 'animate-out' : ''}
 `}
 >
 {/* Облачко мыслей */}
 <div className={`
 absolute -top-16 left-1/2 -transform
8 text-center text-slate-200 leading-tight
 ${isHead ? 'opacity-100 border-1px solid black' : ''}
 `}>
 <div className="font-bold text-slate-200">
 {customer.bubbleText}
 <div className="absolute -bottom-16 left-1/2
it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 w-14 h-14 rounded-2xl border-1px solid black
relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-slate-200
 ${isHead ? 'bg-amber-500' : ''}
 `}>
 {isHead && isTail ? 'h+t' : ''}

)}
 </div>

```

```

 0 flex items-center gap-2 text-[11px] text-f
 >
 {item}
 </div>
)))
 </div>
})
</div>

{/* Пол */}
<div className="w-full h-3 bg-gradient-to-r f
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-indigo-5
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect, useContext, useMem
```

```

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveWordsRaw = vars ? vizArray(vars.words) ?? v
const liveWordsSig = useMemo(() => (liveWordsRaw ? JS
const liveState = vars ? vizNumber(vars.state) ?? viz
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null;
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
 if (!liveWordsSig) return;
 const parsed = (JSON.parse(liveWordsSig) as unknown
 .map((w) => (typeof w === "string" ? w : typeof w
 .filter((w) => Boolean(w))
 .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g,
 .filter((w) => w.length > 0)
 .slice(0, 8);
 if (!parsed.length) return;
 if (parsed.join("|") === words.join("|")) return;
 setWords(parsed);
 buildInitialTrie(parsed);
 // eslint-disable-next-line react-hooks/exhaustive-
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[l
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail
 setSimulationStep(liveStep <= 0 ? 0 : failReady ?
 }
}

```

```

 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
}
if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
}
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 });
 }
}

```

```

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить!');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне

 уже обработаны, очередь суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!

 Теперь можно любоваться таблицей переходов!'
);
 }
};

```



```

) : (
 <path d="М 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="М 130 80 Q 125 100 130 120" stro
 <path d="М 120 85 Q 115 100 120 115" stro
 </svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
<span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tra
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leadin
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex
 <Info className="w-3.5 h-3.5 text-blue-40
 Статус: {simulationStep === 0 ? 'Бор рот

```



```
<div className="mb-8 bg-slate-900 border border-slate-800" style={{ padding: 10px }}>
 <h5 className="text-white font-bold mb-3 flex items-center">
 Дерево и Суффиксные (fail) ссы
 </h5>
```

```
{(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6" fill="white">
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 <marker id="fail-arrow" markerWidth="6" markerHeight="6" fill="white">
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 </defs>

 <!-- Draw Edges -->
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 x2={node.x}
 y1={parent.y}
 y2={node.y}
 stroke={isCurrent ? 'white' : 'gray'}
 stroke-width={2}
 />
 <marker
 id={isCurrent ? 'arrowhead' : 'fail-arrow'}
 x={node.x}
 y={node.y}
 angle={Math.atan2(node.y - parent.y, node.x - parent.x) * 180 / Math.PI}
 />
 </g>
);
 })}
 </svg>
);
})}

</div>
```

```

 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
 }}}

```

```

{/* Draw Nodes */}
Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

```

```

 let fill = '#1e293b';
 let stroke = '#475569';

```

```

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

```

```

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}

```

```

 {words.map({w} => {
 <span
 key={w}
 className="bg-slate-800 border border-
er gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:tex
 title="Удалить"
 >
 ×
 </button>

)}}
 </div>
</div>

<form onSubmit={handleAddWord} className="fle
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.valu
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-sla
 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-60
 ition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

```

```

 : 'hover:bg-slate-800/40'
 }` }
 >
 <td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full
 ${node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'}
 }` }
 >
 #{node.id}
 {node.id === 0 &&
 </td>
 <td className="py-2.5 px-3" >
 <span className="bg-slate-800 border border-1
 {node.char}

 </td>
 <td className="py-2.5 px-3 text-slate-500" >
 {node.id === 0 ? '-' : `#${node.id}`}
 </td>
 <td className="py-2.5 px-3" >
 {node.id === 0 ? (

) : (

 #{node.fail}

)}
 </td>
 <td className="py-2.5 px-3" >
 {node.id === 0 || node.term_line === 0 ? (


```

```

 </>
 {liveWordsSig && words = {words.join(", ")}
 {liveNodes !== null && вершин в боре = {liveNodes}
 {liveState !== null && state = {liveState}
 {liveStep !== null && шаг BFS = {liveStep}
 {liveQueue && queue = {JSON.stringify(liveQueue).slice(0, 100)}
 </>
) : (

 открой панель Python: массив <code className="font-mono text-cyan-300">state</code>/<code className="font-mono text-cyan-300">queue</code>

)}
 </div>
</div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber, vizArray } from '../viz-runtime';

```

```

// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
 codeLine: number;
}

```

```

 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
];
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r})`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r})`,
 highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] = tree[${2 * v}] + tree[${2 * v + 1}]`,
 tate: { ...tree }, highlightNodes: [v], mergeChildren: [2 * v, 2 * v + 1] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень: ${arr[0]}`,
 Highlight: [0, n - 1] });
return steps;
}

```

## // 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

```

```

let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l]`});

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res }, { id: id++, desc: `Сумма = ${res}. l++.`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l], result: res });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res }, { id: id++, desc: `Сумма = ${res}. r--`, codeLine: 5, treeState: tree, highlightNodes: [r], arrayHighlight: [r], result: res });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${qL + n}, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res`});
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], ${qL}, ${qR})`, codeLine: 5, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val: number; }

```

```

" if (r & 1) res += tree[--r];",
" l >>= 1; r >>= 1;",
" return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' } }[color];

 return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr: number[] }) {
 const p = Player({ steps, color });
 const runtime = useVizRuntime();

```



```


 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt-2" >
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full" >
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
 </div>
);
}, [step, clr, fullTree, liveV, liveTree]);

return (
 <div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 }/* Header */>
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-b-2 border-emerald-500/30`>
 <h4 className="font-bold text-sm text-white flex-1">Исходный код

 </div>

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b-2 border-emerald-500/30">
 <div className="text-[11px] font-bold text-slate-400">
 Исходный мас
 <span className="text-slate-400 font-mono tex
 </div>
 <div className="flex flex-wrap gap-1 justify-center">
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >=
 const isLiveI = liveI === i;
 return (
 <div key={i} className={`flex flex-col it
 -amber-400 ring-2 ring-amber-400/40' : inRa

```

```

 ' : color === 'emerald' ? 'text-emerald-300
 = {step.result}

 })
 </div>

 { /* Controls */ }
 <div className="px-3 py-2.5 bg-slate-950 border-t
 <div className="flex items-center bg-slate-900
 <button onClick={() => { setPlaying(false); s
 unded" title="Сброс"><RotateCcw className="
 <button onClick={() => { setPlaying(false); s
 Шаг назад" className="p-1.5 text-slate-400
 Name="w-3.5 h-3.5" /></button>
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-5
 {playing ? <Pause className="w-3 h-3" /> :
 </button>
 <button onClick={() => { setPlaying(false); s
 р вперед" title="Шаг вперед" className="p-1
 <ChevronRight className="w-3.5 h-3.5" /></b
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

```

```

const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.f
 setArr(a);
 setCustomInput(a.join(', '));
};

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSte
const queryBUSteps = useMemo(() => genQueryBottomUpSte

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border bor
 /* ---- TOP BAR: array editor + query range ----
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4
 /* Array input */
 <form onSubmit={handleApply} className="flex-
 <div className="flex items-center justify-b
 <label className="text-[10px] font-bold up
 label>
 <button type="button" onClick={randomize}
 "><RefreshCw className="w-3.5 h-3.5" /></bu
 </div>
 <div className="flex gap-2">
 <input
 value={customInput}
 onChange={e => setCustomInput(e.target.
 className="flex-1 bg-slate-900 border b
 one focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indig
 ition-colors">Применить</button>
 </div>
 <div className="flex flex-wrap gap-1.5 pt-1

```

```

 </div>
 <div className="text-[10px] text-slate-500" style={style}>
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{\text{arr.slice}(qL, qR)\}$
 </div>
 </div>
 </div>
 </div>

 {/* ---- TABS ---- */}
 <div className="flex flex-wrap items-center gap-1" style={style}>
 <button onClick={() => setView('build')} className="border border-indigo-500 text-slate-200" style={style}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} className="border border-emerald-500 text-slate-200" style={style}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('theory')} className="border border-blue-500 text-slate-200" style={style}>
 <Info className="w-3.5 h-3.5" /> Почему / Сра
 </button>
 </div>

 {/* ---- TAB CONTENT ---- */}
 {view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=""
 steps={buildSteps}
 code={BUILD_CODE}
 color="indigo"
 arr={arr}
 />
)}

```

```

 <div className="text-slate-400">Корень: L
 <div className="text-slate-300">mid = {(0
 <div className="text-indigo-300">→ Левый:
 <div className="text-emerald-300">→ Правый:
h - 1) >> 1)) эл.)</div>
 <div className="text-slate-500 mt-2">Всего
 } внутренних.</div>
</div>
</div>

{/* Comparison cards */}
<div className="grid grid-cols-1 xl:grid-cols-2"
 <div className="bg-slate-950 p-5 rounded-xl"
 <div className="absolute -top-3 left-4 bg-emerald-500
e border border-emerald-500 shadow-md">
 Запрос Сверху-вниз (Рекурсия)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Начинаем с корня. Если отрезок узла полн
 наче спускаемся в обоих детей.
 </p>
 <div className="space-y-1.5 text-xs">
 <div className="flex justify-between border-b border-slate-700"
span className="text-rose-400 font-mono">0(
 <div className="flex justify-between border-b border-slate-700"
>
 <div className="flex justify-between">✓ легко</div>
 </div>
 </div>
 <div className="bg-slate-950 p-5 rounded-xl"
 <div className="absolute -top-3 left-4 bg-amber-500
rder border border-amber-500 shadow-md">
 Запрос Снизу-вверх (Итерация)
 </div>
 <p className="text-xs text-slate-300 mb-3">
 Стартуем с двух указателей (l и r) на л
 ём его левого соседа. Поднимаемся пока l {"

```

```

interface SidebarProps {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Мобильный режим: список живёт в выдвижной панели */
 onNavigate?: () => void;
 /** Узкий режим: номер билета и название в одну строку */
 compact?: boolean;
}

export const Sidebar: React.FC<SidebarProps> = ({ selectedChapterId,
 const [query, setQuery] = useState("");

 /**
 * Группы – это разделы пособия (SECTIONS), а не случайные
 * связанные темы стоят рядом, порядок разделов не зависит от
 * страница встретилась первой.
 */
 const groups = useMemo(() => {
 const q = query.trim().toLowerCase();
 const filtered = q ? chapters.filter((c) => c.title.toLowerCase().includes(q)) : chapters;
 return SECTIONS.map((section) => ({
 section,
 items: filtered.filter((c) => sectionOf(c.id).id === section.id).filter((g) => g.items.length > 0);
 })), [query]);

 return (
 <aside className="w-full bg-slate-900/80 lg:bg-transparent lg:rounded-r-2xl" >
 <div className={compact ? "p-2 pb-1.5 shrink-0" : "p-2 pb-1.5 shrink-0"} >
 {compact && (
 <h3 className="text-[10px] font-bold text-slate-400" >
 <Compass className="w-3.5 h-3.5 text-indigo-400" />
 </h3>
)}
 <h3 className={compact ? "text-xs font-bold text-slate-400" : "text-sm font-normal text-slate-400"} >
 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 </div>
 </aside>
);
}

```

```

</div>
<div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 // «18–20. Графы. Основное дерево...» → 3
 const num = chapter.title.match(/^(\d+)?/)[1]
 const label = num ? chapter.title.slice(0, num) : chapter.title
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id)
 onNavigate?.()
 window.scrollTo({ top: 0, behavior: 'smooth' })
 }}
 title={chapter.title}
 aria-current={isSelected ? "page" : undefined}
 className={`w-full text-left rounded-md
 ${isSelected ? "bg-indigo-600/15 border-indigo-600/15" : "bg-slate-800/40 border-transparent"}
 `>
 <div>

 {num ? num[1] : ""}

 {label}
 {hasViz && (

)}

 {!compact && (
 <ChevronRight

```

```

 id: string;
 text: string;
 probability: number;
}

export const Simulator: React.FC = () => {
 const [activeTab, setActiveTab] = useState<'stack' |

 // Stack state
 const [stack, setStack] = useState<Plate[]>([
 { id: '1', name: 'Тарелка из-под пасты', icon: ' ' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: ' ' },
 { id: '3', name: 'Блюдец от торта', icon: ' ' },
]));
 const [popMessage, setPopMessage] = useState<string |

 // Queue state
 const [queue, setQueue] = useState<Person[]>([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]));
 const [dequeueMessage, setDequeueMessage] = useState<

 // Heap state
 const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"', },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', },
 { id: '3', text: 'Кандидат: "Привет, я испанский ин"', },
]));
 const [newCustomText, setNewCustomText] = useState('');
 const [newCustomProb, setNewCustomProb] = useState(0.);
 const [heapMessage, setHeapMessage] = useState<string

 // Stack handlers
 const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: ' ' },

```



```

 { name: 'Курьер доставки', icon: ' ' },
 { name: 'Усталый админ', icon: ' ' },
 { name: 'Турист с рюкзаком', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setDequeueMessage(` Встал в конец очереди: ${newPerson.name} `);
};

const servePerson = () => {
 if (queue.length === 0) {
 setDequeueMessage("⚠ Очередь пуста! Суп ждет нового посетителя.");
 return;
 }
 const firstPerson = queue[0];
 setQueue(prev => prev.slice(1));
 setDequeueMessage(` Выдан суп первому в очереди (Фамилия: ${firstPerson.name}). `);
};

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {

```

```

return (
 <div className="bg-slate-900/90 border border-slate
 <div className="border-b border-slate-800 pb-6 mb
 <h2 className="text-2xl font-bold text-transpar
 p-3">
 Интерактивный симулятор структур данных
 </h2>
 <p className="text-slate-400 text-sm mt-2">
 Проверь работу принципов LIFO, FIFO и Приоритет
 </p>
 </div>

 { /* Tabs */ }
 <div className="grid grid-cols-1 md:grid-cols-3 g
 <button
 onClick={() => setActiveTab('stack')}
 className={`flex items-center justify-between
 activeTab === 'stack'
 ? 'bg-blue-600/20 border-blue-500 text-bl
 : 'bg-slate-800/60 border-slate-700/60 te
 }`}
 >
 <div className="flex items-center gap-3">
 <Layers className={`w-5 h-5 ${activeTab ===
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>
 </div>
 <span className="text-xs bg-blue-500/20 text-l
 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'

```

```

{activeTab === 'stack' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция стопки тарелок
 <span className="text-xs font-mono bg-b
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Новые тарелки ставятся наверх. Мыть мож
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPlate}
 className="flex-1 md:flex-none flex ite
 rounded-xl text-sm font-bold shadow-lg tra
 >
 <Plus className="w-4 h-4" /> Поставить
 </button>
 <button
 onClick={popPlate}
 className="flex-1 md:flex-none flex ite
 border-blue-500/40 px-4 py-2.5 rounded-xl
 >
 Помыть верхнюю
 </button>
 <button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>
 </div>
}

```

```

 >
 <div className="flex items-center"
 {plate.name}
 <span className={`font-bold text-${plate.name}
 {plate.name}
 </div>
 <span className={`text-xs font-mono
 isTop ? 'bg-blue-500 text-white' : ''}>
 {isTop ? 'Готово к помывке' : ''}
 </div>
);
 }
 }
 </div>
)
<div className="mt-6 text-xs text-slate-500"
 ПОЛОЖИЛ ПОСЛЕДНЕЙ → ПОМЫЛ ПЕРВОЙ (LIFO) *
</div>
</div>
</div>
))

```

```

{/* QUEUE TAB */}
{activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-white">
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-in
 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1">
 Кто первым пришел, тот первым получил суп
 </p>

```

```

{queue.length === 0 ? (
 <div className="text-center py-12 text-slate-500">
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Очередь пуста
 <p className="text-xs mt-1">Добавьте нового человека
 </div>
) : (
 <div className="w-full flex items-center justify-between">
 <div className="flex flex-col items-center justify-between w-300">

 Очередь

 </div>

 <div className="text-indigo-500 flex items-center justify-center">
 <ArrowRight className="w-6 h-6 animate-pulse">
 </div>

 {queue.map((person, index) => {
 const isFirst = index === 0;
 return (
 <div
 key={person.id}
 className={`flex flex-col items-center justify-between w-300
 ${isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-white
relative'
 : 'bg-slate-900/90 border-slate-500'}
 `}
 >
 {isFirst && (
 <span className="absolute -top-12 left-1/2 transform translate-x-50%
se tracking-wider shadow">
 Первый (Head)

)}
 {person.name}


```

```

 </p>
 </div>
 <div className="flex items-center gap-3 w-full">
 <button
 onClick={popHeap}
 className="flex-1 md:flex-none flex items-center justify-center gap-2 py-2.5 rounded-xl text-sm font-bold shadow-sm"
 >
 Взять кандидата с макс. вероятностью
 </button>
 <button
 onClick={resetHeap}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-slate-700 transition-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
</div>

{/* Add Form */}
<form onSubmit={addHypothesis} className="bg-slate-12 gap-4 items-end">
 <div className="md:col-span-6">
 <label className="block text-xs font-bold">Напишите текст гипотезы
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target.value)}
 placeholder="например: 'Привет, я лучший!'"
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
 <div className="md:col-span-3">
 <label className="block text-xs font-bold">Вероятность:
 {probability}
 </div>
</form>

```

```

 <p className="text-base font-bold">Куча
 <p className="text-xs mt-1">Добавьте ги
 </div>
) : (
 <div className="w-full max-w-2xl space-y-1">
 <div className="flex items-center justify-between">
 КАНДИДАТ / ГИПОТЕЗА
 ВЕРОЯТНОСТЬ (ПРИОРИТЕТ)
 </div>

 {sortedHeap.map(({item, index}) => {
 const isTop = index === 0;
 const percent = (item.probability * 100).toFixed(
 return (
 <div
 key={item.id}
 className={`flex items-center justify-between ${
 isTop
 ? 'bg-gradient-to-r from-emerald-500 to-emerald-
00/10 scale-[1.01]`
 : 'bg-slate-900/80 border-slate-800`
 }`}
 >
 <div className="flex items-center justify-between">
 <span className={`flex items-center justify-between ${
 isTop ? 'bg-emerald-500 text-white' : 'text-slate-200'
 }`}>
 #{index + 1}

 <div>
 <div className={`font-bold text-white ${
 isTop ? 'text-white' : 'text-slate-200'
 }`}
 {item.text}
 </div>
 {isTop && (
 <span className="text-[10px] font-normal text-white"
 -0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд


```

```

import React, { useContext, useEffect, useState } from
import { VizChapterContext, emitVizDemo } from "../data
import { SplayRotationsViz } from "./SplayRotationsViz"
import SplayWalkViz from "./SplayWalkViz";
import { SplayRotationSandbox } from "./SplayRotationSan

/**
 * Страница splay – три вкладки по нарастанию сложности
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Z
 * (прицел → поворот, призраки старых позиций, обхо
 * 2. «Большое дерево» – те же случаи внутри настоящего
 * поворот – кадры «прицел → отстёгиваем среднее по
 * 3. «Собери сам» – песочница: подними узел кликами,
 */
type Tab = "cases" | "big" | "play";

export const SplayPlayground: React.FC<{ defaultTab?: T
 const [tab, setTab] = useState<Tab>(defaultTab);
 const chapterId = useContext(VizChapterContext);

 // вкладка = подрежим страницы: панель Python подстав
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl bor
 <div className="flex flex-wrap items-center justifi
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-whit

 </div>
 <div>
 <h3 className="text-2xl font-bold text-whit
 <p className="text-sm text-indigo-300">Один
 </div>
 </div>

```



```
import React, { useCallback, useMemo, useState } from "react";
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, Undo } from "lucide-react";
```

```
/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерево,
 * поднять отмеченный узел в корень. Клик по узлу = оди
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */
```

```
export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null });
```

```
const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null;
```

```
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
```

```
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf(3) };
 const n4: TNode = { key: 4, left: n2, right: leaf(5) };
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
};
```

```

 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

```

```

 width: cols * colW,
 height: rows * rowH + 20,
 };
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.l
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; co
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() =>
 const path = useMemo(() => new Set(findPath(tree, tar
 const expected = useMemo(() => (solved ? null : canon

 const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с ке

```

```

 {PRESETS.map((p, i) => {
 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11pt]
 i === presetIdx ? "bg-indigo-600 text-white" : "bg-white text-indigo-600"
 }`}
 >
 {p.name}
 </button>
 })}
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800" >
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
 />
);
 })}
 </div>
 </div>

```

```

 >
 <Undo2 className="w-3.5 h-3.5" /> Отменить
 </button>
 <button
 onClick={() => reset()}
 className="flex items-center gap-1.5 px-3 py-2
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Заново
 </button>
 <button
 onClick={() => setShowAnswer((s) => !s)}
 className={`flex items-center gap-1.5 px-3 py-2
 showAnswer
 ? "bg-amber-600/20 border-amber-500 text-slate-500"
 : "bg-slate-800 border-slate-700 text-slate-500"
 }`
 >
 {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
 {showAnswer ? "Скрыть разбор" : "Сдаться / посмотреть"}
 </button>

 поворотов: {moves.length} • минимум: {minimalMoves}

 </div>

 {showAnswer && !solved && expected && (
 <div className="mt-3 text-[12px] bg-amber-950/40 p-3" >
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
)}

 {solved && (
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5
 wrong === 0

```

```
import React, { useEffect, useMemo, useRef, useState }
import { Gauge, Pause, Play, RotateCcw, Terminal } from
import { useVizRuntime, vizNumber, vizString } from "..
```

```
/**
```

```
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zig-Zag
```

```
 *
```

```
 * Что сделано ради понятности:
```

```
 * * у узлов настоящие числа (20 / 40 / 60), а не абстрактные
```

```
 * сразу видно, что дерево остаётся деревом поиска;
```

```
 * * под деревом печатается обход слева направо: он ОДНОУРОВНЕВНЫЙ
```

```
 * это и есть доказательство, что поворот ничего не ломает
```

```
 * * кадр «прицел» ничего не двигает, только подсвечивает
```

```
 * * на кадре «результат» остаются призраки – пунктирные
```

```
 * местах и стрелки «откуда → куда»;
```

```
 * * по умолчанию анимация НЕ крутится сама: пользователь
```

```
 * идёт в своём темпе. Автопрокрутка включается кнопкой
```

```
 */
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
```

```
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
```

```
 pos: Pos;
```

```
 edges: [NodeId, NodeId][];
```

```
 kind: "start" | "aim" | "result";
```

```
 title: string;
```

```
 text: string;
```

```
 /** Пара, которую крутим на этом шаге: кадр-«прицел».
```

```
 focus?: [NodeId, NodeId];
```

```
 /** Кто переехал на этом кадре – рисуем призрак и стрелки
```

```
 moved?: NodeId[];
```

```
 ms: number;
```

```
}
```

```
interface Case {
```

```
 key: "zig" | "zig-zig" | "zig-zag";
```

```
 label: string;
```

```

 {
 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 – 20",
 text: "Ничего пока не двигается. Смотри на подсвеченное ребро",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p", "C"}],
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддеревом от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C: [212, 112],
 edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g: [255, 40],
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
};

```

```

edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 в корне, ветка x подтянулась",
text: "Сравни с первым кадром: A было на глубине
 ась. Splay не «уменьшает высоту вообще», он под
 сумме запросов (амортизация).",
moved: [{"x", "p", "A", "B"}],
ms: RESULT_MS,
},
],
};

```

```

/* ----- Zig-Zag -----
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p — дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым — нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
",
 }
]
}

```



```

const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];

const COLOR: Record<string, { fill: string; stroke: string }> = {
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};

const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id === "B";

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C }> = ({
 frame,
 prev,
 keys,
}) => {
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
 const ghosts = frame.kind === "result" ? (moved ?? []) : [];

 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" refX="10" refY="0">
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

 {/* призраки: где узел стоял до поворота */}
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 <g>
 <rect
 x={from[0]} y={from[1]} width={r} height={r}
 fill={COLOR[from].fill} stroke={COLOR[from].stroke}
 opacity={0.5}
 />
 <rect
 x={to[0]} y={to[1]} width={r} height={r}
 fill={COLOR[to].fill} stroke={COLOR[to].stroke}
 opacity={0.5}
 />
 <path
 d={`M${from[0]},${from[1]} L${to[0]},${to[1]}`}
 stroke={COLOR[from].stroke}
 stroke-width={2}
 opacity={0.5}
 />
 <rect
 x={to[0]} y={to[1]} width={r} height={r}
 fill={COLOR[from].fill} stroke={COLOR[from].stroke}
 opacity={0.5}
 />
 </g>
 })}
 </svg>
);
};

```

```

 });
 }}}

 {(Object.keys(pos) as NodeId[]).map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id))
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke=
 <circle r={r} fill={c.fill} stroke={c.stroke}
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font=
 {id}
 </text>
)}
 </g>
);
 })}
}
```



```

 className={`px-1.5 py-0.5 rounded font-mono text-sm
 /\d/.test(t) ? "bg-emerald-500/15 text-white" : "text-slate-300"
 }`}
 >
 {t}

)))

 </div>
 <p className="text-[11px] text-slate-500 mt-1">
</div>
</div>

{/* индикатор автопрокрутки */}
<div className="h-1 w-full bg-slate-800 rounded-f
 <div
 key={` ${caseIdx} - ${frame} - ${playing} - ${slow} `}
 className="h-full bg-indigo-500"
 style={
 playing
 ? { animation: `demoProgress ${duration}m
 : { width: "100%", background: "#334155"
 }
 />
 </div>

<div className="flex flex-wrap items-center gap-2"
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800
 d:hover:text-slate-300 text-xs font-bold tr
 >
 ← Назад
 </button>

```

```

</button>

<div className="flex items-center gap-3 ml-auto
 {[["x", "p", "g"] as const].map((id) =>
 active.keys[id] === undefined ? null : {
 <span key={id} className="flex items-cent
 <span className="w-3 h-3 rounded-full"

 }
)}
</div>
</div>

{/* что сейчас сообщил компилятор */}
<div className="mt-3 pt-3 border-t border-slate-8
 <span className="text-[10px] uppercase tracking
 <Terminal className="w-3.5 h-3.5" /> из компи

 {liveCase || liveX !== null || liveStep !== nul
 <div>
 {liveCase && {
 <span className="text-[11px] font-mono bg
 case = {liveCase}

 }}
 {liveX !== null && <span className="text-[1
 py-1">x = {liveX}}
 {liveP !== null && <span className="text-[1
 py-1">p = {liveP}}
 {liveG !== null && <span className="text-[1
 py-1">g = {liveG || "нет (Zig)"}}
 {liveStep !== null && <span className="text
 -2 py-1">step = {liveStep}}
 </div>
) : {

 запусти код из панели Python: переменная <s
 случай,

```

```

}

export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
 n.right = r;
 r.parent = n;
 }
 return n;
}

export function bstInsert(root: SN | null, key: number): SN | null {
 if (!root) return { key, left: null, right: null, parent: null };
 const r = cloneWithParents(root)!;
 let cur: SN = r;
 for (;;) {
 if (key < cur.key) {
 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: cur };
 return r;
 }
 cur = cur.right;
 }
 }
}

```

```

const rm = before.filter((e) => !after.includes(e));
const ad = after.filter((e) => !before.includes(e));
const parts: string[] = [];
for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
}
return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |
export interface SplayStep {
 kind: SplayStepKind;
 root: SN | null;
 aimEdge?: [number, number];
 /** Кадр cut: ребро, по которому отстёгивается среднее */
 cutEdge?: [number, number];
 cutLabel?: string;
 /** Куда β переедет: сторона у верхнего узла. */
 cutDest?: string;
 pathKeys?: number[];
 caseLabel?: string;
 note: string;
 rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
 rootIn: SN | null,
 key: number,
 isInsert = false,
): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root)
 return { steps: [{ kind: "done", root: null, note:

```

```

 aimEdge,
 caseLabel: label,
 note: aimNote,
 rotCount,
 });
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.left;
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `β = поддереву ${mid.key}` : "β пуст",
 cutDest: mid ? (p.left === child ? "справа у p" : "слева у p") : "никуда",
 caseLabel: label,
 note: mid
 ? `Отстёгиваем среднее поддерево β = ${mid.key}
 и у них общий — после поворота переедет ${p.key}
 : `Среднего поддерева нет (β пусто): ${child.key}
 rotCount,
 });
 const before = collectEdges(root);
 rotateUp(child);
 fixRoot();
 rotCount += 1;
 const diff = diffEdges(before, collectEdges(root));
 steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 note: `Поворот: ${child.key} встал над ${p.key}.
 rotCount,
 });
};

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {

```



```

 [p.key, target.key],
 `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: по
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zag",
 note: `Второе ребро: ${target.parent.key}—${target.key}
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zag",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key}
);
}
}

steps.push({
 kind: "done",
 root: cloneWithParents(root)!,
 note: ` ${target.key} в корне за ${rotCount} ${rotCount}
 rotCount,
});
return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
 let d = 0;
 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }

```

```

<svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font
 <line x1="55" y1="46" x2="30" y2="86" stroke=
 <line x1="55" y1="46" x2="80" y2="86" stroke=
 <line x1="30" y1="86" x2="12" y2="126" stroke=
 <line
 x1="30" y1="86" x2="48" y2="126"
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 3 : 2}
 strokeDasharray={isCut ? "5,3" : "none"}
 />
 <circle cx="55" cy="46" r="15" fill="#1e293b"
 <text x="55" y="50" textAnchor="middle" fill=
 <circle cx="30" cy="86" r="15" fill="#78350f"
 <text x="30" y="90" textAnchor="middle" fill=
 <circle cx="12" cy="126" r="13" fill="#0f172a"
 <text x="12" y="130" textAnchor="middle" fill=
 <circle
 cx="48" cy="126" r="13"
 fill={isCut ? "#78350f" : "#0f172a"}
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 2.5 : 1.5}
 strokeDasharray={isCut ? "4,3" : "none"}
 />
 <text x="48" y="130" textAnchor="middle" fill=
 <circle cx="80" cy="126" r="13" fill="#0f172a"
 <text x="80" y="130" textAnchor="middle" fill=
 <text x="98" y="92" fill={isCut ? "#f59e0b" :
 {isCut ? " β отсёрнута" : "β при x"}}
 </text>
 <text x="16" y="160" fill="#64748b" fontSize=
 </g>
 <text x="163" y="96" fill="#10b981" fontSize="20"
 <g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" fontSize="11" font
 <line x1="70" y1="46" x2="45" y2="86" stroke=
 <line x1="70" y1="46" x2="95" y2="86" stroke=

```

```

const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
 walk(root, 0);
 return pos;
};

const SplayWalkViz: React.FC = () => {
 const initial = useMemo(buildInitial, []);
 const [steps, setSteps] = useState<SplayStep[] | null>(null);
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [input, setInput] = useState("");
 const [treeVers, setTreeVers] = useState(0);
 const baseRef = useRef<SN>(initial);
 const timer = useRef<ReturnType<typeof setTimeout> | null>(null);

 const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key, isInsert);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
 };

 useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 }
 }, [playing, steps, idx]);

```

```

 return new Set(collectEdges(steps[idx - 1].root));
 }, [steps, step, idx]);
const movedCount = useMemo(() => {
 if (!prevEdgeSet) return 0;
 return edges.filter((e) => !prevEdgeSet.has(`${e.a}`))
 .length;
}, [prevEdgeSet, edges]);

const ino0 = useMemo(() => (steps ? inorderKeys(steps) : 0));
const inoNow = inorderKeys(viewRoot).join(",");
const checkOk = !steps || ino0 === inoNow;

const doInsert = () => {
 const v = parseInt(input, 10);
 if (Number.isNaN(v)) return;
 const withNew = bstInsert(baseRef.current, v);
 baseRef.current = withNew;
 setTreeVers((t) => t + 1);
 runKey(v, true);
 setInput("");
};

const stepColor = (k: number): string => {
 if (!step) return "#0f172a";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e[0] === k) return "#1e293b";
 if (e[1] === k) return "#78350f";
) return "#0f172a";
 if (step.kind === "rot" && step.pathKeys?.includes(k)) return "#0f172a";
 return "#0f172a";
};

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#0f172a";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
) return "#64748b";
 return "#64748b";
};

```

```

 onClick={() => {
 baseRef.current = buildInitial();
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers(({v} => v + 1));
 }}
 className="px-3 py-2 rounded-lg text-xs font-medium"
 >
 Исходное
 </button>
 <button
 onClick={() => {
 const shuffled = [...INITIAL_KEYS].sort(() => Math.random() - 0.5);
 let r: SN | null = null;
 for (const k of shuffled) r = bstInsert(r, k);
 baseRef.current = r!;
 setSteps(null);
 setIdx(0);
 setPlaying(false);
 setTreeVers(({v} => v + 1));
 }}
 className="px-3 py-2 rounded-lg text-xs font-medium"
 >
 Перемешать
 </button>
 </div>
 </div>

 {/* Плеер */}
 {steps && (
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setIdx(0); setPlaying(false); }}
 className="px-3 py-2 rounded-lg text-xs font-medium disabled:opacity-50"
 >
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setSteps(null); }}
 className="px-3 py-2 rounded-lg text-xs font-medium disabled:opacity-50"
 >
 Пауза
 </button>
 </div>
)}

```

```

.cutEdge[1];
const isMoved = !isAim && !isCutEdge && pre
return (
 <g key={`e-${e.a}-${e.b}`}>
 <line
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={isAim ? "#f59e0b" : isCutEdge ? "#f59e0b"}
 strokeWidth={isAim ? 4 : isCutEdge ? 2}
 strokeDasharray={isAim ? "6,4" : isCutEdge ? "1,2"}
 className="transition-all duration-500"
 />
 {isCutEdge && (
 <text x={(a.px + b.px) / 2 + 14} y={(a.py + b.py) / 2 + 14}
 nospace" className="transition-all duration-500"
 {step?.cutLabel} → {step?.cutDest}
 </text>
)}
 </g>
);
}})
}
nodes.map((n) => {
 const aimE = step?.aimEdge ?? (step?.kind === "aim" ? step?.aimEdge : null);
 const isAimNode = aimE?.includes(n.key);
 return (
 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`}
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current).includes(n.key);
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 />
 </g>
);
});

```

```

 <div className="w-full lg:w-[300px] shrink-0">
 <MechanismInset step={step} />
 </div>
 </div>
 </div>

 { /* Счётчики + код */ }
 <div className="flex flex-col lg:flex-row gap-6 m-6">
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <h4 className="text-sm font-bold text-rose-300">Шаги
 {steps ? (
 <table className="w-full text-sm">
 <tbody className="font-mono">
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">г.
 <td className="py-2 text-right text-slate-400">
 </td>
 </tr>
 <tr className="border-b border-slate-800">
 <td className="py-2 text-slate-400">в.
 <td className="py-2 text-right text-slate-400">
 </td>
 </tr>
 <tr>
 <td className="py-2 text-slate-400">п.
 <td className="py-2 text-right text-slate-400">
 </td>
 </tr>
 </tbody>
 </table>
) : (
 <p className="text-xs text-slate-400">Занесите
 </p>
 <p className="text-xs text-slate-500 mt-3">Одн
 </p>
 </div>
 </div>
 <div className="w-full lg:w-1/2 bg-slate-900/60">
 <h4 className="text-sm font-bold text-slate-200">Шаги
 <div className="bg-slate-950/80 rounded-lg p-4">
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&

```

```

 animState: 'in' | 'idle' | 'washing' | 'clean';
}

const PLATE_PRESETS = [
 { name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { name: 'Блюдец от торта', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { name: 'Тарелка от тако', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { name: 'Тарелка из-под суши', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { name: 'Сковорода от глазуньи', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
];

let counter = 0;

export const StackViz: React.FC = () => {
 const [dirtyStack, setDirtyStack] = useState<Plate[]>([
 { id: 'p1', name: 'Тарелка из-под спагетти', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { id: 'p2', name: 'Тарелка из-под пиццы', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
 { id: 'p3', name: 'Блюдец от торта', emoji: ' ', color: 'from-indigo-500/20 to-indigo-600/20 border-indigo-500', isDirty: false, animState: 'idle' },
]));

 const runtime = useVizRuntime();
 const liveStack = vizArray(runtime?.variables?.st) ?? [];
 const displayStack: Plate[] = liveStack
 ? liveStack.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-indigo-500/20 to-indigo-600/20 border-indigo-500",
 isDirty: true,
 animState: "idle",
 }))
 : dirtyStack;

 const [cleanRack, setCleanRack] = useState<Plate[]>([

```



```

 if (dirtyStack.length === 0) {
 setShake(true);
 addLog('△ POP: Нечего мыть! Все тарелки чистые.')
 setTimeout(() => setShake(false), 400);
 return;
 }

 setIsWashing(true);
 const topPlate = dirtyStack[dirtyStack.length - 1];

 addLog(`  POP: Начинаем мыть верхнюю тарелку с ${topPlate.emoji}`);

 // Step 1: Start washing animation (sponge and soap)
 setDirtyStack(prev => prev.map(p => p.id !== topPlate.id));
 setShowSoap(true);

 setTimeout(() => {
 // Step 2: Wash complete. Plate becomes clean and
 const cleanPlate: Plate = {
 ...topPlate,
 isDirty: false,
 animState: 'clean',
 };

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, 1));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(`  Готово! Чистая тарелка ${topPlate.emoji}`);
 }, 850);
 }, [dirtyStack, isWashing]);

const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', State: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', State: 'in' },
]);
};

```

```

 <button
 onClick={popPlate}
 disabled={isWashing || dirtyStack.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-slate-800 to-slate-900
 opacity-40 disabled:cursor-not-allowed text-white font-medium
 py-3.5 rounded-2xl transition-all cursor-pointer focus:outline-none"
 >
 Помыть верхнюю (POP)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium
 cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12 gap-4">

 { /* ЛЕВАЯ КОЛОНКА: СТОПКА ГРЯЗНЫХ ТАРЕЛОК */ }
 <div className="md:col-span-7 bg-slate-900 rounded-2xl p-4 min-h-[380px]">
 <div className="absolute top-4 left-4 text-[12px] font-mono text-slate-400">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 { /* Визуализация раковины */ }
 <div className="absolute top-4 right-4 flex items-center justify-center gap-2">
 <div className="r-slate-800/80 text-[10px] font-mono">

 РАКОВИНА
 </div>

 { /* Мыльные пузыри при мытье */ }
 </div>
 </div>
 </div>

```

```

 return (
 <div
 key={plate.id}
 className={`
 w-full max-w-[280px] h-10 rounded
 hadow-md select-none transition-all duration
 ${plate.color}
 ${plate.animState === 'in' ? 'anim
 ${plate.animState === 'washing' ?
 ${isTop ? 'ring-4 ring-blue-500/50
 `}
 >
 {plate.em
 <span className="text-slate-800 tex
 {isTop && (
 <span className="text-[9px] bg-bl
 LIFO

)}
 </div>
);
 }}}
 </div>
)}

 {/* Столешница раковины */}
 <div className="w-full h-4 bg-gradient-to-r f
</div>

 {/* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */}
 <div className="md:col-span-5 bg-slate-900 roun
 <div className="absolute top-4 left-4 text-[1
 Сушилка для чистой посуды
 </div>

 <div className="absolute top-4 right-4 flex i
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pu

```

```
 >
 {idx === 0 ?
 {entry}
 </div>
)}}
 </div>
</div>
);
};
```

---

ФАЙЛ 75/111: src/components/StringAlgorithmsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useRef, useMemo } from "react";
import { useVizRuntime, vizArray, vizNumber, vizString } from "viz";
import { Play, Pause, SkipForward, Undo, RefreshCw } from "lucide-react";
```

```
function generateKmpSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const pi = new Array(n).fill(0);
 const steps: any[] = [];
```

```
 steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });
```

```
 for (let i = 1; i < n; i++) {
 let j = pi[i - 1];
```

```
 steps.push({ i, j, pi: [...pi], highlight: [i, j] });
 ;
```

```
 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j] });
 // ид назад: откат j = pi[j - 1] = pi[j - 1];
 }
```

```

 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре-
 }. Копипаста сработала.` });
 }

 steps.push({ i, l, r, zTarget: z[i], zSource: i
 while (i + z[i] < n && s[z[i]] === s[i + z[i]])
 steps.push({ i, l, r, zTarget: z[i], zSource
 падают! Окно растёт.` });
 z[i]++;
 }

 if (i + z[i] < n) {
 steps.push({ i, l, r, zTarget: z[i], zSource
 [i]]}' разные. Стоп.` });
 } else {
 steps.push({ i, l, r, z: [...z], desc: `Упё-
 }

 steps.push({ i, l, r, z: [...z], desc: `Фиксиру-

 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 steps.push({ i, l, r, z: [...z], desc: ` 0
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

export function StringAlgorithmsViz({ defaultMode = "kmp"
const [mode, setMode] = useState<"kmp" | "z">(default
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

```

```

const nextStep = () => { setAutoPlay(false); setStepId
const prevStep = () => { setAutoPlay(false); setStepId
const reset = () => { setAutoPlay(false); setStepId

const baseStep = steps[stepIdx] || steps[0];
const vars = runtime?.variables;
const liveValues = vizArray(mode === "kmp" ? vars?.l
const hasLiveValues = !!vars && Object.prototype.has
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
const liveL = vizNumber(vars?.l);
const liveR = vizNumber(vars?.r);
const compilerLinked = liveI !== null || hasLiveValues
const step = compilerLinked
 ? {
 ...baseStep,
 i: liveI ?? baseStep.i,
 j: liveJ ?? baseStep.j,
 l: liveL ?? baseStep.l,
 r: liveR ?? baseStep.r,
 [mode === "kmp" ? "pi" : "z"]: hasLiveValues
 desc: `Компилятор: i=${liveI ?? "-"}${mode ===
 }
 : baseStep;

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </div>
 </div>
)

```

```

 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 bgColor = "bg-emerald-900";
 borderColor = "border-emerald-900";
 }
 if (step.zTarget === index) {
 borderColor = "border-aqua-500";
 bgColor = step.match === "L" ? "bg-emerald-900/40" : "bg-emerald-900/40";
 }
 }

 return (
 <div key={index} className={
 /* L/R markers for Z */
 {mode === "z" && step.l <= index && index <= step.r
 ? "border-2 border-aqua-500"
 : ""}
 >
 {mode === "z" && step.l <= index && index <= step.r
 ? <div className="absolute left-0 top-0 bottom-0 right-0 border-2 border-aqua-500" />
 : ""}

 {/* Index */}
 <span className={`text-${mode === "z" ? "emerald-400" : "gray-400"} font-mono text-lg transition-colors`}
 >{index}

 {/* Char Box */}
 <div className={`w-8 h-8 flex items-center justify-center text-center text-${mode === "z" ? "emerald-400" : "gray-400"} font-mono text-lg transition-colors`}
 >{char}</div>

 {/* Value array box */}
 <div className="text-[10px] font-mono font-bold"
 >{mode === "kmp" ? "KMP" : "Value Array"}</div>

 {/* KMP fingers */}
 <div className="text-[10px] font-mono font-bold"
 >{mode === "kmp" ? "KMP" : "Value Array"}</div>
 </div>
);
 }
}

```

```

 <div className="absolute top-0 right-0
 ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_0px_rgba(34,197,94,0.2)]
 })
 {step.found && mode === 'edit' ? (
 <div className="absolute top-0 right-0
 ulse z-0 bg-emerald-500/20 shadow-[0_0_15px_0px_rgba(34,197,94,0.2)]
 })
 </div>
)
 }
 }
</div>
</div>

<div className="flex flex-col sm:flex-row gap-2" style={{width: '100%'}}>
 {/* Controls */}
 <div className="flex bg-slate-900 border-1 border-slate-800 rounded-lg p-2" style={{width: '100%'}}>
 <button onClick={prevStep} disabled={!step.found} className="text-white rounded-lg ml-2 border-1 border-slate-800 hover:text-white hover:bg-slate-800" style={{width: '100%'}}>
 <Undo strokeWidth={3} size={16} />
 </button>
 <button onClick={playPause} className="text-white rounded-lg ml-2 border-1 border-slate-800 hover:text-white hover:bg-slate-800" style={{width: '100%'}}>
 {autoplay ? <Play size={16} /> : <Pause size={16} />}
 </button>
 <button onClick={nextStep} disabled={!step.found} className="text-white rounded-lg ml-2 border-1 border-slate-800 hover:text-white hover:bg-slate-800" style={{width: '100%'}}>
 <SkipForward strokeWidth={3} size={16} />
 </button>
 <button onClick={reset} aria-label="Reset" className="text-white rounded-lg ml-2 border-1 border-slate-800 hover:text-white hover:bg-slate-800" style={{width: '100%'}}>
 <RefreshCw strokeWidth={3} size={16} />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border-1 border-slate-800 rounded-lg p-2 overflow-hidden">
 <div className="absolute top-0 right-0 bottom-0 left-0">

```



```
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) {
 l = i;
 r = i + z[i] - 1;
 }
}
}
```

```
 </pre>
 </div>
</div>
);
}
```

---

ФАЙЛ 76/111: src/components/Tooltip.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from "react";

interface TooltipProps {
 /** Содержимое подсказки (можно JSX). */
 content: React.ReactNode;
 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху
 side?: "top" | "bottom";
 className?: string;
}

/**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
export const Tooltip: React.FC<TooltipProps> = ({ content, children }) => {
 const [open, setOpen] = useState(false);
```

```
import {
 Play,
 Pause,
 RotateCcw,
 Clock,
} from "lucide-react";

interface TNode {
 id: number;
 x: number;
 y: number;
 label: string;
 name: string;
}

interface TEdge {
 u: number;
 v: number;
}

const dagNodes: TNode[] = [
 { id: 0, x: 100, y: 150, label: "0", name: "Матан 1 (Математика)" },
 { id: 1, x: 280, y: 70, label: "1", name: "Дискретка (Дискретная математика)" },
 { id: 2, x: 280, y: 230, label: "2", name: "Линейный алгебра (Линейная алгебра)" },
 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы (Алгоритмы)" },
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное обучение (Машинное обучение)" },
];

const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];

interface TStep {
 desc: string;
```

```

 };

 recordStep(
 "Начало топологической сортировки. Используем кла
 null,
);

 // Adj list
 const adj: number[][] = Array.from({ length: 5 }, (
 dagEdges.forEach((e) => adj[e.u].push(e.v)));

 const dfs = (u: number) => {
 visited.add(u);
 dfsStack.push(u);
 recordStep(
 `DFS: зашли в вершину ${u} (${dagNodes[u].name
 u,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs(to);
 } else if (!fullyProcessed.has(to)) {
 // Explaining potential cycle detected (not in
 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
 }
 }

 fullyProcessed.add(u);
 dfsStack.pop();
 topoList.push(u); // prepend behavior: we write i
 recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,

```

```

 setIsPlaying(false);
 }
 return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getTopoListString = () => {
 return [...step.topoList]
 .reverse()
 .map((id) => dagNodes[id].name)
 .join(" → ");
};

return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Clock className="w-5 h-5 text-indigo-400" />
 Топологическая сортировка (Зависимости обучен
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>
 </div>
 </div>
);

```

```

>
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
</marker>
</defs>

```

```

{ /* Edges */ }
{ dagEdges.map((edge, idx) => {
 const u = dagNodes.find((n) => n.id === e
 const v = dagNodes.find((n) => n.id === e

 const isActive =
 (step.currentNode === edge.u || step.cu
 step.visited.has(edge.v);

 return (
 <line
 key={`edge-${idx}`}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isActive ? "#818cf8" : "#334155"}
 strokeWidth={isActive ? "3" : "2"}
 markerEnd={`url(#${isActive ? "dag-ar
 className="transition-all duration-300
 />
);
}})

```

```

{ /* Nodes */ }
{ dagNodes.map((node) => {
 const isCurrent = step.currentNode === no
 const isVisited = step.visited.has(node.i
 const isProcessed = step.fullyProcessed.h

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";

```

```

 ({node.label}) {node.name.split(" ")
 </text>
 <text
 x={node.x}
 y={node.y + 12}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="9px"
 className="pointer-events-none"
 >
 {node.name.split(" ").slice(1).join(" ")}
 </text>
</g>
);
 }}}
</svg>
</div>

{/* Console column */}
<div className="lg:col-span-4 bg-slate-950 p-5 rounded-00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-400">
 Статус сортировки
 </h4>

 <div className="bg-slate-900 border border-slate-800 p-4">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded">

```

```

 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
t-white"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s
t-white"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 78/111: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useContext, useEffect, useMemo, useState } from "react";
import { Terminal } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime,

```

```
/**
```

```
 * Интерактивный Треар – шесть режимов:
```

```
 * build – сборка: ВИДИМАЯ сортировка точек (и чем с
```

```
 * координатная плоскость НАВЕРХУ (ассоциаци
```

```
 * чек-лист инвариантов (автопроверка, как у
```

```
 * split – разрез по x с двумя корзинами (грамотная
```

```

 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 1, grade: -4 },
 { key: 7, grade: 2 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 12, grade: 0 },
];
 // (3,5) дубль ключа/приоритета – уберём: ввод без повт
 export const INPUT_CLEAN: Pair[] = INPUT_ORDER.filter((p) => !INPUT_ORDER.some(
 (q) => q.key === p.key && q.grade === p.grade

 const sortedByY = (ps: Pair[]): Pair[] => [...ps].sort((a, b) => a.grade - b.grade);

 /* ----- чистые алгоритмы (экспортируются для тестов) ----- */

 export function insert(root: TNode | null, p: Pair): TNode | null {
 if (!root) return { id: pairId(p.key, p.grade), key: p.key, grade: p.grade, left: null, right: null };
 if (p.key <= root.key) root.left = insert(root.left, p);
 else root.right = insert(root.right, p);
 return root;
 }

 export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of sortedByY(ps)) root = insert(root, p);
 return root;
 }

 export interface SplitResult {
 l: TNode | null;
 r: TNode | null;
 trace: { atId: NodeId; goLeft: boolean; note: string };
 }

 export function splitTree(root: TNode | null, x: number): SplitResult {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } => {
 if (!t) return { l: null, r: null };
 if (t.key < x) return go(t.right);
 if (t.key > x) return go(t.left);
 return { l: t.left, r: t.right };
 };
 const { l, r } = go(root);
 const atId = root ? root.id : 0;
 return { l, r, trace: [...trace, { atId, goLeft: l === null, note: "" }] };
 }

```



```

 b.left = go(a, b.left);
 return b;
 };
 return { root: go(a, b), trace };
}

export function eraseNode(root: TNode | null, key: number): TNode | null {
 const trace: { atId: NodeId | null; note: string }[] = [];
 const go = (t: TNode | null): TNode | null => {
 if (!t) return null;
 if (key < t.key) {
 trace.push({ atId: t.id, note: `${key} < ${t.key}` });
 t.left = go(t.left);
 return t;
 }
 if (key > t.key) {
 trace.push({ atId: t.id, note: `${key} > ${t.key}` });
 t.right = go(t.right);
 return t;
 }
 trace.push({ atId: t.id, note: `Нашли (${t.key}), ${t.key}` });
 const m = mergeTree(t.left, t.right);
 trace.push(...m.trace.map((s) => ({ atId: s.chosenId, note: s.note })));
 return m.root;
 };
 return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null;

```

/\* ----- валидация treap (для чек-листа и стресс-теста) ----- \*/

```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean; } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;

```

```
};
```

```
interface TreeViewProps {
 root: TNode | null;
 width?: number;
 height?: number;
 nodeColor?: (id: NodeId) => string;
 nodeStroke?: (id: NodeId) => string;
 edgeColor?: (from: NodeId, to: NodeId) => string;
 edgeDash?: (from: NodeId, to: NodeId) => boolean;
 pulseId?: NodeId | null;
 subtitle?: string;
}
```

```
const TreeView: React.FC<TreeViewProps> = ({ root, width, height, ...props }) => {
 const pos = useMemo(() => layoutTree(root), [root]);
 const nodes = useMemo(() => [...pos.entries()].map(([id, node]) => node), [pos]);
 const edges = useMemo(() => treeEdges(root), [root]);
 const autoW = Math.max(420, ...nodes.map((n) => n.px + n.rx));
 const autoH = Math.max(260, ...nodes.map((n) => n.py + n.ry));
 const w = width ?? autoW;
 const h = height ?? autoH;
 const byId = (id: NodeId) => nodes.find((n) => n.id === id);
 return (
 <div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full">
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : 'black'}
 strokeWidth={2.5}
 strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
)
 })}
 </svg>
 </div>
);
}
```

```

selected?: Pair | null;
cursor?: { x: number; y: number } | null;
badEdge?: { a: Pair; b: Pair } | null;
onPointDown?: (p: Pair) => void;
onPointUp?: (p: Pair) => void;
onMove?: (e: React.PointerEvent<SVGSVGElement>) => void;
}
const Plane: React.FC<PlaneProps> = ({ points, drawnEdges }) => {
 const byPair = (p: Pair) => ({ x: PX(p.key), y: PY(p.key) });
 return (
 <svg
 viewBox={`0 0 ${PL_W} ${PL_H}`}
 className={`w-full ${interactive ? "touch-none select-none" : ""}`}
 onPointerMove={onMove}
 style={interactive ? { cursor: "crosshair" } : undefined}
 >
 <defs>
 <pattern id="pl-grid" width="14" height="14" patternUnits="userSpaceOnUse">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1"/>
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16} fill="white" stroke="black" stroke-width="1"/>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1"/>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H - 16} stroke="black" stroke-width="1"/>
 {[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16} fill="black" font-family="monospace" font-size="12"/>
 })}
 {[{-4, 0, 4, 8].map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) + 16} fill="black" font-family="monospace" font-size="12"/>
 })}
 <text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8" font-family="monospace" font-size="12">0</text>
 <text x={PX(1) - 26} y="14" fill="#94a3b8" font-family="monospace" font-size="12">1</text>

 {drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.key && badEdge.b.key === e.b.key) ||

```

```

 ly="monospace">{pending.key}</text>
 </g>
 })
 </svg>
};
};

/* ===== ИГРА: соединение мышкой =====

export type GEdge = { a: Pair; b: Pair }; // a – родите.

export const CANON_GAME: GEdge[] = (() => {
 const es: GEdge[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ a: { key: n.key, grade: n.g
 if (n.right) { es.push({ a: { key: n.key, grade: n.g
 };
 walk(buildTree(POINTS));
 return es;
})();

const sameP = (p: Pair, q: Pair) => p.key === q.key &&
const gpid = (p: Pair) => `${p.key}:${p.grade}`;

/** Цепочка родителей от точки вверх. */
const parentOf = (edges: GEdge[], id: string): GEdge |
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pa
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur))?.a;
 }

```

```

const child = a.grade > b.grade ? b : a;
if (parentOf(edges, gpid(child)))
 return { ok: false, reason: `У точки (${child.key});
 линию». ` };
if (reaches(edges, parent, child))
 return { ok: false, reason: "Цикл: линия замкнёт саму себя" };
const next = [...edges, { a: parent, b: child }];
if (!forestIsBst(next, POINTS))
 return { ok: false, reason: "Линия вниз по у допустима" };
return { ok: true };
}

/* ===== РЕЖИМ BUILD ===== */

type BuildPhase = "sort" | "connect" | "game";
type SortAlgo = "none" | "quicksort" | "counting";
type ConnectOrder = "y-desc" | "left-right";

const parsePairs = (raw: string): { pairs: Pair[]; errors: string[] } => {
 const pairs: Pair[] = [];
 const errors: string[] = [];
 const seen = new Set<string>();
 const chunks = raw.split(/[,;\n]+/).map((c) => c.trim());
 for (const chunk of chunks) {
 const nums = chunk.split(/[\sxx*]+/).filter((v) => !Number.isNaN(+v));
 if (nums.length !== 2 || nums.some((v) => !Number.isFinite(+v))) {
 errors.push(`«${chunk}» – нужно ровно два числа: ${nums.join(' ')} `);
 continue;
 }
 const id = `${nums[0]}:${nums[1]}`;
 if (seen.has(id)) {
 errors.push(`точка (${nums[0]}; ${nums[1]}) дважды `);
 continue;
 }
 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
 }
 const xs = pairs.map((p) => p.key);

```

```

 grew = true;
 }
}
}
if (childComp.has(pid(parent))) return { ok: false, reason: "уже есть" };
return {
 ok: true,
 edge: { a: parent, b: child },
 reason:
 x.grade > y.grade ? undefined : "линия в treap в порядке";
};
}

export function checkGameState(
 points: Pair[],
 edges: GameEdge[],
): { heapOk: boolean; singleParent: boolean; connected: boolean } {
 const n = points.length;
 if (n < 2) return { heapOk: true, singleParent: true, connected: true };
 const heapOk = edges.every((e) => e.a.grade > e.b.grade);
 const parentCount = new Map<string, number>();
 for (const e of edges) parentCount.set(pid(e.b), (parentCount.get(pid(e.b)) || 0) + 1);
 const singleParent = [...parentCount.values()].every(v => v === 1);
 // связность: компоненты через union-find
 const comp = new Map<string, string>(points.map((p) => [pid(p), pid(p)]));
 const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 return x;
 };
 for (const e of edges) {
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
 }
 const comps = new Set(points.map((p) => find(pid(p))));
 return { heapOk, singleParent, connected: comps.size === 1 };
}

```

```

 if (right) walk(right);
 };
 walk(roots[0]);
 if (seen.size !== n) bstOk = false;
 for (let i = 1; i < inorder.length; i++)
 if (inorder[i] < inorder[i - 1]) bstOk = false;
 }
 return { heapOk, singleParent, connected, countOk, bstOk };
}

```

/\*\* Дерево из проведённых линий — для показа результата

```

export function edgesToTree(edges: GameEdge[]): TNode | null {
 const parentOf = new Map<string, GameEdge>();
 for (const e of edges) parentOf.set(pid(e.b), e);
 const roots = edges.filter((e) => !parentOf.has(pid(e.a)));
 if (!roots.length && edges.length) return null;
 const nodes = new Map<string, TNode>();
 const make = (p: Pair): TNode => ({
 id: pid(p),
 key: p.key,
 grade: p.grade,
 left: null,
 right: null,
 });
 const rootPair = roots.length ? roots[0].a : edges.length > 0 ? edges[0].a : null;
 if (!rootPair) return null;
 const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p));
 if (kids.length > 2) return node; // невалидно — по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
 };
 return build(rootPair);
}

```

```

 left); }
 if (node.right) { es.push({ a: { key: node.key, g
 de.right); }
 };
 walk(userTree);
 return es;
}, [userTree]);
const leftRightEdges = useMemo(() => [...canonEdges].

useEffect(() => {
 if (!playing || phase === "game") return;
 const t = setTimeout(() => {
 if (phase === "sort") {
 if (algo === "none") { setPlaying(false); return;
 if (sortIdx < 3) setSortIdx(sortIdx + 1);
 else setPlaying(false);
 } else {
 const total = order === "y-desc" ? canonEdges.l
 if (connIdx < total) setConnIdx(connIdx + 1);
 else setPlaying(false);
 }
 }, 900);
 return () => clearTimeout(t);
}, [playing, phase, sortIdx, connIdx, algo, order, ca

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---

```



```

 if (phase !== "game") return;
 if (!sel) {
 setSel(p);
 setGameMsg(`Выбрана (${p.key}; ${p.grade}) – теперь
 } else if (sel.key === p.key && sel.grade === p.grade) {
 setSel(null);
 setGameMsg(null);
 } else {
 attemptConnect(sel, p);
 setSel(null);
 }
 };

const pointUp = (p: Pair) => {
 if (phase !== "game" || !sel) return;
 if (sel.key === p.key && sel.grade === p.grade) return;
 attemptConnect(sel, p);
 setSel(null);
 setCursor(null);
};

const planeMove = (e: React.PointerEvent<SVGSVGElement>) => {
 if (!sel) return;
 const rect = (e.currentTarget as SVGSVGElement).getBoundingClientRect();
 setCursor({ x: ((e.clientX - rect.left) / rect.width) * 100 });
};

const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
};

const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующий раз");
};

return (
 <div className="space-y-5">

```

```

<>
 <span className="text-xs text-slate-400"
 {[["quicksort", "counting"] as const].m
 <button
 key={a}
 onClick={() => { setAlgo(a); setSort
Math.round(n * Math.log2(n)) : 0} сравнений
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg
600 text-white" : "bg-slate-700 text-slate-1
 >
 {a === "quicksort" ? "quicksort (ср
 </button>
)})}
</>
) : (
 <>
 <span className="text-xs text-slate-400"
 <button
 onClick={() => startConnect("y-desc")}
 className={`px-3 py-1.5 rounded-lg te
bg-emerald-600 text-white" : "bg-slate-700
 >
 по y ↓ (алгоритм)
 </button>
 <button
 onClick={() => startConnect("left-right")}
 className={`px-3 py-1.5 rounded-lg te
"bg-emerald-600 text-white" : "bg-slate-700
 >
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg te
go-600 text-white" : "bg-indigo-500/80 text
 >

```

```

 ево: линии вниз по у, обход по х отсортирован
) : gameMsg ? {
 <>Δ {gameMsg}</>
) : {
 <>Кликни точку-родителя (выше по «!»), за
)
) : connDone ? {
 <> Проведено {connEdges.length} линий в по
 : точки прибиты, дерево одно. А теперь
) : {
 <>Линий проведено: {connIdx} из {connEdgesA
 })
 </div>
</div>

```

```

/* Чек-лист инвариантов */
<div className={`rounded-xl p-4 border transition
 er-slate-700 bg-slate-900/40`} >
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {phase === "game" ? (
 <>
 {verdict.heapOk ? " " : " "} каждое р
 {verdict.singleParent ? " " : " "} у
 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
)
 }

 {phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">

```

```

const fullTree = useMemo(() => buildTree(POINTS), [])
const split = useMemo(() => splitTree(buildTree(POINTS), fullTree), [fullTree])
const steps = split.trace;
const idx = Math.min(stepIdx, steps.length - 1);

// mid из кода выбирает разрез, step листает кадры сплитов
useEffect(() => {
 if (live.mid === null || !SPLIT_KEYS.includes(live.mid)) {
 setX0(live.mid);
 setStepIdx(0);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.mid]);
useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [live.step, steps.length]);
const done = idx >= steps.length - 1;

const inL = new Set<NodeId>();
const inR = new Set<NodeId>();
steps.slice(0, idx + 1).forEach((s) => (s.goLeft ? inL.add(s.id) : inR.add(s.id)));

const lTree = split.l;
const rTree = split.r;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Number(e.target.value)); }}>
 <option value="0">0
 <option value="1">1
 <option value="2">2
 <option value="3">3
 <option value="6">6
 <option value="7">7
 <option value="9">9
 <option value="12">12
 <option value="45">45
 </select>
 <button onClick={() => setStepIdx(Math.max(0, idx - 1))} disabled={done}
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(stepIdx + 1, steps.length - 1))} disabled={done}
 font-bold bg-indigo-600 text-white disabled:opacity-30">
 </div>
 </div>
)

```

```

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const { l, r } = useMemo(() => splitTree(buildTree(P0), live), [live]);
 const merged = useMemo(() => mergeTree(cloneTree(l), cloneTree(r)), [l, r]);
 const steps = merged.trace;

 // step из кода листает кадры слияния
 useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.step, steps.length]);
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;
 const cur = steps[idx];

 return (
 <div className="space-y-4">
 <div className="flex items-center gap-3">
 <button onClick={() => setStepIdx(Math.max(0, idx - 1))}
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 2))}
 font-bold bg-indigo-600 text-white disabled:opacity-30">
 mar {idx}
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 <div className="bg-sky-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-sky-300">Left Tree
 <TreeView root={l} width={480} height={220} n={steps.length} />
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 border border-slate-700">
 <h5 className="text-sm font-bold text-rose-300">Right Tree
 <TreeView root={r} width={480} height={220} n={steps.length} />
 </div>
 </div>
 </div>
);
};

```

```

useEffect(() => {
 if (live.key === null || !POINTS.some((p) => p.key === live.key)) {
 setKey(live.key);
 setStepIdx(0);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [live.key]);
useEffect(() => {
 if (live.step === null) return;
 setStepIdx(Math.max(0, Math.min(steps.length - 1, live.step)));
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [live.step, steps.length]);
const idx = Math.min(stepIdx, steps.length - 1);
const done = idx >= steps.length - 1;

const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId) {
 return ids;
 }, [steps, idx]);
const pathSet = new Set(pathIds);

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Удалить
 <select value={key} onChange={(e) => { setKey(N
 te-700 rounded-lg px-3 py-1.5 text-sm font-monospace
 {POINTS.map((p) => <option key={p.key} value={p.key}>
 </select>
 <button onClick={() => setStepIdx(Math.max(0, live.stepIdx))}
 bg-slate-700 text-white disabled:opacity-30">Назад</button>
 <button onClick={() => setStepIdx(Math.min(stepIdx, steps.length - 1))}
 font-bold bg-indigo-600 text-white disabled:opacity-30">Вперед</button>
 мар {idx + 1} из {steps.length}
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4">

```

```

const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null)
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
}, [full]);
const nodeCount = slots.filter(Boolean).length;

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="text-white bg-slate-700 text-slate-300">heap</button>
 <button onClick={() => setPick("treap")} className="text-white bg-slate-700 text-slate-300">treap</button>
 </div>

 {pick === "heap" ? (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-slate-300">Heap</h4>
 <div className="flex gap-2 overflow-x-auto mb-4">
 {heapArr.slice(1).map((v, i) => (
 <div key={i} className="px-3 py-2 rounded">
 <div className="text-slate-500 text-[10px]>{v}</div>
 <div className="text-white font-bold">{i}</div>
 <div className="text-emerald-400 text-[10px]>{2i/2i+1}</div>
 </div>
))}
 </div>
 <p className="text-xs text-slate-400">Полное дерево, где элемент с индексом $2i/2i+1$ попадает в узел.</p>
 </div>
) : null}
 </div>
)

```

```

 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
 const target = 7;
 const sortedByKey = [...shuffled].sort((a, b) => a.key - b.key);
 const [stage, setStage] = useState<0 | 1 | 2>(0); // 0 - начало, 1 - поиск, 2 - результат
 const [bidx, setBidx] = useState(0);

 // шаги бинарного поиска на отсортированном массиве
 const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number; found: boolean; comparisons: number }[] = [];
 let lo = 0, hi = sortedByKey.length - 1;
 while (lo <= hi) {
 const mid = Math.floor((lo + hi) / 2);
 if (sortedByKey[mid].key === target) { steps.push({ lo, hi, mid, found: true, comparisons: steps.length }); break; }
 if (sortedByKey[mid].key < target) { steps.push({ lo, hi, mid, found: false, comparisons: steps.length }); lo = mid + 1; }
 else { steps.push({ lo, hi, mid, found: false, comparisons: steps.length }); hi = mid - 1; }
 }
 return steps;
 }, [sortedByKey]);

 const wrongMid = Math.floor((shuffled.length - 1) / 2);
 const comparisons = stage === 2 ? bidx + 1 : 0;

 return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0 ? 1 : 2); }}>

```



```

 {stage === 2 && bidx >= bsSteps.length - 1 && <
 сравнения. Но чтобы поиск вообще заработал,
 сравнения на всех. «Отсортирую бинарно
 </div>

 <div className="text-xs text-slate-400 bg-slate-900 p-2" >
 Есть ли разница, чем сортировать? Сравн
 ня граница, каждое сравнение даёт 1 бит. Но
 ля маленьких целых у (вкладка «Собрать», кноп
 задать порядок соединения.
 </div>
 </div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);
 const [sel, setSel] = useState<Pair | null>(null);
 const [hover, setHover] = useState<Pair | null>(null);
 const [msg, setMsg] = useState<string>("Кликни точку-1");

 const v = gameVerdict(edges, POINTS);

 const click = (p: Pair) => {
 if (!sel) {
 setSel(p);
 setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кли
 return;
 }
 if (sameP(sel, p)) {
 setSel(null);
 return;
 }
 const res = tryConnect(edges, sel, p);
 if (res.ok) {

```

```

 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399" : "#a5b4fc"}
 strokeWidth={i === edges.length - 1 ? 3.5 : 1}
 className="transition-all duration-300"
 />
)))

{POINTS.map((p) => {
 const isSel = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) && !isSel;
 return (
 <g key={`gp-${p.key}:${p.grade}`} onClick={handleClick}
 className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={10}
 stroke={isHover ? "#94a3b8" : "#38bdf8"}
 transition="all duration-200" />
 <text x={PX(p.key)} y={PY(p.grade) + 30}
 space">{p.key}</text>
 <text x={PX(p.key)} y={PY(p.grade) + 24}
 space">{p.grade}</text>
 </g>
);
})}
</svg>
</div>

```

```

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition duration-300
 at-900/40`} >
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {v.heap ? " " : " "}
 {v.oneParent ? " " : " "}
 {v.acyclic ? " " : " "}
 {v.bst ? " " : " "}

 </h4>

```

```

 const m = it.match(/-?\d+(?:\.\d+)?/g);
 if (m && m.length >= 2) parts.push(`${m[0]} ${m[1]}`);
 });
 return parts.length ? parts.join(", ") : null;
}, [vars]);

const mid = vars ? vizNumber(vars.mid) : null;
const key = vars ? vizNumber(vars.key) : null;
const step = vars ? vizNumber(vars.step) ?? vizNumber(1) : null;
const height = vars ? vizNumber(vars.height) ?? vizNumber(1) : null;
const n = vars ? vizNumber(vars.n) : null;
return { vars, pairsText, mid, key, step, height, n }
};

/* ===== КОРПУС ===== */

type Tab = "build" | "split" | "merge" | "erase" | "layers";

const TABS: { id: Tab; label: string }[] = [
 { id: "build", label: "Собрать" },
 { id: "split", label: "Split $\times$ " },
 { id: "merge", label: "Merge" },
 { id: "erase", label: "Erase" },
 { id: "layers", label: "Миф: $2k/2k+1$ " },
 { id: "search", label: "Поиск $\neq$ сортировка" },
 { id: "game", label: "Игра" },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");
 const chapterId = useContext(VizChapterContext);
 const live = useTreapLive();

 // вкладка = подрежим страницы: панель Python подстав.
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

```

```

 py-1">n = {live.n}
 {live.height !== null && <span className="text-
 px-2 py-1">height = {live.height}}
 {live.mid !== null && <span className="text-
 2 py-1">mid = {live.mid}}
 {live.key !== null && <span className="text-
 x-2 py-1">key = {live.key}}
 {live.step !== null && <span className="tex
 x-2 py-1">step = {live.step}}
 </>
) : (

 открой панель Python: переменные <code clas
 merald-300">mid</code>, <code className="for
 step</code> управляют этой демонстрацией

)}
 </div>

 {tab === "build" && <BuildMode />}
 {tab === "split" && <SplitMode />}
 {tab === "merge" && <MergeMode />}
 {tab === "erase" && <EraseMode />}
 {tab === "layers" && <LayersMode />}
 {tab === "search" && <SearchMode />}
 {tab === "game" && <GameMode />}
</div>
);
};

```

ФАЙЛ 79/111: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

```

```
 awaiting={!linked && !!runtime}
 line={runtime?.line}
 detail={detail}
 demoTitle={demoTitle}
 className={className}
 />
);
}
```

---

ФАЙЛ 80/111: src/components/vizRegistry.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React from "react";

import { ArticulationPointsViz } from "./ArticulationPointsViz";
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimulator";
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphComponentsViz } from "./GraphComponentsViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { ComplexityViz } from "./ComplexityViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import { PlanarColoringViz } from "./PlanarColoringViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
```

```

 title: "Куча (priority queue)",
 hint: "Добавляйте гипотезы — куча сама держит лучшую",
 Component: HeapViz,
},
"segment-trees": {
 title: "Дерево отрезков",
 hint: "Запросы и обновления на отрезке за $O(\log n)$.",
 Component: SegmentTreeVisualizer,
},
"sparse-table": {
 title: "Разрезанная таблица (1D и 2D)",
 hint: "Наведите курсор на любую ячейку — подсветится",
 Component: SparseTableViz,
},
treap: {
 title: "Treap: плоскость → дерево → Split/Merge/Erge",
 hint: "7 вкладок: Собрать (сортировка + соединение)
 обери дерево мышкой, судья-инварианты объясняет к",
 Component: TreapBuildViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно слож",
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: механика → большое дерево → собери с",
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.",
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:

```

```
 hint: "Иерархия $P \subseteq NP \subseteq PSPACE \subseteq EXP$, сведение $A \leq$
 Component: ComplexityViz,
 },
 "string-kmp": { title: "Префикс-функция (КМП)", Component: () => {
 "string-z-func": { title: "Z-функция", Component: () => {
 "aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => {
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
 },
 },
 },
 intro: {
 title: "Интерактивный симулятор базовых структур: с
 hint: "Проверь LIFO, FIFO и приоритетную очередь на
 Component: Simulator,
 },
 "everyday-basics": {
 title: "Бытовой тренажёр: стек, очередь, куча",
 hint: "Тарелки, очередь в столовой и мешок гипотез",
 Component: Simulator,
 },
};
```

```
export const getViz = (id?: string): VizEntry | undefined;
```

---

ФАЙЛ 81/111: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";
```

: "Сейчас показана встроенная демонстрация компонента. Когда  
визуализация переключится на ваши переменные.";

```
return (
 <div className={`flex flex-wrap items-center gap-1.5`} >
 <Tooltip side="bottom" content={tip}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center justify-center gap-1.5 rounded-full border
 ${linked ? "border-emerald-500/50 bg-emerald-500/10" : "border-slate-600/50 bg-slate-800/10"}
 ${!linked ? "border-sky-500/50 bg-sky-500/10 text-slate-800" : "border-slate-600/50 bg-slate-800/10 text-slate-400"}
 `} >
 {linked ? <Cpu className="h-3 w-3" /> : awaiting}
 {linked ? "данные из компилятора" : awaiting}
 {!linked ? "встроенное демо" : ""}
 {linked && detail ?
 {demoTitle}
 : ""}

 </Tooltip>

 {!linked && !awaiting && demoTitle ? (

 сценарий: {demoTitle}

) : null}

 {mismatch ? (
 <Tooltip side="bottom" content={mismatch}>
 <span
 tabIndex={0}
 className="cursor-help rounded-full border border-slate-600/50 bg-slate-800/10 text-slate-400 text-sm padding-2" >
 {mismatch}
 размерность не совпадает

 </Tooltip>
) : null}
 </div>
)
```



```

// Ветка H -> I -> S
5: { id: 5, label: 'HI', char: 'I', x: 350, y: 260, d: 100, r: 180 },
6: { id: 6, label: 'HIS', char: 'S', x: 320, y: 360, d: 100, r: 180 },
// Ветка S -> H -> E
7: { id: 7, label: 'S', char: 'S', x: 550, y: 160, d: 100, r: 180 },
8: { id: 8, label: 'SH', char: 'H', x: 580, y: 260, d: 100, r: 180 },
9: { id: 9, label: 'SHE', char: 'E', x: 610, y: 360, d: 100, r: 180 },
 = 2 (HE)
};

// Нормальные переходы бора
const TRIE_TRANSITIONS: { [key: number]: { [char: string]: number } } = {
 0: { H: 1, S: 7 },
 1: { E: 2, I: 5 },
 2: { R: 3 },
 3: { S: 4 },
 4: {},
 5: { S: 6 },
 6: {},
 7: { H: 8 },
 8: { E: 9 },
 9: {},
};

export const WaterfallAnimationWidget: React.FC = () => {
 // Переключатель режимов: 'training' (Полный BFS обход)
 const [activeMode, setActiveMode] = useState<'training'>('training');

 /* — связь с Python-компилятором (PAGE_SYNC["aho-corasick"])
 text — сканируемый текст, state — вершина автомата
 matches — найденные вхождения (индекс конца, слово)
 */
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
 }, [chapterId]);
}

```

```

 if (m) return { index: Number(m[1]), word: m[2] };
 }
 return null;
 })
 .filter((x): x is { index: number; word: string } => x)
 if (parsed.length) setFoundMatches(parsed);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [vars]));

const compilerLinked = Boolean(liveText || (liveState === 'SEARCH' || liveState === 'TRAINING'));

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>('');
const [currentIndex, setCurrentIndex] = useState<number>(0);
const [currentNode, setCurrentNode] = useState<number>(0);
const [isSearchDone, setIsSearchDone] = useState<boolean>(false);
const [searchLog, setSearchLog] = useState<string>('');
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 });
const [foundMatches, setFoundMatches] = useState<{ index: number; word: string }[]>([]);
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0);

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
 const clean = e.target.value.toUpperCase().replace(/ /g, '');
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
};

```

```

 logMsg += `В корне нет перехода по '${char}'. Л
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${orig
 рывок по fail-ссылке в #${curr} (${WATERFALL_I
 .`;
 setJumpPath({ from: originalCurr, to: nextNode,
 }
 curr = nextNode;
}

const currentMatches: { index: number; word: string
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 WATERFALL_NODES[temp].words.forEach((word) :
 currentMatches.push({ index: currentInd
 matchedWordsForLog.push(word);
 });
 }
 temp = WATERFALL_NODES[temp].term_link;
}

if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords
 setActiveSearchCodeLine(4);
}

setCurrentNode(curr);
setCurrentIndex(currentIndex + 1);
setSearchLog(logMsg);
if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
}
};

```

```

 desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель: ROOT. Рыба-разведчик плывет в ROOT и ищет ветку по "E".',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель: #2 (HE). Буквы "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 8,
 title: 'Шаг 5 (Depth 2): Вершина #8 (SH) — КАК СТОИМ?',
 desc: 'ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на Depth 2. Родитель: #5 (HI). Рыба плывет в ROOT и ищет ветку "H". У корня нет ветки с суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
},
{
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель: #8 (SH). Буквы "H" и "S" подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainStep];

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border-1px border-slate-700" >
 { /* Шапка и переключатель режимов */ }
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех функций
 </p>
 </div>
 </div>
 </div>

 { /* Переключатель режимов */ }
 <div className="bg-slate-900 p-1.5 rounded-xl border-1px border-slate-800" >
 <button
 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2 rounded-lg
 ${activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
 : 'text-slate-400 hover:text-white'}
 `}
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2 rounded-lg
 ${activeMode === 'search'

```

```

 </button>
)})
 </div>
 </div>

 <div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg
ion-all"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => M
 disabled={trainStep === TRAINING_STEPS_
 className="flex-1 bg-indigo-600 hover:b
ition-all shadow-lg shadow-indigo-900/50"
 >
 {trainStep === TRAINING_STEPS_INFO.leng
 </button>
 </div>
</div>

 /* Блок с кодом построения fail-ссылки и опи
 <div className="lg:col-span-2 bg-slate-900/90
 >
 <div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 ❌ Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-

```

```

 </h5>
 <div className="bg-slate-800 p-4 rounded-
w-inner min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
 </div>
</div>
) : (
/* ПАНЕЛЬ РЕЖИМА ПОИСКА */
<div className="grid grid-cols-1 lg:grid-cols-3"
 <div className="bg-slate-900/60 p-5 rounded-x
 <div>
 <h5 className="text-white font-bold mb-2"
 » Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4"
 Введи текст, чтобы лосось просканировал
 </p>
 <input
 type="text"
 value={textToScan}
 onChange={handleTextChange}
 maxLength={15}
 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border b
 cus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1"
 <span className="text-xs text-slate-400"
 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-cent
 idx === currentIndex

```

```

 <div className={`p-1.5 rounded flex item
border-blue-500 text-blue-300 font-bold' :
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && Инициализация}
 </div>
 <div className={`p-1.5 rounded flex item
border-rose-500 text-rose-300 font-bold an
 2. while (curr != root && !trie
 {activeSearchCodeLine === 2 && Выход по fail!}
 </div>
 <div className={`p-1.5 rounded flex item
border-blue-400 text-blue-200 font-bold' :
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && Продолжение по течению}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-emerald-500 text-emerald-300 font
 4. if (is_terminal[curr]) report
 {activeSearchCodeLine === 4 && Совпадение!}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-300 font-bold ml-4">
 <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded border border-slate-700 items-center">
 {searchLog}
 </div>
</div>
</div>
</div>

```



```

 { depth: 4, y: 460, label: 'Depth 4 (HERS
].map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14"
 />
 <text x="32" y={level.y + 4} fill="#94a5b8"
 {level.label}
 </text>
 </g>
)})

```

```

 /* Рендер прямых переходов (потоков воды)
 {Object.entries(TRIE_TRANSITIONS).map(([fromId, toId]) => {
 const fromNode = WATERFALL_NODES[Number(fromId)];
 return Object.entries(children).map(([childId, child]) => {
 const toNode = WATERFALL_NODES[toId];

```

```

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ||
 type === 'normal';

```

```

 // Подсветка в режиме обучения (когда ищем совпадения)
 let isTrainingExamined = activeMode === 'training' ||
 let isMatchBranch = isTrainingExamined && childId === toId;

```

```

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 /* Линия течения воды */

```

```

 /* Рендер fail-ссылок (пунктирные водопады)
 {Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.fail];

 // В режиме обучения показываем fail-ссылку
 // Найдем индекс шага, на котором обработали node
 const stepIdxForNode = TRAINING_STEPS_INFLECTION - node.id;
 if (activeMode === 'training' && trainStep === stepIdxForNode) {

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && !node.failLink;

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;
 const dy = targetNode.y - node.y;
 const cx = node.x + dx / 2 + (node.id % 2 === 0 ? -10 : 10);
 const cy = node.y + dy / 2 - 30;

 return (
 <g key={`fail-${node.id}`}>
 <path
 d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${targetNode.x} ${targetNode.y}`
 fill="none"
 stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
 strokeWidth={isHighlighted || isJustEstablished ? 2 : 1}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b98f">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
 }
 })}
 </div>
</div>

```

```

 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

 {/* Метка искомой вершины в режиме об
 {isTargetChild && {
 <g transform={`translate(${node.x},
 <rect x="-35" y="-12" width="70" h
 <text x="0" y="2" fontSize="10" f
 ИЩЕМ FAIL
 </text>
 </g>
 }}

 {/* Индикатор словарных слов */}
 {hasWords && {
 <g transform={`translate(${node.x +
 <circle cx="0" cy="0" r="9" fill=
 <text x="0" y="3" fontSize="10" f
 *
 </text>
 </g>
 }}
 </g>
);
 }}}

```

```

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}
<g>
 {/* Круг-подложка под рыбу */}
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5
 stroke="#ffffff"

```

```

 >
 <CheckCircle2 className="w-4 h-4 text
 {match.word}
 <span className="text-[10px] bg-emera
 Индекс {match.index}

 </div>
)))
</div>
})
</div>
})
}

{/* что сейчас сообщил компилятор */}
<div className="mt-6 flex flex-wrap items-center <
 <span className="text-[10px] uppercase tracking
 <Terminal className="w-3.5 h-3.5" /> из компи

{compilerLinked ? (
 <
 {liveText && <span className="text-[11px] f
 >text = {liveText}}
 {liveI !== null && <span className="text-[1
 py-1">i = {liveI}{liveText ? ` (${liveText.
 {liveState !== null && (
 <span className="text-[11px] font-mono bg
 state = {liveState}{WATERFALL_NODES[liv

)}
 {liveMatchesSig && <span className="text-[1
 d px-2 py-1">matches: {foundMatches.map((m)
 {liveFail !== undefined && <span className=
 ed-md px-2 py-1">fail = {JSON.stringify(liv
 </>
) : (

 открой панель Python и запусти Ахо–Корасик:
 "font-mono text-cyan-300">state</code>, <co

```

```
const A1 = [3, 1, 4, 1, 5, 9, 2, 6];
```

```
const A2 = [
 [1, 2, 3, 4],
 [5, 6, 7, 8],
 [9, 1, 2, 3],
 [4, 5, 6, 7],
];
```

```
const cellBase =
 "flex items-center justify-center rounded-md font-mono
 w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] te
```

```
export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 const chapterId = useContext(VizChapterContext);
```

```
 // Вкладка 2D = своё демо (свой скелет): компилятор п
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode === "2d"
 }, [chapterId, mode]);
```

```
 return (
 <div className="w-full bg-slate-950 rounded-2xl bor
 <div className="flex gap-2 mb-4 overflow-x-auto n
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 }`
)
)
)
)
```

```

 ? { from: hover.i, to: hover.i }
 : null;

const sum = pref[range.r + 1] - pref[range.l];

return (
 <div className="space-y-5">
 <div className="overflow-x-auto pb-1">
 <div className="inline-block min-w-full">
 {/* индексы */}
 <div className="flex gap-1.5 mb-1 pl-[52px]">
 {A1.map((_, i) => (
 <div key={i} className={`_${cellBase} !h-5
 {i}
 </div>
))}
 </div>

 {/* массив a */}
 <div className="flex gap-1.5 items-center mb-1">
 <div className="w-[46px] shrink-0 text-right">
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 const isDrivenAdd = shownDriven !== null
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r
 className={`_${cellBase} cursor-pointe
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2
 : inCover
 ? "bg-indigo-500 text-white rin
 : inRange
 ? "bg-indigo-900/70 text-indi

```

```

: isL
? "bg-rose-700 text-white
: "bg-slate-900 text-slate-300
} `}
title={blank || masked ? `P[${i}] - n
>
{blank || masked ? "." : String(shown)}
</div>
);
}}
</div>
</div>
</div>

```

```

{/* Пояснение под курсором */}
<div className="min-h-[62px] bg-slate-900 border border-slate-800" >
 {shownDriven !== null && !hover ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">Компилятор
 i =

 {hasLiveP && <> =
 Пустой массив

 </p>
) : hover?.kind === "pref" ? (
 hover.i === 0 ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 — это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hover.i)}

 </p>
)
)
)
}

```

```

/* ----- 2D -----
const Prefix2D: React.FC = () => {
 const n = A2.length;
 const m = A2[0].length;
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveI = vizNumber(vars?.i);
 const liveJ = vizNumber(vars?.j);

 const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array<number>().length);
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1];
 }
 }
 return p;
 }, [n, m]);

 const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
 const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });

 /**
 * Матрицы из кода пользователя рисуются В СВОЕЙ размерности.
 *
 * Раньше значения S[i][j] протискивались в демо-сетку,
 * размере матрицы часть ячеек становилась «пустыми»,
 * шаги компилятора не совпадали с картинкой.
 */
 const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
 const liveS = liveMatrix(livePrefSource?.value);
 const hasLiveS = liveS !== null;

 const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
 const liveA = liveMatrix(liveArrSource?.value);

 const prefRows = liveS?.rows ?? P.length;

```



```

const c1 = Math.max(0, Math.min(rect.c1, prefCols - 1);
const r2 = Math.max(r1, Math.min(rect.r2, prefRows - 1);
const c2 = Math.max(c1, Math.min(rect.c2, prefCols - 1);

const D = cellAt(r1, c1);
const B = cellAt(r1, c2 + 1);
const Cc = cellAt(r2 + 1, c1);
const Aa = cellAt(r2 + 1, c2 + 1);
const total = Aa !== null && B !== null && Cc !== null && D !== null;
const fmt = (v: number | null) => (v === null ? "-" : v.toString());

const cornerOf = (i: number, j: number) => {
 if (i === r2 + 1 && j === c2 + 1) return "A";
 if (i === r1 && j === c2 + 1) return "B";
 if (i === r2 + 1 && j === c1) return "C";
 if (i === r1 && j === c1) return "D";
 return null;
};

return (
 <div className="space-y-5">
 {sizeMismatch ? (
 <p className="rounded-lg border border-amber-500 p-5">
 {sizeMismatch}
 </p>
) : null}
 <div className="grid gap-5 lg:grid-cols-2">
 {/* Исходная матрица */}
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Исходная матрица {liveA ? `${liveArrSource?.length}` : ""}
 чтобы задать прямоугольник
 </div>
 <div className="inline-block">
 {srcGrid.map((row, i) => (
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= r1 && i <= r2 &&

```

```

 corner === "A"
 ? "bg-emerald-600 text-white"
 : corner === "D"
 ? "bg-sky-600 text-white"
 : corner
 ? "bg-rose-600 text-white"
 : "bg-slate-900 border border
return {
 <div
 key={j}
 onMouseEnter={() => setHover({ i,
 onMouseLeave={() => setHover(null
 className={`${cellBase} cursor-he
 isCompilerCell
 ? "bg-emerald-600 text-white
 : isHover
 ? "bg-amber-500 text-white
 : blank
 ? "bg-slate-950 text-slate
 : cornerColor
 }`}
 title={hasLiveS ? `${livePrefSour
 >
 {blank ? "." : String(shownValue)}
 {corner && !isHover && {
 <span className="absolute -top-
ht">
 {corner}

 }}
 </div>
);
 }}}
</div>
)))
</div>
</div>
</div>

```

```

import React, { useContext, useEffect, useRef, useState }
import {
 emitVizDemo,
 liveMatrix,
 useVizRuntime,
 vizArray,
 vizNumber,
 vizRecord,
 VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () => []);
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)]);
 }
}

// Data for 2D Matrix (Sparse Table 2D)
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],

```



```

/** Размерность таблицы пользователя: сетка демо 8x4,
const liveTable = liveMatrix(runtimeVars?.st);
const dimMismatch = liveTable
 ? liveTable.rows !== N || liveTable.cols !== LOG
 ? `Демо-сетка построена для массива из ${N} элемен
 блица ниже рисует ячейки демо-сетки: значения з
 панели переменных компилятора.`
 : null
: hasLiveSource && liveSource && liveSource.length
 ? `Демо-массив из ${N} элементов, а в вашем коде
 ся демо-размера ${N}x${LOG}.`
 : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-500">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 {dimMismatch ? {

```

```

 : String(v);
 return {
 <div key={idx} className="flex flex-col i
 <div
 className={`flex items-center justify
w,44px)}] h-[clamp(26px,8vw,44px)] text-[cla
 inCover ? "bg-indigo-500 text-white
 }`}
 >
 {display}
 </div>
 <span className="text-[9px] text-slate-
 </div>
);
}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {compilerLin
 j)}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}}$ = {1 << hover.j}:{ " "}

 min({arr1D.slice(covered.from, covered.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое чи
 </p>
)}
</div>

```

```

const sourceValue = j === 0 ? liveSou
const runtimeValue = liveValue ?? sou
const runtimeHasValue = runtimeValue
const manualVisible =
 isValid &&
 (j === 0 || computeSteps.findIndex(
const isVisible = compilerLinked ? run

let isActive = !!liveTarget && liveTa
let isSrc1 = false;
let isSrc2 = false;

if (compilerLinked && liveTarget && l
 isSrc1 = i === liveTarget.i && j ===
 isSrc2 = i === liveTarget.i + (1 <<
} else if (!compilerLinked && step > 1
 const currentStep = computeSteps[ste
 isActive = currentStep.i === i && c
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 =
 if (currentStep.i + (1 << (current
 }
}

// Готовые вычисленные уровни демо не
// уровень j=0 всегда берём из уже су
const shownValue = compilerLinked ? r
const displayValue =
 shownValue === undefined || shownVa
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return (
 <div key={j} className="relative fl
 {!isValid ? (
 <div className="w-[clamp(30px, 9

```

```

 animate={{ opacity: 1 }}
 className="absolute pointer-events-none"
 style={{
 width: 100,
 height: isSrc2 ? 40 + (1 <= isSrc2 ? 40 : 0) : 40,
 right: -50,
 top: 24,
 overflow: "visible",
 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc2 ? 40 : 0} 0 0`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#10b981"}
 strokeWidth="3"
 />
 </motion.svg>
)}
 </div>
);
}}}
</React.Fragment>
)}}
</div>
</div>
</div>

```

```

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4">
 {compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">

 i =
 – активна ячейка
 {(hasLiveTable || hasLiveSource) && <> =
 ?.[liveTarget.j] ?? (liveTarget.j === 0 ? liveTarget.j : 1)

 </p>
) : step > 0 && step <= maxStep ? (

```



```

const Viz2DBuild: React.FC = () => {
 const [k, setK] = useState(1);
 const [hover, setHover] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
 const [locked, setLocked] = useState<{ r: number; c: number }>({ r: 0, c: 0 });
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const liveK = vizNumber(vars?.k);
 /**
 * Шаг, закреплённый кликом по вкладке. Без него liveK
 * перетирал выбор пользователя и вкладки «Шаг 1/2/3»
 * клик есть, а матрица не меняется. Закрепление живё
 */
 const [pinnedK, setPinnedK] = useState<number | null>(null);
 const prevLiveK = useRef<number | null>(liveK);
 useEffect(() => {
 if (liveK !== prevLiveK.current) {
 prevLiveK.current = liveK;
 setPinnedK(null);
 }
 }, [liveK]);
 const liveR = vizNumber(vars?.r);
 const liveC = vizNumber(vars?.c);
 const liveTable = vizRecord(vars?.st2);
 const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
 const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
 const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
 const codeCell = liveR !== null && liveC !== null ? { r: liveR, c: liveC } : null;
 const compilerLinked = codeCell !== null || liveK !== null;
 const shownK = pinnedK !== null
 ? pinnedK
 : liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;

 /** 2D-демонстрация нарисована на фиксированной сетке
 const liveSource2D = liveMatrix(vars?.A);

```

```

 'text-slate-400 hover:text-white hover:bg-
 >
 {step === 0 ? "Оригинал (1x1)" : `Шаг ${s
 </button>
 }}}
 </div>
 {pinnedK !== null && compilerLinked && {
 <p className="-mt-6 mb-6 text-[11px] text-sla
 Шаг закреплён вручную: когда <span className
 демонстрация вернётся к шагу кода.
 </p>
 }}

 <div className="flex flex-col items-center gap-
 scrollbar">
 {[...Array(shownK + 1)].map(({_, i}) => shownK
 const stepL = 1 << step;
 const stepSize = 8 - stepL + 1;
 const isCurr = step === shownK;

 return (
 <React.Fragment key={step}>
 {idx > 0 && <div className="text-slate
 <div className={`flex flex-col items-c
 <h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
 rder-indigo-800/40 text-indigo-300' : 'bg-s
 </h4>
 <div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
 go-900/40 shadow-[0_0_25px_rgba(99,102,241,
 eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

```

```

highlightClass = 'bg-blue-500';
] md:scale-[1.08]';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emerald-500';
9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amber-500';
3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';
 }
}

return (
 <div
 key={idx}
 onMouseEnter={() => { if(isTop) setTop(true); }}
 onMouseLeave={() => { if(isTop) setTop(false); }}
 onClick={() => {
 if(isCurr) {
 if(locked && locked.value === idx) return;
 else setLocked({r: idx, c: curr});
 }
 }}
 className={`flex items-center justify-center gap-2 p-2
 ${isCurr ? 'w-[clamp(24px,6.5vw,38px)] h-[clamp(24px,6.5vw,38px)]' : 'w-[clamp(20px,5.2vw,30px)] h-[clamp(20px,5.2vw,30px)] min-w-[20px] min-h-[20px]'}
 ${hasLiveTable ? 'st2' : ''}
 `}
 >
 {isBlank ? "•" : String(shortName)}
 </div>
);
}}
</div>
</div>
</div>
</React.Fragment>
)

```

```

<div className="bg-[#0a0fle] rounded-xl p-5 shadow-inner overflow-x-auto">
 // Те
 int<
 ="text-amber-300">1);

 // Ква
 ST</
 <span className="text-rose
 e="text-amber-300">1], <sp
 <span className="text-blue
 ext-amber-300">1][k-<span className=
 pan>

 <span className="text-emer
 ="text-amber-300">1][k-<span classNa
 span>

 <span className="text-ambe
 white">c + pL][k-<span className="te
 ame="text-slate-500">// Правый Низ<b
 {'}'}));
</div>

```

```

{activeCell ? (
 <div className="bg-slate-950 p-5 rounded
 .2)} mt-2 transition-all">
 <p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
 ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
 }][{shownK}][{shownK}]

 </p>

```

```

<div className="space-y-2 pt-3 pl-2
 {{{}} => {
 const prevL = 1 << (shownK -

```

```

) : {
 <div className="bg-slate-950/40 border
 mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L}
 </div>
 </div>
)}
 </div>
)}
 </div>
</div>
);
};

```

```

const Viz2DQuery: React.FC = () => {
 const [r1, setR1] = useState(1);
 const [c1, setC1] = useState(1);
 const [r2, setR2] = useState(5);
 const [c2, setC2] = useState(6);
 const runtime = useVizRuntime();

 // Координаты запроса — те же захардкоженные имена,
 useEffect(() => {
 const vars = runtime?.variables;
 const nr1 = vizNumber(vars?.r1);
 const nc1 = vizNumber(vars?.c1);
 const nr2 = vizNumber(vars?.r2);
 const nc2 = vizNumber(vars?.c2);
 if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
 if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
 if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
 if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
 }, [runtime]);

```

```

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((na
const queryMismatch = liveCoords.some((v) => v !== n
 ? `Координаты запроса в вашем коде выходят за дем

```

```

 </div>
)
}}}

<div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>

 {/* Показываем 4 угла в самой матрице */}
 <div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px)`, height: `ca
 <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px)`, l
 <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)`, width: `calc(`${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(`${(Ly / 8) * 100
 </div>

 <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={

```

```

ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b

 // 2.
 int<

В запроса)

r1][<span className="text-white bg-r

во, чтобы правый край уперся в c2)<b

r1][<span className="text-white bg-b

рх, чтобы нижний край уперся в r2)<b

nded">r2 - Lx + 1][<span className="

и влево от c2)

">r2 - Lx + 1][<span className="text
 {'}'}));
</div>

<div className="w-full flex justify-cent
 <div className="flex flex-col items-ce
 <h4 className="text-slate-400 font-bo
 Откуда берутся эти 4 числа: матриц
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
 </h4>
 <div className="grid gap-[2px] p-2.5
Columns: `repeat(${matCols}, 1fr)`}}>

```

```


 <span className="text-blue-400 m
n>
 <span className="text-emerald-400
span>
 <span className="text-amber-400 r
 <span className="text-slate-400
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>
 </div>
)
 }
}
```

## РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

ФАЙЛ 85/111: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 137 | РАЗМЕР

```

 </div>
 </main>
 </div>

 <script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold l
 }
 </script>

```



```

 if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors' : 'px-3 py-1 rounded transition-colors');
 }

 // Initialize theme and mode
 setTheme('dark');

 // Setup intersection observer for TOC highlighting
 document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer-btn');
 const drawerOverlay = document.getElementById('drawer-overlay');

 function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('hidden');
 if (isClosed) {
 mobileDrawer.classList.remove('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.remove('hidden');
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove('hidden'), 10);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add('hidden');
 if (drawerOverlay) {
 drawerOverlay.classList.add('hidden');
 }
 document.body.style.overflow = "auto";
 }
 }

 mobileMenuBtn.addEventListener('click', toggleDrawer);
 closeDrawerBtn.addEventListener('click', toggleDrawer);
 drawerOverlay.addEventListener('click', toggleDrawer);
 });

```

```

const id = entry.target.id;
document.querySelectorAll('aside')
 .forEach(function(link) {
 if (link.getAttribute('href') === id) {
 link.classList.add('bg-#ccc');
 } else {
 link.classList.remove('bg-#ccc');
 }
 });
});
}, observerOptions);

document.querySelectorAll('section[id^="questions"]')
 .forEach(function(section) {
 observer.observe(section);
 });
});
</script>
</body>
</html>

```

ФАЙЛ 86/111: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР: 1024x768

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.
 <script>
 window.MathJax = {
 tex: {
```

```
mjx-container {
 max-width: 100% !important;
}
mjx-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjx-container[display="true"]::-webkit-scrollbar
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-
 .uno-card::before, .uno-card::after { font-
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 40px; font-size: 4px; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important; }
 .formula-scroll { width: 100%; height: 100%; }
}
.uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
}
```

```
 background-image: repeating-linear-gradient(
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Se
 }
body.theme-notebook .bg-slate-900, body.theme-n
 late-700 {
 background-color: rgba(255, 255, 255, 0.7)
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
 }
body.theme-notebook .text-slate-200, body.theme
 ant; }
body.theme-notebook .text-slate-400 { color: #5
body.theme-notebook .border-slate-600, body.the
 : #94a3b8 !important; }
body.theme-notebook .font-mono { font-family: "
body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1);
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
}
.creepy-emoji {
 display: inline-block;
 transform-origin: center;
```

```

</marker>
<marker id="arrow-pink" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b0" stroke="none"/>
</marker>
<marker id="arrow-cyan" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4c1" stroke="none"/>
</marker>
<marker id="arrow-indigo" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" stroke="none"/>
</marker>
<marker id="arrow-lime" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63d" stroke="none"/>
</marker>
<marker id="arrow-green" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" stroke="none"/>
</marker>

<!-- ЛЕГО БЛОКИ -->
<g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20" fill="yellow" stroke="black" stroke-width="1"/>
 <rect x="6" y="-5" width="8" height="6" fill="yellow" stroke="black" stroke-width="1"/>
 <rect x="26" y="-5" width="8" height="6" fill="yellow" stroke="black" stroke-width="1"/>
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<g id="lego-red">
 <rect x="0" y="0" width="40" height="20" fill="red" stroke="black" stroke-width="1"/>
 <rect x="6" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
 <rect x="26" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10" fill="red" stroke="black" stroke-width="1"/>
 <rect x="6" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
 <rect x="26" y="-5" width="8" height="6" fill="red" stroke="black" stroke-width="1"/>
 <line x1="2" y1="2" x2="38" y2="2" stroke="black" stroke-width="1"/>
</g>

<!-- МЯСОРУБКА -->
<g id="meat-grinder">

```

```
nslate-x-full transition-transform duration-300
w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto
<div class="h-full flex flex-col p-6 overflow-hidden
lg:rounded-2xl lg:border lg:border-slate-700">
 <div class="flex justify-between items-center">
 <h3 class="font-black text-transparent bg-clip-text
 ider text-sm">
```

Содержание

```
</h3>
<button id="close-drawer-btn" class="text-slate-400
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6"
 <path stroke-linecap="round" stroke-width="2" d="M6 18
 </svg>
</button>
</div>
<nav class="space-y-3 text-sm font-medium">
 <details class="group" open>
 <summary class="list-none cursor-pointer text-slate-400

 1. Алгебраические структуры

 </summary>
 <div class="pl-6 space-y-2 text-sm">
 1.1. Группы
 1.2. Кольца
 1.3. Поля
 1.4. Модули
 </div>
 </details>
 <details class="group">
 <summary class="list-none cursor-pointer text-slate-400

 2. Комплексные числа

```



```

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text-rose-500">

a>

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочленов

 </summary>
 <div class="pl-6 space-y-2 text-cyan-500">


```



```


 </summary>
 <div class="pl-6 mt-2 space-y-2">

deg)

ition-colors border border-rose-500/30"> K
 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer">

 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer">

 16. Неприводимые многоч

 </summary>
 <div class="pl-6 mt-2 space-y-2">


```

```

 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
});

// Scroll spy logic
document.addEventListener('DOMConten
 const sections = document.query
 ="q5-"], div[id^="q6-"], div[id^="q7-"], di
 v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;

 document.querySelectorAll('.gro
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });

const observer = new Intersection
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting
 visibleId = entry.t

 // Highlight active
 document.querySelec
 a.style.fontWeig
 if (a.href.ends
 a.style.fon
 }
 });

```

```
modal.style.flexDirection = 'column';
modal.style.alignItems = 'center';
modal.style.justifyContent = 'center';

const box = document.createElement('div');
box.style.width = '80%';
box.style.height = '80%';
box.style.backgroundColor = '#1a202c';
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('div');
header.innerText = 'Экспорт в PDF';
header.style.marginBottom = '10px';

const subHeader = document.createElement('div');
subHeader.innerText = 'Из-за особенностей браузеров, экспорт в PDF осуществляется в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```

 modal.appendChild(box);
 document.body.appendChild(modal)
 }

```

```

function downloadMD() {
 let clone = document.querySelector('#clone')

 // Smart markdown parsing based on https://github.com/tekyo/markdown-it
 // Smart markdown parsing based on https://github.com/tekyo/markdown-it
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.classList.contains('img-caption')) {
 caption = el.nextElementSibling.textContent;
 }
 el.textContent = '\n\n[Изображение: ' + caption + ']';
 });
 clone.querySelectorAll('.img-caption').forEach(el => {
 el.remove();
 });

 let text = clone.innerHTML;
 text = text.replace(/\n{3,}/g, '\n\n');

 if (window !== window.top) {
 fallbackModal(text, 'Markdown');
 return;
 }

 const blob = new Blob([text], { type: 'text/markdown' });
 const url = URL.createObjectURL(blob);
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.md';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
}

```

```

body.bg-white [class*="f
nt; }

body.bg-white [class*="f
body.bg-white .analogy-
body.bg-white .img-plac
body.bg-white .info-blo

body.bg-white .analogy-
or: #94a3b8 !important; }
body.bg-white .img-capt

/* Tweak card blocks */
body.bg-white .card-red

body.bg-white aside { b
#334155 !important; }
body.bg-white .text-whi

/* Ensure active links
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuc
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre

```

```

 </button>
 <a href="algebra.pdf" target="_blank"
-sm font-bold flex items-center justify-center gap-2>
 PDF

 <button onclick="downloadMD()"
-bold flex items-center justify-center gap-2>
 Markdown
 </button>
 <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center gap-2>
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-500">
 <div class="flex items-center gap-2">
 <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white">
 Темная тема </button>
 <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white">
 Светлая тема </button>
 </div>
 <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-white">
 Терминал </button>
 </div>
 </div>
 </div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12">
 <div class="flex min-h-0">

```

```

<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4"
 <div class="grid grid-cols-3 gap-4"
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tran
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tran
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 2: Базовые операции / Евкли
 <a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tran
 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-10" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t

```

```


 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
-l-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл
 <a href="#question-17" onclick="setM

```



```

 <div class="font-bold text-slate-500">

 </div>
 </div>

 <!-- ЛИКБЕЗ SECTION INSERTED HERE -->
 <section id="proof-algorithms" class="mb-6">
 <div class="flex items-center mb-6">
 <div class="bg-red-500/20 p-3 rounded" style="width: 100px; height: 100px; display: flex; align-items: center; justify-content: center; margin: 0 auto;">
 <svg class="w-8 h-8 text-red-500" style="width: 100%; height: 100%;"/>
 </div>
 <h2 class="text-3xl pr-4 border-l-2 border-l-red-500">
 Алгоритмы доказательства
 </h2>
 </div>

 <div class="prose prose-invert max-w-4xl">
 <p class="text-lg">
 Хватит тупить. Доказательство есть для того, чтобы жать на кнопки. Вот жесть.
 </p>

 <h2 class="text-2xl font-bold border-top-2px solid black">
 Меметика
 </h2>

 <!-- Прямое -->
 <div class="bg-slate-800/80 p-6">
 <h3 class="text-xl text-blue-400">Прямое доказательство
 <p class="text-sm text-slate-400">
 Рассмотрим следующие утверждения:
 <ul class="list-disc pl-5">
 1. Все натуральные числа являются натуральными.

 2. Все натуральные числа являются натуральными.

 3. Все натуральные числа являются натуральными.

 <p class="text-sm text-slate-400">
 Следовательно, все натуральные числа являются натуральными.
 </p>
 </div>
 </div>
 </div>
</div>
```

```
 <span class="text-w
усок k , замени его на жареного карася из

 </div>
```

```
 <!-- Двойная делимость -->
 <div class="bg-slate-800 p-6
 <h3 class="text-xl text-ora
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
вверх по уравнениям).
 <span class="text-w
рху вниз, показывая, что $\Delta \vdots r_i$
 <span class="text-w
х. Твой d больше.

 </div>
```

```
 <h2 class="text-2xl font-bold b
оритмы экзекуции)</h2>
```

```
 <div class="grid grid-cols-1 md
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мертв
 </div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
 c . Пока получается 0 – корень жив. Как то
корня минус один.</p>
 </div>
```

```

 <p class="text-sm text-gray-500">
 x двух, потом последних.

 <code class="text-x">
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-gray-500">
 есть сложение – есть ноль. Если есть умножение
 <code class="text-x">
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-gray-500">
 авь их драться. Умножение всегда врывается
 <code class="text-x">
 </div>

 </div>
 </section>
 </div>

 <div id="questions-container">
 <!-- ===== ВОПРОС 1 =====>

```

```

for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен – пропускаем */
 }
 }
}
return parts.join("\n");
}

```

```

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family:
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding:
.export-head { border-bottom: 1px solid #1e293b; padding:
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size:
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; font
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; font
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem;
.export-term .cx { color: #6ee7b7; font-size: .78rem; font
.export-foot { margin-top: 3rem; border-top: 1px solid
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65;

```

```

 * вёл бы в никуда, а битые ссылки – ровно то, от чего
 */
 if (opts.ticketAnchors) {
 try {
 annotateTicketLinks(host, { selfId: opts.selfId }
 host.querySelectorAll<HTMLAnchorElement>("a[data-
 a.setAttribute("href", `#ch-${a.dataset.goto}`)
 a.removeAttribute("data-goto");
 });
 } catch {
 /* ссылки – улучшение, а не обязательная часть файла
 }
 }

 const ids: string[] = [];
 host.querySelectorAll<HTMLElement>(".term-hint").forEach(
 const id = el.dataset.termId;
 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
));

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false)
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);

```

```
 Это автономная копия: стили внутри файла, интернет
 Интерактивные тренажёры и анимации в статичный HTML
 они живут в онлайн-версии: <a href="${escapeHtml(op
</div>
```

```
 <article class="prose prose-invert max-w-none">
 ${opts.body}
 </article>
```

```
 ${opts.gloss}
```

```
 <footer class="export-foot">Универсальное пособие по
</div>
</body>
</html>`;
}
```

```
/**
```

```
 * Картинки встраиваются в файл data-URL: автономная ко
 * иллюстрации офлайн (раньше src вел на ассет приложен
 * картинки исчезали – оставались только самые первые,
 */
```

```
async function inlineImages(html: string): Promise<string> {
 const doc = new DOMParser().parseFromString(`<body>${html}</body>`, 'text/html');
 const imgs = Array.from(doc.querySelectorAll("img[src]"));
 await Promise.all(
 imgs.map(async (img) => {
 const src = img.getAttribute("src") ?? "";
 if (!src || src.startsWith("data:")) return;
 try {
 const res = await fetch(src);
 if (!res.ok) return;
 const blob = await res.blob();
 if (blob.size > 4_000_000) return;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 });
 img.src = `data:image/png;base64,${data}`;
 } catch {
 // ignore
 }
 })
);
 return doc.body.innerHTML;
}
```

```

 `<figure class="export-fig">${m.name ? `Режим «${escapeHtml(
);
 }
 return figures.length
 ? `<div class="export-figs"><h3 class="export-fig
 : "";
} catch {
 return "";
}
}

```

```

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}

```

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^s*(\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}`;
}

```

```

/** Собрать автономный HTML одной главы. */
export async function buildChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body: rawBody, termIds } = prepareBody(chapter);
 const body = (await vizFigures(chapter.id)) + (await vizText(chapter.id));
 return wrap({
 title: chapter.title,

```

```
 <ol style="margin:0;padding-left:1.1rem;font-size:.85em">
 </nav>
 ${bodies.join("\n")}`;

```

```
 const html = wrap({
 title: "Универсальное пособие по алгоритмам",
 subtitle: `Все темы, ${chapters.length} шт.`,
 css,
 body,
 gloss: glossarySection(allTerms, true),
 origin: window.location.origin,
 });
 return html;
 }

```

```
/** Выгрузить всё пособие одним файлом. */
export async function downloadBookHtml(chapters: Chapter[],
 triggerDownload(await buildBookHtml(chapters), "posobie.html")
}

```

---

ФАЙЛ 89/111: src/utils/ticketLinks.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 165 | РАЗМЕР: 7.4 KB

---

```
import { useEffect } from "react";
import { chapterTopics } from "../data/content";

```

```
/**

```

```
 * Перекрёстные ссылки между билетами.

```

```
 *

```

```
 * В тексте страниц десятки упоминаний вида «...берите Форда
 * или «сердце Краскала (билет 18)». Раньше это был про
 * номер, но попасть на него не мог — ссылка была «битой»
 * в никуда. Здесь такие упоминания превращаются в настоящие
 *

```

```
 * * в приложении — `<a data-goto="bellman-ford">`, к

```



```

 if (found && found !== id) return undefined;
 found = id;
 }
 return found;
}

function hrefFor(id: string, lite: boolean): string {
 return lite ? `?lite=1&topic=${encodeURIComponent(id)}` : ''
}

/**
 * Строковая версия – для текстовой (`?lite=1`) версии,
 * через `dangerouslySetInnerHTML`, а эффекты разметки
 * HTML режется на «тег» и «текст», меняются только тек
 * вне `pre/code/a`.
 */
export function linkifyTicketsHtml(html: string, opts: {
 let inCode = 0;
 let inLink = 0;
 return html
 .split(/(<[^\>]*>)/g)
 .map((part) => {
 if (!part) return part;
 if (part.startsWith("<")) {
 const closing = part.startsWith("</");
 const tag = part.slice(closing ? 2 : 1).split(/
 if (tag === "pre" || tag === "code" || tag ===
 inCode = Math.max(0, inCode + (closing ? -1 :
 } else if (tag === "a") {
 inLink = Math.max(0, inLink + (closing ? -1 :
 }
 return part;
 }
 })
 if (inCode > 0 || inLink > 0) return part;
 return part.replace(TICKET_RE, (whole, a: string,
 const from = Number(a);
 const to = b ? Number(b) : from;
 const id = chapterForRange(from, to);

```

```

 TICKET_RE.lastIndex = 0;
 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;
 while ((m = TICKET_RE.exec(text)) !== null) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) continue;
 if (m.index > last) frag.appendChild(document.cre
 const a = document.createElement("a");
 a.className = "ticket-link";
 a.dataset.goto = id;
 a.setAttribute("href", hrefFor(id, opts.lite ?? f
 a.textContent = m[0];
 frag.appendChild(a);
 last = m.index + m[0].length;
 created += 1;
 }
 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.c
 textNode.parentNode?.replaceChild(frag, textNode);
}
return created;
}

/** Размечает ссылки при смене главы и подстраховывается
export function useTicketLinks(
 ref: React.RefObject<HTMLElement | null>,
 deps: unknown[],
 opts: { selfId?: string; lite?: boolean } = {}
) {
 useEffect(() => {
 const el = ref.current;
 if (el) annotateTicketLinks(el, opts);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, deps);
}

```

```

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\\s{2

const rows = chapters.map((c) => {
 const html = c.content ?? "";
 const analogies = (html.match(/Аналогия/gi) ?? []).length;
 return {
 id: c.id,
 title: c.title,
 num: Number.parseInt(c.title.match(/^((\\d+)\\.\\/)?.[1]
 analogies,
 hasAnalogy: analogies > 0,
 inlineSvg: /<svg[\\s>]/i.test(html),
 image: /<img[\\s>]/i.test(html),
 interactive: vizIds.has(c.id),
 chars: html.length,
 };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy &&
const gapViz = rows.filter((r) => r.hasAnalogy && !has2
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
 console.log("| --- | --- | --- | --- | --- | --- |");
 for (const r of rows) {

```

```

 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});
```

```
const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY_LABELS, GLOSSARY } = await import(path
```

```
const esc = (s) => s.replace(/[\.*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\
const re = new RegExp(
 `(?<![\\p{L}\\p{N}_-])(${GLOSSARY_LABELS.map((l) => e
 "giu"
);
```

```
const map = new Map(GLOSSARY_LABELS.map((l) => [l.label
```

```
// те же лимиты, что и в src/hooks/useTermHints.ts
```

```
const MAX_PER_SURFACE = 2;
```

```
const MAX_PER_TERM = 8;
```

```
const bad = [];
```

```
const used = new Set();
```

```
console.log("тем | подсв. | глава");
```

```
for (const c of chapters) {
```

```
 const text = c.content
```

```
 .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi
```

```
 .replace(/<[^\>]+>/g, " ");
```

```
const perTerm = new Map();
```

```
const perSurface = new Map();
```

```
let highlights = 0;
```

```
const ids = new Set();
```

```
for (const m of text.matchAll(re)) {
```

```
 const surface = m[1].toLowerCase();
```

```
 const id = map.get(surface);
```

```
 if (!id) continue;
```

```
 const ut = perTerm.get(id) ?? 0;
```

```
 const us = perSurface.get(surface) ?? 0;
```

```
 if (ut >= MAX PER TERM || us >= MAX PER SURFACE) co
```

-----  
\* сам, длинные таблицы/код/боксы режутся на фрагменты  
\*/

```
import fs from "node:fs";
import path from "node:path";
import { build } from "esbuild";
import PDFDocument from "pdfkit";
import { ROOT, OUT_DIR, ensureDir, humanSize } from "../utils";
import { htmlToBlocks } from "../html-to-blocks.mjs";
```

```
const GUIDE_PDF_PATH = path.join(OUT_DIR, "guide_lite.pdf");
const SHOTS_DIR = path.join(ROOT, "tmp", "viz-shots");
const ENTRY = path.join(ROOT, "tmp", "guide-entry.ts");
```

/\*\* Векторные схемы графов и таблицы состояний для PDF

```
const VECTOR_VISUALIZERS = {
 dijkstra: {
 title: "Векторный граф релаксации (Дейкстра)",
 nodes: [
 { id: "A", label: "A", x: 40, y: 60, dist: "0", s: 1000 },
 { id: "B", label: "B", x: 130, y: 30, dist: "3", s: 1000 },
 { id: "C", label: "C", x: 130, y: 90, dist: "2", s: 1000 },
 { id: "G", label: "G", x: 220, y: 90, dist: "5", s: 1000 },
 { id: "D", label: "D", x: 220, y: 30, dist: "6", s: 1000 },
 { id: "E", label: "E", x: 310, y: 90, dist: "7", s: 1000 },
 { id: "F", label: "F", x: 400, y: 60, dist: "8", s: 1000 },
],
 edges: [
 { u: "A", v: "B", w: 4, highlighted: true },
 { u: "A", v: "C", w: 2, highlighted: true },
 { u: "C", v: "B", w: 1, highlighted: true },
 { u: "B", v: "D", w: 5 },
 { u: "C", v: "G", w: 3, highlighted: true },
 { u: "C", v: "E", w: 8 },
 { u: "G", v: "D", w: 1, highlighted: true },
 { u: "G", v: "E", w: 2, highlighted: true },
 { u: "D", v: "F", w: 2 },
],
 },
};
```

```

 ["Пара 2", "B→C (-2)", "dist[C]=3", "нормальный ход"],
 ["Пара 3", "C→D, C→E", "dist[D]=6, dist[E]=7", "нормальный ход"],
 ["Пара 4", "E→F (1)", "dist[F]=8", "нормальный ход"],
 ["Итерация V-1", "все рёбра сошлись", "без изменений"],
 ["Итерация V (контроль)", "все рёбра сошлись", "без изменений"],
],
},
floyd: {
 title: "Матрица кратчайших путей (Фloyd–Уоршелл)",
 table: [
 ["из \ в", "S (0)", "A (1)", "B (2)", "C (3)", "D (4)"],
 ["S (0)", "0", "4", "5", "3", "6"],
 ["A (1)", "∞", "0", "∞", "1", "4"],
 ["B (2)", "∞", "∞", "0", "-2", "1"],
 ["C (3)", "∞", "∞", "∞", "0", "3"],
 ["D (4)", "∞", "∞", "∞", "∞", "0"],
],
},
mst: {
 title: "Минимальный осто́в (Краскал / Прим / Бору́вка)",
 nodes: [
 { id: "1", label: "1", x: 50, y: 30, status: "done" },
 { id: "2", label: "2", x: 150, y: 30, status: "done" },
 { id: "3", label: "3", x: 250, y: 30, status: "done" },
 { id: "4", label: "4", x: 50, y: 90, status: "done" },
 { id: "5", label: "5", x: 150, y: 90, status: "done" },
 { id: "6", label: "6", x: 250, y: 90, status: "done" },
 { id: "7", label: "7", x: 350, y: 60, status: "done" },
],
 edges: [
 { u: "1", v: "2", w: 2, highlighted: true },
 { u: "2", v: "3", w: 3, highlighted: true },
 { u: "1", v: "4", w: 1, highlighted: true },
 { u: "4", v: "5", w: 4, highlighted: true },
 { u: "2", v: "5", w: 5 },
 { u: "3", v: "6", w: 2, highlighted: true },
 { u: "5", v: "6", w: 6 },
 { u: "6", v: "7", w: 3, highlighted: true },
],
}

```

```

mono: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
 "/usr/local/share/fonts/DejaVuSansMono.ttf",
 "C:/Windows/Fonts/consola.ttf",
],
};

const pick = (list) => list.find((f) => fs.existsSync(f));

/**
 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает — их оставляем.
 */
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}
22}]/gu;

/** Русское склонение счётных существительных: 1 страница
const plural = (n, one, few, many) => {
 const mod10 = Math.abs(n) % 10;
 const mod100 = Math.abs(n) % 100;
 if (mod10 === 1 && mod100 !== 11) return one;
 if (mod10 >= 2 && mod10 <= 4 && (mod100 < 10 || mod100 > 10))
 return many;
};

const plain = (text) =>
 String(text ?? "")
 .replace(UNSUPPORTED, "")
 .replace(/\s{2,}/g, " ")
 .replace(/\u0000/g, "")
 .trim();

/* ----- данные приложения (через esbuild) ----- */

async function loadAppData() {
 ensureDir(path.dirname(ENTRY));
 fs.writeFileSync(

```

```

 }
 }
}
return [];
}

```

/\* \_\_\_\_\_ отрисовка PDF \_\_\_\_\_ \*/

```

class Renderer {
 constructor(doc, fonts) {
 this.doc = doc;
 this.fonts = fonts;
 this.footerTitle = "";
 this.pages = 0;
 this.cy = TOP;
 }

 /**
 * Колонтитул текущей страницы (печатаем до перехода к следующей)
 *
 * Важно: на время печати нижнее поле страницы уменьшаем, чтобы
 * решает, что текст не влез, и молча добавляет пустые строки.
 */
 stampFooter() {
 if (this.pages === 0 || !this.doc.page) return;
 const prevBottom = this.doc.page.margins.bottom;
 this.doc.page.margins.bottom = 16;
 const y = PAGE.height - PAGE.margin + 4;
 this.doc.font(this.fonts.regular).fontSize(7.6).fill(0);
 this.doc.text("Универсальное пособие • алгоритмы и структуры данных",
 width: CONTENT_W * 0.62,
 align: "left",
 lineBreak: false,
 ellipsis: true,
));
 this.doc.text(`${this.footerTitle} • ${this.pages}`,
 width: CONTENT_W * 0.38,
 align: "right",
));
 }
}

```



```

 */
 splitToFit(text, opts, max) {
 const measure = (t) => this.height(t, opts);
 const source = String(text);
 if (measure(source) <= max) return [source];
 const chunks = [];
 let current = "";
 const flush = () => {
 if (current.trim()) chunks.push(current.trim());
 current = "";
 };
 for (const sentence of source.split(/(?<=[.!?…;:])\s/)) {
 const withSentence = current ? `${current} ${sentence}` : sentence;
 if (measure(withSentence) <= max) {
 current = withSentence;
 continue;
 }
 flush();
 if (measure(sentence) <= max) {
 current = sentence;
 continue;
 }
 for (const word of sentence.split(/\s+/)) {
 const withWord = current ? `${current} ${word}` : word;
 if (measure(withWord) <= max) current = withWord;
 else {
 flush();
 current = word;
 }
 }
 }
 flush();
 return chunks.length ? chunks : [source];
 }

 rule(color = "#c7d2fe", width = 1) {
 const y = this.cy;
 this.doc.moveTo(PAGE.margin, y).lineTo(PAGE.margin, y);
 }
 }
}

```

```

const font = mono ? this.fonts.mono : italic && thi
const width = CONTENT_W - indent;
const opts = { font, size, width, lineGap };
const chunks = this.splitToFit(text, opts, BOTTOM -
chunks.forEach((chunk, index) => {
 const last = index === chunks.length - 1;
 const h = this.height(chunk, opts);
 this.ensure(h + (last ? gapAfter : 0), 6);
 const y0 = this.cy;
 this.doc.font(font).fontSize(size).fillColor(colo
 this.cy = Math.max(y0 + h, this.doc.y) + (last ?
 if (this.cy > BOTTOM) this.newPage();
}));
}

bullet(rawText, { ordinal = null, depth = 0, size = 9
const text = plain(rawText);
if (!text) return;
const indent = 16 + depth * 14;
const marker = ordinal ? `${ordinal}.` : "•";
const width = CONTENT_W - indent - 4;
const opts = { font: this.fonts.regular, size, width
const chunks = this.splitToFit(text, opts, BOTTOM -
chunks.forEach((chunk, index) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 2.5, 6);
 const y0 = this.cy;
 if (index === 0) {
 this.doc.font(this.fonts.regular).fontSize(size
 gn: "right", lineGap: 1 });
 }
 this.doc.font(this.fonts.regular).fontSize(size).
 this.cy = Math.max(y0 + h, this.doc.y) + 2.5;
 if (this.cy > BOTTOM) this.newPage();
}));
}

/** Код: режим на фрагменты по высоте страницы, кажды

```

```

this.doc
 .roundedRect(PAGE.margin, startY, boxW, height, 6)
 .fillAndStroke("#f8fafc", "#cbd5e1");

if (title) {
 this.doc
 .font(this.fonts.bold)
 .fontSize(9.5)
 .fillColor("#1e293b")
 .text(title, PAGE.margin + 12, startY + 8);
}

// Рёбра графа
for (const e of edges) {
 const fromNode = nodes.find((n) => n.id === e.u);
 const toNode = nodes.find((n) => n.id === e.v);
 if (!fromNode || !toNode) continue;

 const x1 = PAGE.margin + fromNode.x;
 const y1 = startY + fromNode.y;
 const x2 = PAGE.margin + toNode.x;
 const y2 = startY + toNode.y;

 const strokeColor = e.highlighted ? "#059669" : "#000000";
 const strokeWidth = e.highlighted ? 2 : 1;

 this.doc
 .moveTo(x1, y1)
 .lineTo(x2, y2)
 .lineWidth(strokeWidth)
 .strokeColor(strokeColor)
 .stroke();

 if (e.w !== undefined) {
 const mx = (x1 + x2) / 2;
 const my = (y1 + y2) / 2;
 this.doc

```

```

 this.cy = startY + height + 8;
 }

 /** Таблица: строки режутся по страницам, заголовок по
table(rawRows, { size = 8.5 } = {}) {
 const rows = rawRows.map((r) => r.map((c) => plain(c)))
 if (!rows.length) return;
 const cols = Math.max(...rows.map((r) => r.length))
 const widths = new Array(cols).fill(CONTENT_W / cols)
 const pad = 5;
 const header = rows[0];

 const rowHeight = (cells) => {
 let max = 0;
 for (let i = 0; i < cols; i++) {
 const h = this.height(cells[i] ?? "", { font: this.font })
 max = Math.max(max, h);
 }
 return max + pad * 2;
 };

 const drawRow = (cells, y, isHeader, zebra) => {
 const h = rowHeight(cells);
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill(
 for (let i = 0; i < cols; i++) {
 const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0)
 this.doc
 .font(isHeader ? this.fonts.bold : this.fonts.regular)
 .fontSize(size)
 .fillColor("#1f2937")
 .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] })
 }
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1)
 return h;
 };

 this.ensure(rowHeight(header) + 8, 14);

```

```

 let img;
 try {
 img = this.doc.openImage(file);
 } catch {
 return false;
 }
 const capH = caption ? 14 : 0;
 const scale = Math.min(maxWidth / img.width, (maxHe
 const w = img.width * scale;
 const h = img.height * scale;
 this.ensure(h + capH + 10, 20);
 const y0 = this.cy;
 const x0 = PAGE.margin + (CONTENT_W - w) / 2;
 this.doc.image(file, x0, y0, { width: w, height: h
 this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
 this.cy = y0 + h + 4;
 if (caption) {
 this.doc.font(this.fonts.regular).fontSize(8).fil
 enter" });
 this.cy = Math.max(this.cy, this.doc.y) + 6;
 }
 return true;
 }
}

```

```

/** Бокс с заголовком и строками; длинные режутся на
box(rawTitle, rawLines, { color = "#eef2ff", border =
 const title = plain(rawTitle);
 const lines = rawLines.map((l) => plain(l)).filter(
 const inner = CONTENT_W - 20;
 let rest = lines.slice();
 let first = true;
 while (rest.length || first) {
 first = false;
 const headH = this.height(title, { font: this.fon
 this.ensure(headH + 24, 16);
 const y0 = this.cy;
 // сколько строк влезет на эту страницу
 const avail = BOTTOM - y0 - headH - 16;

```

```

const doc = new PDFDocument({
 size: [PAGE.width, PAGE.height],
 margin: PAGE.margin,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – алгоритмы и струк
 Author: "algv0 · автоэкспорт",
 Subject: "Текст всех тем + визуализации, без кода
 CreationDate: new Date(),
 },
});
const stream = fs.createWriteStream(GUIDE_PDF_PATH);
doc.pipe(stream);

const R = new Renderer(doc, fonts);
const today = new Date().toISOString().slice(0, 10);
const totalTopics = new Set(Object.values(chapterTopics));
const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]).length;

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false });
R.h1("Алгоритмы и структуры данных", { size: 17, color: "#374151" });
R.para(
 `Билеты 1-${totalTopics} · ${chapters.length} ${plusSign} ${vizCount}, "интерактивная демонстрация", "интерактивный код"`,
 { size: 10.5, color: "#4b5563" }
);
R.para(
 "Автоматический компактный сборник: весь текст тем + визуализации. Навигация приложения и код панели Python",
 { size: 9.6, color: "#374151" }
);
R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

```

```

for (const b of htmlToBlocks(chapter.content)) {
 if (b.type === "p" && /^Билет\s/i.test(b.text)) c
 if (b.type === "heading" && b.level <= 2 && b.tex

 if (b.type === "heading") {
 if (b.level <= 2) R.h2(b.text);
 else if (b.level === 3) R.h3(b.text);
 else R.para(b.text, { size: 10.2, color: "#334111" });
 } else if (b.type === "summary") {
 R.para("► " + b.text, { size: 9.8, color: "#373737" });
 } else if (b.type === "li") {
 R.bullet(b.text, { ordinal: b.ordinal, depth: b
 } else if (b.type === "code") {
 R.code(b.text);
 } else if (b.type === "table") {
 R.table(b.rows);
 } else if (b.type === "quote") {
 R.quote(b.text);
 } else if (b.type === "caption") {
 R.para(b.text, { size: 8.3, color: "#6b7280", i
 } else {
 R.para(b.text);
 }
}

```

```

/* демонстрация: интерактивная схема (векторные гра
const viz = VIZ_REGISTRY[chapter.id];
const vecViz = VECTOR_VISUALIZERS[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) :

```

```

if (viz || syncKeys.length || vecViz) {
 R.h2("Демонстрация и интерактивная схема");
 if (viz) {
 R.para(viz.title, { size: 10.2, color: "#1e293b" });
 if (viz.hint) R.para(viz.hint, { size: 9.1, col
 }

```

```

 if (vecViz) {

```

```

 stream.on("error", reject);
 });

 const size = fs.statSync(GUIDE_PDF_PATH).size;
 console.log("[guide] PDF пособия собран");
 console.log(`        тем: ${chapters.length} · снимков: ${pages.length}`);
 console.log(`        выход: ${path.relative(ROOT, GUIDE_PDF_PATH)}`);

 // Самопроверка: страниц в файле должно быть ровно столько, сколько
 // рендерер. Расхождение означает, что pdftk создал файл с лишними
 // в файл без нижнего колонтитула и без нумерации.
 const pdfLib = await import("pdf-lib");
 const physical = (await pdfLib.PDFDocument.load(fs.readFileSync(GUIDE_PDF_PATH))).getPages().length;
 console.log(`        страниц: ${R.pages} (рендер) / ${physical} (файл)`);
 if (R.pages > 100) console.warn(`        ВНИМАНИЕ: ${R.pages} страниц`);
 if (physical !== R.pages) {
 console.error(`        ОШИБКА: pdftk добавил ${physical - R.pages} страниц`);
 process.exitCode = 1;
 }
}

main().catch((err) => {
 console.error("Ошибка сборки PDF пособия:", err.message);
 process.exit(1);
});

```

ФАЙЛ 93/111: scripts/export/build-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 64 | РАЗМЕР: 1.5 КБ

```

/**
 * ШАГ 3: PNG -> PDF
 *
 * Склеивает отрендеренные страницы tmp/export-pages/pages/ в
 * в единый документ public/export/full_code_all_pages.pdf
 */

```



```

 doc.addPage({ size: [A4.width, A4.height], margin: 10 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
 }

 doc.end();
 await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
 });

 console.log("[3/3] png -> pdf");
 console.log(`      страниц: ${pages.length}`);
 console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});

```

ФАЙЛ 94/111: scripts/export/capture-viz-shots.mjs  
 КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 122 | РАЗМЕР: 1.5 КБ

```

/**
 * Снимки визуализаций для PDF-конспекта: tmp/viz-shots.
 *
 * Headless-браузер открывает каждую страницу с демонстрацией
 * блок #chapter-viz. У страниц с вкладками (sparse-tabs)
 * снимается каждая вкладка — файл получает имя `chapter-id-tab.pdf`
 * и PDF подписывает его как «режим «вкладка»».
 *
 * npm run export:shots # сервер уже поднят
 * SHOTS_BASE_URL=http://127.0.0.1:4173 npm run export:shots
 *
 * В CI это делает rebuild-pdf.yml: сборка → vite preview →

```

```

 logLevel: "warning",
 });
 const mod = await import("data:text/javascript;base64");
 return mod.activeVizIds;
}

fs.mkdirSync(OUT_DIR, { recursive: true });

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: `);

const browser = await puppeteer.launch({
 headless: true,
 args: ["--no-sandbox", "--disable-setuid-sandbox", "--no-dev-shm-usage"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
for (const id of ids) {
 try {
 await page.goto(`${BASE_URL}/?topic=${id}`, { waitUntil: 'load' });
 await page.waitForSelector("#chapter-viz", { timeout: 10000 });
 // демо-анимации: даём кадру собраться до осмысленного состояния
 await new Promise((r) => setTimeout(r, 1600));
 const viz = await page.$("#chapter-viz");
 if (!viz) throw new Error("нет #chapter-viz");
 await viz.screenshot({ path: path.join(OUT_DIR, `${id}.png`), type: "png" });
 done += 1;

 for (const tab of TABS[id] ?? []) {
 try {
 const clicked = await page.evaluate((label) => {
 const root = document.querySelector("#chapter-viz");
 if (!root) return false;
 const btn = Array.from(root.querySelectorAll("button")).find(
 (b) => b.textContent ?? "" === label
);
 if (!btn) return false;
 btn.click();
 return true;
 }, label);
 if (!clicked) continue;
 await page.waitForSelector("#chapter-viz", { timeout: 10000 });
 await page.screenshot({ path: path.join(OUT_DIR, `${id}-${label}.png`), type: "png" });
 done += 1;
 } catch (e) {
 failed += 1;
 }
 }
 } catch (e) {
 failed += 1;
 }
}

console.log(`[export:shots] страниц со визуализациями: ${done} (ошибок: ${failed})`);

```

```
/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текст
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть
 * с фоллбэком на обход файловой системы, если git недо
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человекочитаемые категории для оглавления. */
const CATEGORIES = [
 [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
 [/^src\/data\/$/, "Контент и данные пособия"],
 [/^src\/components\/visualizers\/$/, "Визуализаторы (в
 [/^src\/components\/$/, "Интерактивные компоненты и си
 [/^src\/layout\/$/, "HTML-разметка (layout)"],
 [/^src\/utils\/$/, "Утилиты"],
 [/^src\/$/, "Ядро приложения"],
 [/^scripts\/$/, "Скрипты сборки и экспорта"],
 [/^\.github\/$/, "CI / автоматизация"],
 [/^[^\/]+$/, "Конфигурация проекта"],
```

```

 };
 walk(ROOT);
}

return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 });
}

/** Достаём заголовки глав из данных пособия (без запуска) */
function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("src/"))) {
 const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/g;
 let m;
 while ((m = re.exec(src)) !== null) {
 const [, id, title] = m;
 if (seen.has(id)) continue;
 seen.add(id);
 chapters.push({ id, title, file: rel });
 }
 }
 return chapters.sort((a, b) => {
 const na = parseInt(a.title.match(/(\d+)/)?.[0] ?? 0);
 const nb = parseInt(b.title.match(/(\d+)/)?.[0] ?? 0);
 return na - nb;
 });
}

```

```

 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")} . ${rel}`);
});
push();
push();

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel));
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} |`);
 push(SUBHR);
 push();
 push(content.replace(/\n+$/, ""));
 push();
 push();
});

push(HR);
push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
push(HR);

const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");

```

```
/** DPI раstra. Можно переопределить: EXPORT_DENSITY=
density: Number(process.env.EXPORT_DENSITY ?? 150),
/** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
monochrome: process.env.EXPORT_GRAYSCALE !== "1",
size: "A4",
font: "DejaVu-Sans-Mono",
fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSan
pointsize: 8,
/** Максимум символов в строке (далее – жёсткий пере
cols: 138,
/** Строк содержимого на странице (+2 служебные: шапка
rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
 for (const cmd of ["magick", "convert"]) {
 try {
 execFileSync(cmd, ["-version"], { stdio: "ignore" });
 return cmd;
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
```

```
});
```

```
const HEADING = new Set(["h1", "h2", "h3", "h4", "h5",
```

```
/**
```

```
 * Разбор HTML в дерево: { tag, attrs, children }, текст
```

```
 * Намеренно простой: страницы пособия генерируются нами
```

```
 */
```

```
export function parseHtml(html) {
```

```
 const root = { tag: "#root", attrs: {}, children: []
```

```
 const stack = [root];
```

```
 const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/g;
```

```
 let m;
```

```
 while ((m = re.exec(html)) !== null) {
```

```
 const [full, closeTag, openTag, attrsStr, selfClose] = m;
```

```
 if (full.startsWith("<!--")) continue;
```

```
 if (closeTag) {
```

```
 const tag = closeTag.toLowerCase();
```

```
 // закрываем до совпадения (прощает незакрытые <pre> и <code>)
```

```
 for (let i = stack.length - 1; i >= 1; i--) {
```

```
 if (stack[i].tag === tag) {
```

```
 stack.length = i;
```

```
 break;
```

```
 }
```

```
 }
```

```
 continue;
```

```
 }
```

```
 if (openTag) {
```

```
 const tag = openTag.toLowerCase();
```

```
 const attrs = {};
```

```
 const attrRe = /([a-zA-Z-_:]+)\s*=\s*"([^"]*)"|'([^\']*)'|([a-zA-Z-_:]+)\s*/g;
```

```
 let a;
```

```
 while ((a = attrRe.exec(attrsStr || "")) !== null) {
```

```
 const node = { tag, attrs, children: [] };
```

```
 stack[stack.length - 1].children.push(node);
```

```
 if (!VOID.has(tag) && !selfClose) stack.push(node);
```

-----  
const hasBlockChild = (node) => node.children.some(isBl

/\*\* Текст инлайнового поддерева в порядке документа. \*/

```
function inlineText(node) {
 let out = "";
 for (const c of node.children) {
 if (c.tag === "#text") {
 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {
 out += inlineText(c);
 }
 }
 return clean(out);
}
```

/\*\* Весь текст поддерева (для ячеек таблиц и подписей).

```
function collectText(node) {
 if (node.tag === "#text") return clean(node.text);
 if (SKIP.has(node.tag)) return "";
 if (node.tag === "table") return tableRows(node).map(
 return clean(node.children.map(collectText).join(" "))
 }
}
```

/\*\* Сырой текст внутри <pre> – переносы строк важны. \*/

```
function collectRawPre(node) {
 if (node.tag === "#text") return node.text;
 if (node.tag === "br") return "\n";
 if (SKIP.has(node.tag) && node.tag !== "img") return
 return node.children.map(collectRawPre).join("");
}
```

/\*\* Таблица → массив строк из массивов ячеек. \*/



```

 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, de
 }
 };

const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) return;

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 if (node.tag === "pre") {
 const text = decodeEntities(collectRawPre(node));
 if (text.trim()) push({ type: "code", text: text });
 return;
 }

 // есть блочные дети: идём по порядку, инлайн-пробе
 if (hasBlockChild(node)) {
 let buf = "";
 const flush = () => {
 const t = clean(buf);
 if (t) push({ type: "p", text: t });
 buf = "";
 };
 for (const c of node.children) {
 if (c.tag === "#text") {
 buf += c.text;
 continue;
 }
 if (SKIP.has(c.tag)) continue;
 if (!BLOCK.has(c.tag)) {
 buf += inlineText(c) + " ";
 }
 }
 }

```

```
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join("
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.
 else if (b.type === "caption") lines.push(" " + b
 else lines.push(b.text, "");
}
return lines.join("\n").replace(/\n{3,}/g, "\n\n").tr
}
```

---

ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002
 */
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";

const execFileAsync = promisify(execFile);

/** Раскрываем табы и жёстко переносим длинные строки,
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(
```

```

 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / $
 }}стр. ${idx + 1} / ${total}`;
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 10));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}

```

Универсальное пособие • полный исходный код

```

import { PAGE_SYNC, buildInitTemplate } from "../../src.
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../../
```

```
type Py = { runPythonAsync: (src: string) => Promise<un
```

```
export interface TestResult {
 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}
```

```
const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;
```

```
/* -----
/* УТИЛИТЫ
/* -----
```

```
const sleep = (ms: number) => new Promise((r) => setTim
```

```
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}
```

```
function bodyText(): string {
 return (document.body.textContent ?? "").replace(/\s+
}
```

```
function allButtons(): string[] {
 return Array.from(document.querySelectorAll("button")
 (b) => (b.getAttribute("aria-label") || b.getAttrib
```

```
function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, "
}
```

```
function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("t
}
```

```
function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window
 if (!setter) throw new Error("нет нативного setter'a"
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble
 });
}
```

```
async function key(k: string, init: KeyboardEventInit =
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("ke
 await sleep(20);
 });
}
```

```
/** Ждём появления признака в DOM (текст/предикат), пер
async function waitFor(pred: () => boolean, timeoutMs =
 const deadline = Date.now() + timeoutMs;
 while (Date.now() < deadline) {
 if (pred()) return true;
 await settle(step);
 }
 return pred();
}
```

```
const RUN_DONE = [
 "Трасса готова автоматически",
 "Нет шагов трассы",
 "Пустой код",
```

```

 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 const entry = VIZ_REGISTRY[vizId];
 if (!entry) throw new Error(`в реестре нет визуализации`);
 const Comp = entry.Component;
 root = createRoot(host);
 await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 <Comp />
 </VizChapterContext.Provider>
);
 });
 await settle(60);
 return host;
}

const BAD_DOM = [
 { re: /\bNaN\b/, why: "NaN в разметке" },
 { re: /\bundefined\b/, why: "undefined в разметке" },
 { re: /\[object Object\]/, why: "[object Object] в разметке" },
];

/**
 * Битые подстановки видны в КОРОТКИХ текстовых узлах (
 * вершины): «undefined», «NaN», «[object Object]». В д
 * псевдокода те же слова законны – их не трогаем.
 */
function domDefects(scope: HTMLElement | null): string[] {
 if (!scope) return [];
 const found: string[] = [];
 const walker = document.createTreeWalker(scope, 4 /*

```

```

}

/* -----
/* сценарии
/* -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").split(",");
const skipGroups = new Set((process.env.HAR_SKIP || "").split(","));
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(","));

export async function runSuite({ py }: { py: Py }): Promise<void> {
 const chapterList = onlyChapters.length ? chapters.filter(chapter => onlyChapters.includes(chapter.id)) : chapters;
 const want = (group: string) => !skipGroups.has(group);
 // перехват console.error приложения (React пишет сюда)
 console.error = (...args: unknown[]) => {
 consoleErrors.push(args.map(a => (a instanceof Error ? a.message : a)));
 };

 /* — 1. Панель Python: реальный запуск на каждой главе
 for (const chapter of chapterList) {
 const group = "A. страницы";
 await test(group, `${chapter.id}: монтируется, бейдж: ${chapter.id}`);
 await mountApp(chapter.id);
 const txt = bodyText();
 if (!txt.includes(chapter.title.slice(0, 18))) throw new Error(`не содержит заголовок`);
 if (!VIZ_REGISTRY[chapter.id]) return "визуализация не настроена";
 if (!/встроенное демо|данные из компилятора/.test(txt)) return "не содержит демо";
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в DOM: ${defects.join(", ")}`);
 const nameless = namelessButtons(host!);
 if (nameless.length) throw new Error(`кнопок без id: ${nameless.join(", ")}`);
 return `бейдж «встроенное демо» на месте, кнопок: ${nameless.length}`;
 });

 await test(group, `${chapter.id}: панель открывается`);
 await mountApp(chapter.id);

```

```

 { name: "словарь словарей d[0][1]", vars: { n: 4, k: 2,
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9,
 { name: "ключи-пары \"{i, j}\"", vars: { n: 3, k: 0,
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)":
 { name: "плоский список — это НЕ матрица", vars: {
 { name: "80×80 (обрезка до 64)", vars: { n: 80, k:
 (_, c) => (r === c ? 0 : 1 + ((r + c) % 50))) }
 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0,
 { name: "матрица под именем D и k/i/j вне диапазона
 [10, 11, 12, 0]] }, expect: "4×4" },
];

```

```

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 await act(async () => {
 emitVizState("floyd", { line: 3, func: "<module:
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!txt.includes(c.expect)) throw new Error(`в р
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return `бейдж: ${txt.match(/данные из компилятор
 });
}

```

```

/* — 3. Префиксные суммы 2D и разреженная таблица —
const gridCases: { viz: string; chapter: string; name
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 9, 10, 11, 12]], S: [[1, 3, 6, 10], [6, 14, 24, 3
 тора/ },
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 pref: [[1, 3], [4, 10]], r1: 0, c1: 0, r2: 1, c2
 { viz: "prefix-sums-2d", chapter: "prefix-sums-2d#2
 9]], S: [[1, 3, 6], [5, 12, 21], [12, 27, 45]],
 { viz: "sparse-table", chapter: "sparse-table", nam

```



```

 await click(findButton("Шаг 2 (4x4)"), "вкладка шаг
 const txt = (host?.textContent ?? "").replace(/\s+/
 if (!/Матрица блоков 4x4/.test(txt)) throw new Error
 g0 = grids()[0];
 const cls = (g0.children[0] as HTMLElement)?.classN
 // пустая активная клетка (код смотрит на (0,0), st
 if (!/border-dashed/.test(cls) || !/indigo/.test(cl
 throw new Error(`пустая активная клетка нарисован
 }
 if (domDefects(host).length) throw new Error(`артеф
 return "оригинал из А с первого шага, вкладка «Шаг
 });

```

```

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [
 ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6,
 ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w:
] as [string, Record<string, unknown>][[]]) {
 await test("D. графы путей", `${viz}: вершины 0...4 в
 await mountViz(viz);
 const before = allButtons().length;
 const playEnabled = (findButton(/^Пуск|Пуск$/)) as
 await act(async () => {
 emitVizState(viz, { line: 5, func: "<module>",
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!/ваши расстояния|dist/i.test(txt)) throw new
 const step = findButton(/^Шаг$|Шаг /) as HTMLButt
 if (step && !step.disabled) throw new Error("кноп
 if (domDefects(host).length) throw new Error(`арт
 await act(async () => {
 clearVizState(viz);
 await sleep(30);
 });
 const step2 = findButton(/^Шаг$|Шаг /) as HTMLBut
 if (step2 && step2.disabled) throw new Error("кноп

```

```

const txt = (host?.textContent ?? "").replace(/\\s
if (!/данные из компилятора/.test(txt)) throw new
const defects = domDefects(host);
if (defects.length) throw new Error(`артефакты в
return `шагов ${result.steps.length}${result.trun
).join(", ")}`;
});
}

```

/\* — 6. Поведение редактора: пустой код, ошибки, гла.

```

const ch = "dijkstra";

```

```

await test("F. редактор", "только комментарии: редакт
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# просто комментарий\n# и ещё один\n"
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`не дождались сообщения
 if (!textarea()) throw new Error("редактор исчез по
 if (!/встроенное демо/.test(bodyText())) throw new
 return "редактор на месте, бейдж «встроенное демо»"
});

```

```

await test("F. редактор", "синтаксическая ошибка: ред
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "def broken(:\n pass\n");
 const done = await waitFor(() => /Код не выполнен
 if (!done) throw new Error(`ошибка синтаксиса не по
 if (!textarea()) throw new Error("редактор исчез на
 if (!/invalid syntax|SyntaxError/i.test(bodyText()))
 return "ошибка показана, редактор остался";
});

```

```

const init = ta.value;
const editorButtons = allButtons();
const ref = PAGE_SYNC[ch]?.code ?? "";
if (!ref) throw new Error("у страницы нет эталонного кода");
const done0 = await waitFor(() => RUN_DONE.some((m) => m === "done"));
if (!done0) throw new Error(`инициализация не выполнена`);

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор не должен быть пустым
await click(findButton("Вставить эталонный код"), "Вставить эталонный код");
const trace = await waitFor(() => !!findButton(/Остановить выполнение/));
if (!trace) throw new Error(`эталон не запустился. проверьте код`);
if (!textarea()) throw new Error("редактор исчез во время выполнения");
if (/Ошибка:/.test(panelText())) throw new Error(`эталон не выполнен`);
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.startsWith("#"));
if (!ta2?.value.includes(firstRefLine)) {
 throw new Error(`выполняется не тот текст, что лежит в эталоне`);
}
const mustBe = [
 { what: "пуск/автопроход", re: /Запустить код|Продолжить выполнение/ },
 { what: "копирование кода", re: /Скопировать код|Копировать текст/ },
 { what: "заккрыть панель", re: /Заккрыть компилятор/ },
 { what: "шаг вперёд", re: /Шаг вперёд|вперёд/i },
 { what: "шаг назад", re: /Шаг назад|назад/i },
 { what: "свернуть листинг", re: /Свернуть листинг/ },
];
const btns = allButtons().join(" | ");
for (const m of mustBe) {
 if (!m.re.test(btns)) throw new Error(`в панели не найдено: ${m.what}`);
}
// На первом шаге переменных ещё нет (отладчик встал на паузу)
// поэтому связь с демонстрацией проверяем, дойдя до конца трассы
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error(`эталон выполнен до конца трассы`);

// — глаз ВЫКЛ: кода нет вовсе, ничего не выполняется

```

```
return "у каждой страницы свой код, возврат сохраняя";
});
```

```
await test("F. редактор", "редактор живёт и во время п
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`трасса не собралась. Па
 if (!textarea()) throw new Error("во время прогона
 if (!findButton(/Остановить прогон трассы/)) throw
 // Листинг сворачивается (освобождая место редактор
 // и редактор остаются на месте.
 const codeLines = () => panel()?.querySelectorAll("
 if (codeLines() === 0) throw new Error("в раскрытой
 await click(findButton("Свернуть листинг трассы"),
 await settle(80);
 if (!textarea()) throw new Error("после сворачивани
 if (codeLines() !== 0) throw new Error(`листинг не
 if (!findButton("В конец трассы")) throw new Error(
 await click(findButton("Показать листинг трассы"),
 await settle(80);
 if (codeLines() === 0) throw new Error("листинг не
 // Esc: прогон остановлен, трасса убрана, редактор
 await key("Escape");
 await settle(150);
 if (!textarea()) throw new Error("Esc убрал редактор
 if (findButton(/Остановить прогон трассы/)) throw n
 if (!/встроенное демо/.test(bodyText())) throw new
 return "редактор всегда на месте: трасса рядом, сво
});
```

```
await test("F. редактор", "заккрытие панели во время п
 await mountApp("floyd"); // 869 шагов: прогон долги
 await click(findButton("Боковая панель Python"), "п
 await waitFor(() => panel() !== null, 4000);
 await click(findButton("Заккрыть компилятор"), "крес
 await settle(150);
 if (panel()) throw new Error("панель не закрылась")
```

```

if (!listing) throw new Error("панель не показала трассы");
// 2) код выполнен, отладчик на первой строке: переменных пока нет
const awaiting = await waitFor(() => /переменных пока нет/.test(bodyText()));
if (!awaiting) throw new Error("`после запуска бейджик не появился");
const demo = await waitFor(() => $(bodyText()).match(/встроенное демо|данные из компилятора/));
// 3) дошли до конца трассы – картинка обязана жить
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("в конце трассы визуализация не появилась");
// 4) сняли связь (Esc + пустой код) – снова демо
await key("Escape");
await settle(80);
const ta = textarea();
if (!ta) throw new Error("редактор не вернулся по Escape");
typeInto(ta, "");
const cleared = await waitFor(() => /встроенное демо/);
if (!cleared) throw new Error("после очистки кода бейджик не появился");
if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не появились");
return "демо → выполнено/переменных нет → данные из компилятора";
});

```

/\* — 7. Текстовый режим для PDF/парсеров

---

/\* — 9. Ссылки между билетами, разделы содержания, бейджики

```

await test("Н. доступность", "содержание сжимается в PDF");
await mountApp("bridges-code");
const root = () => document.querySelector<HTMLElement>("#content");
const toggle = () => findButton(/Сжать содержание|Скрыть/);
if (!root()) throw new Error("нет сайдбара");
if (!/lg:w-56/.test(root()!.className)) throw new Error("неверный размер");
await click(toggle(), "сжать");
if (!/lg:w-44/.test(root()!.className)) throw new Error("неверный размер");
const items = Array.from(root()!.querySelectorAll("h2"));
if (items.length < chapters.length) throw new Error("не все главы");
if (root()!.querySelector("input")) throw new Error("неверный размер");
await click(toggle(), "скрыть");
if (!/lg:w-0/.test(root()!.className)) throw new Error("неверный размер");
await click(toggle(), "развернуть");

```

```

 if (!href.includes("topic=prefix-sums-2d")) throw new Error("неверная ссылка");
 await click(toPrefix, "ссылка на билет 5");
 await settle(120);
 const txt = bodyText();
 if (!/Двумерная матрица префиксных сумм/.test(txt))
 throw new Error(`клик по «билет 5» не открыл страницу`);
 return `ссылок на странице: ${links.length}, переходим к следующему билету`;
 });
}

```

```

await test("I. ссылки", "содержание: разделы по порядку", async () => {
 await mountApp("bridges-code");
 const sidebar = document.querySelector("#sidebar-content");
 if (!sidebar) throw new Error("нет сайдбара с содержанием");
 const headers = Array.from(sidebar.querySelectorAll("h2"))
 .map((el) => (el.textContent ?? "").trim())
 .filter((t) => SECTIONS.some((sec) => sec.title === t));
 if (headers.length !== SECTIONS.length) {
 throw new Error(`в содержании ${headers.length} разделов`);
 }
 const expected = SECTIONS.map((sec) => sec.title).join(" | ");
 if (headers.join(" | ") !== expected) throw new Error("неверный порядок разделов");
 // 11 «Мосты» и 12 «Точки сочленения» читаются парой
 const items = Array.from(sidebar.querySelectorAll("div"));
 const iBridges = items.findIndex((t) => t.includes("Мосты"));
 const iArtic = items.findIndex((t) => t.includes("Точки сочленения"));
 if (iBridges < 0 || iArtic < 0) throw new Error("неверный порядок разделов");
 if (Math.abs(iArtic - iBridges) !== 1) throw new Error("неверный порядок разделов");
 // 2 «разрезанная» и 5 «префиксные» – в одном разделе
 if (sectionOf("sparse-table").id !== sectionOf("prefix-sums-2d"))
 throw new Error("билеты 2 и 5 попали в разные разделы");
 if (sectionOf("bridges-code").id !== sectionOf("graph-edges"))
 throw new Error("билеты 11 и 12 попали в разные разделы");
 // порядок чтения: номера билетов сверху вниз не убывающий
 const nums = Array.from(sidebar.querySelectorAll("b"))
 .map((b) => (b.querySelector("span")?.textContent ?? ""));
}

```

```

 });
 if (!section) throw new Error("блиц не отрисован на
 const ask = (label: string | RegExp) =>
 Array.from(section.querySelectorAll("button")).find(
 (b) => b.getAttribute("aria-label") === label
);
 const hay = [b.getAttribute("aria-label"), b.textContent];
 return typeof label === "string" ? hay.includes(label) :
 });
 const shown = () => (section.textContent ?? "").replace(
 /<div>.*</div>/g, ""
);
 if (!shown().includes(list[0].question.slice(0, 24)))
 throw new Error(`первый вопрос не показан: ${list[0].question}`);
 const wrong = list[0].options.findIndex((_, i) => i !== list[0].correctIndex);
 await click(ask(list[0].options[wrong].slice(0, 20)), "неверно");
 await settle(80);
 if (!/Неверно/.test(shown())) throw new Error("на неверно не ответил");
 if (!shown().includes(list[0].explanation.slice(0, 24)))
 throw new Error("после ответа нет разбора – пользуетесь подсказкой");
 const nextBtn = ask("Следующий вопрос");
 if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
 await click(nextBtn, "дальше");
 await settle(80);
 if (!shown().includes(list[1].question.slice(0, 24)))
 // верный ответ
 await click(ask(list[1].options[list[1].correctIndex].slice(0, 20)), "верно");
 await settle(80);
 if (!/Верно/.test(shown())) throw new Error("на верно не ответил");
 if (!/вопрос 2\\//.test(shown())) throw new Error(`с вопросом 2 не работает`);
 return `блиц из ${list.length} вопросов отвечает, правильно`;
 });
}

await test("I. факты", "память: разреженная жирная, по памяти");
const memoryRow = async (topic: string) => {
 await mountApp(topic);
 const rows = Array.from(document.querySelectorAll("table tbody tr"));
 const row = rows.find((r) => /^\s*Память/.test(r.textContent));
 if (!row) throw new Error(`на странице «${topic}» нет таблицы с памятью`);
 return Array.from(row.querySelectorAll("td")).map((td) => td.textContent);
};

```

```

 if (!/время: построение $O(N \cdot M)$, запрос суммы $O(1)$ /)
 throw new Error(`на «${topic}» у карточки труба
 }
 if (!/память: $N \times M$ — как оригинал, тонкая/.test(topic))
 throw new Error(`на «${topic}» у карточки труба
 }
 }
 return "билет 2: $N \times M \times \log N \times \log M$ против $N \times M$; билет 5:
});

```

/\* — 10. Сквозной путь: свой код чужой размерности,

```

await test("К. свой код", "Флойд 4x4 из редактора вед
 await mountApp("floyd");
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(
 ta,
 [
 "n = 4",
 "INF = 999",
 "d = [[0, 3, INF, 7], [8, 0, 2, INF], [5, INF,
 "k = 0",
 "i = 0",
 "j = 0",
 "for k in range(n):",
 " for i in range(n):",
 " for j in range(n):",
 " if d[i][k] + d[k][j] < d[i][j]:",
 " d[i][j] = d[i][k] + d[k][j]",
 "print(d[0], d[1][0])",
].join("\n")
);
 const ready = await waitFor(() => bodyText().includes(
 if (!ready) throw new Error(`своя программа не выпол
 if (!bodyText().includes("[0, 3, 5, 6]")) {
 throw new Error(`Флойд на своей матрице 4x4 посчи

```



```

 if (!linked) throw new Error("демонстрация не приня
 const defects = domDefects(document.querySelector("
 if (defects.length) throw new Error(`артефакты в де
 return "свой массив из 6 чисел: st[0][2] = 1, st[1]
 });

```

```

await test("К. свой код", "Косарайю: эталон сам счита
 await mountApp("scc-kosaraju");
 await click(findButton("Боковая панель Python"), "п
 if (!textarea()) throw new Error("нет редактора");
 await click(findButton("Вставить эталонный код"), "
 const ready = await waitFor(() => bodyText().includ
 if (!ready) throw new Error(`эталон Косарайю не вып
 if (/Ошибка:/.test(panelText())) throw new Error(`э
 const out = panelText();
 if (!/порядок выхода:/.test(out)) throw new Error(`
 // граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SC
 if (!/компонент: 2/.test(out)) throw new Error(`в г
 if (!textarea()) throw new Error("редактор исчез, п
 await click(findButton("В конец трассы"), "в конец
 const linked = await waitFor(() => /данные из компи
 if (!linked) throw new Error("демонстрация не приня
 const defects = domDefects(document.querySelector("
 if (defects.length) throw new Error(`артефакты в де
 return "эталон Косарайю: order считается DFS-ом, на
});

```

```

await test("G. lite-режим", "?lite=1 отдаёт текст без
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 window.history.replaceState({}, "", "/?lite=1&topic
 root = createRoot(host);

```

```
}

/** Сводка для оркестратора. */
export function summary() {
 const byGroup = new Map<string, { ok: number; fail: number }>();
 for (const r of results) {
 const g = byGroup.get(r.group) ?? { ok: 0, fail: 0 };
 g[r.ok ? "ok" : "fail"]++;
 byGroup.set(r.group, g);
 }
 return { total: results.length, passed: results.filter(r => r.ok).length };
}
```

```
/** Инициализация страницы из vizSync – нужна для проверки
export const initTemplateOf = (id: string) => buildInitTemplateOf(id);
```

---

ФАЙЛ 100/111: scripts/verify/check-links.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 1000

---

```
/**
 * Аудит внутренних ссылок пособия: ничего не должно вестись за пределы
 *
 * npm run verify:links
 *
 * Проверяет (по настоящим модулям приложения, собранным в production):
 * 1. билеты 1–24 закрыты страницами, id в chapterTopics соответствует id в glossary;
 * 2. разделы содержания: каждая страница в своём разделе;
 * 3. упоминания «билет N» в тексте ведут на существующий билет;
 * 4. все data-goto / href="?topic=..." указывают на существующий билет;
 * 5. якоря href="#..." есть в той же главе;
 * 6. glossary.simulator существует в реестре и (кроме simulator) имеет свою страницу – иначе модалка не получит данные;
 * 7. glossary.demo – работающая мини-демо;
 * 8. у каждого ключа PAGE SYNC есть запись в реестре.
```

```

 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
});
const { chapters, chapterTopics, SECTIONS, sectionOf, G
 await import(`${process.cwd()}/${OUT}?run=${Date.now(

const chapterIds = new Set(chapters.map((c) => c.id));
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подсказки */
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs",

/* 1. билеты 1–24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopics)) {
 ok(chapterIds.has(id), `chapterTopics ссылается на не
 tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 24; n += 1) {
 ok(covered.has(n), `билет ${n} не закрыт ни одной стр
}

/* 2. разделы содержания */
for (const chapter of chapters) {
 const sec = sectionOf(chapter.id);
 ok(!sec, `страница «${chapter.id}» не попала ни в од
 const tickets = chapterTopics[chapter.id] ?? [];
 ok(
 tickets.every((n) => n >= sec.from && n <= sec.to),
 `билеты страницы «${chapter.id}» (${tickets.join(",
));
}
{
 const bySection = SECTIONS.map((s) => ({
 s,
 nums: chapters.flatMap((c) => (sectionOf(c.id).id =
 }));
}

```

```

 ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
 }
 for (const m of html.matchAll(/href="\?(?:lite=1&)?top
 explicitLinks += 1;
 ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
 }

 // 5. якоря
 for (const m of html.matchAll(/href="#([^\"]+)/g)) {
 ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
 }

 if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
 if (term.simulator) {
 ok(registryIds.includes(term.simulator), `глоссарий
 ok(
 chapterIds.has(term.simulator) || EXTRA_DEMOS.has
 `глоссарий «${term.id}»: simulator "${term.simula
 ые компилятора`
);
 }
 if (term.demo) ok(demoKinds.has(term.demo), `глоссарий
}

/* 8–9. реестр и синхронизация */
for (const key of Object.keys(PAGE_SYNC)) {
 const base = key.split("#")[0];
 ok(registryIds.includes(base), `PAGE_SYNC["${key}"]:
}
for (const id of registryIds) {
 if (!chapterIds.has(id) && !EXTRA_DEMOS.has(id)) {
 FAILS.push(`реестр: визуализация "${id}" не являетс
 }
}

```

```
/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code
 *
 * Каждый референсный Python-код из PAGE_SYNC выполняет
 * (pyodide из node_modules) под тем же трейсером, что и
 * проверяется:
 * 1. код не падает и не обрывается по лимиту шагов;
 * 2. трасса не пустая (иначе панель показала бы пустую);
 * 3. каждая переменная, обещанная странице, реально
 * иначе визуализация не может за ним следовать;
 * 4. в снимках нет «мусорных» значений, которые уронят
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

writeFileSync(
 `${OUT_DIR}/contract-entry.ts`,
 `import { buildDebugRunner } from "../../src/data/debug";
import { PAGE_SYNC } from "../../src/data/vizSync";
import { chapters } from "../../src/data/content";
export const runnerOf = (src: string) => buildDebugRunner(
export const sync = Object.fromEntries(Object.entries(PAGE_SYNC).map(
export const declared = Object.fromEntries(Object.entries(DECLARED).map(
export const chapterIds = chapters.map((c) => c.id);
`
);

const built = await esbuild.build({
 entryPoints: [`${OUT_DIR}/contract-entry.ts`],
});
```

```

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result.error}`);
if (result.truncated) issues.push("трасса оборвана по длине");
if (result.steps.length === 0) issues.push("нет ни одной итерации");
const missing = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .filter((a) => !a.some((x) => ident(x) && seen.has(x)))
 .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обещаний`);
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявлениях и не наполнил – визуализации нечего показывать
 * «защитный» результат вместо работы алгоритма. Именно
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
 Array.isArray(v) ? v.length === 0
 : typeof v === "string" ? v.length === 0
 : typeof v === "object" && v !== null ? Object.keys(v).length === 0
 : false;
const neverFilled = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .map((a) => a.find((x) => ident(x) && seen.has(x)))
 .filter(Boolean)
 .filter((name) => {
 let sawValue = false;
 let sawNonEmpty = false;
 for (const step of [...result.steps, { values: result.final }]) {
 const v = step.values?.[name];
 if (v === undefined || v === null) continue;
 sawValue = true;
 if (!isEmpty(v)) sawNonEmpty = true;
 }
 return sawValue && !sawNonEmpty;
 });

```

```

 takeRecords() {
 return [];
 }
 };

export function createDom({ url = "http://localhost/?to"
 const virtualConsole = new VirtualConsole();
 const jsdomErrors = [];
 virtualConsole.on("jsdomError", (error) => {
 jsdomErrors.push(error?.message || String(error));
 onError?.(error);
 });
 // console.error/warn приложения собираем отдельно —
 const consoleErrors = [];
 virtualConsole.on("error", (...args) => consoleErrors
 virtualConsole.on("warn", () => {});

 const dom = new JSDOM(
 `<!doctype html><html lang="ru"><head><meta charset
 <body><div id="root"></div></body></html>`,
 { url, pretendToBeVisual: true, runScripts: "outsid
);
 const { window } = dom;

 window.ResizeObserver = window.ResizeObserver ?? OBSE
 window.IntersectionObserver = window.IntersectionObse
 window.matchMedia =
 window.matchMedia ??
 ((media) => ({
 matches: false,
 media,
 onchange: null,
 addListener() {},
 removeListener() {},
 addEventListener() {},
 removeEventListener() {},
 dispatchEvent: () => false,
 }));

```

```
 "HTMLCollection",
 "Range",
 "MutationObserver",
 "localStorage",
 "sessionStorage",
 "Blob",
 "File",
 "FileReader",
];
 for (const key of copy) {
 try {
 Object.defineProperty(globalThis, key, { value: w
 } catch {
 /* read-only в Node – не критично */
 }
 }
 globalThis.IS_REACT_ACT_ENVIRONMENT = true;

 return { dom, window, document: window.document, cons
}
```

---

ФАЙЛ 103/111: scripts/verify/live-matrix.test.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 103 | РАЗМ

---

```
/**
 * Модульные проверки разбора «живых» данных компилятор
 *
 * npm run verify:unit
 *
 * liveMatrix() – то, благодаря чему визуализации матриц
 * суммы 2D, разреженная таблица) принимают данные ЛЮБОГО
 * пользователя вместо захардкоженной демо-сетки.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
```



---

```
/* — liveMatrix: формы данных
check("квадратная 3x3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
check("не квадратная 2x4", shape([[1, 2, 3, 4], [5, 6, 7, 8]]))
check("значения ячеек 2x2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]])
check("нечётные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
check("нечётные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9]]))
check("at() вне диапазона → null", m([[1, 2], [3, 4]]).at(2, 2))
check("at() с отрицательным индексом → null", m([[1, 2], [3, 4]]).at(-1, -1))
check("список словарей с числовыми ключами", cells([{"0": 1, "1": 2}, {"0": 3, "1": 4}]))
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }))
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }))
check('ключи-пары "(i, j)" после снимка tuple', cells({ "0, 1": 5 }))
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5 })))
```

---

```
/* — liveMatrix: что матрицей НЕ является
check("плоский список чисел → null", m([1, 2, 3, 4, 5]))
check("словарь по именам → null", m({ dist: 1, prev: 2 }))
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c", "d"])))
```

---

```
/* — liveMatrix: особые значения
check('"inf" → +Infinity', cells([["inf", 1], [2, "-inf"]]))
check('числа-строки "42" → 42', cells([["1", "2"], ["3", "4"]]))
check("булевы значения ячейкой не считаются", m([[true, false], [false, true]]))
check("обрывок до 64x64", shape(Array.from({ length: 8096 }, () => 4), { numbers: true }));
check("hasNumbers=false невозможна для живой матрицы", m({}))
```

---

```
/* — pickLiveVar
check("первое найденное имя по приоритету", pickLiveVar({ x: 1, y: 2, z: 3 }, { x: 1, y: 2, z: 3 }, { x: 1, y: 2, z: 3 }))
check("берёт следующее имя, если первого нет", pickLiveVar({ x: 1, y: 2, z: 3 }, { x: 1, y: 2, z: 3 }, { x: 1, y: 2, z: 3 }))
check("ничего не найдено → null", pickLiveVar({ x: 1 }, { x: 1, y: 2, z: 3 }, { x: 1, y: 2, z: 3 }))
check("нет снимка → null", pickLiveVar(undefined, ["d"], { x: 1, y: 2, z: 3 })))
```

```

 return document.querySelector<HTMLElement>('aside[aria-label="editor"]');
}
function editor(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("text-area");
}
function footer(): string {
 const divs = Array.from(panel()?.querySelectorAll("div"));
 const f = divs.find((d) => d.className.includes("text-area"));
 return (f?.textContent ?? "").replace(/\s+/g, " ");
}
function state(tag: string) {
 const p = panel();
 const ed = editor();
 const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
 const step = txt.match(/шаг[^\s]*\s*\d+\s*\//i);
 process.stdout.write(
 ` ${tag.padEnd(9)} редактор: ${ed ? "ДА" : "нет"} шаг: ${step ? step[0] : "-"} | футер: ${footer()}\n`
);
 process.stdout.write(` ${tag.padEnd(9)} бейдж страницы: ${step ? step[0] : "-"}\n`);
}
function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window.HTMLTextAreaElement.prototype, "value").set;
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubble: true }));
 });
}
async function click(el: Element | null | undefined, label: string) {
 if (!el) throw new Error(`нет кнопки: ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubble: true }));
 await sleep(20);
 });
}
function findButton(match: RegExp): HTMLButtonElement | null {
 return Array.from(document.querySelectorAll("button")).find((b) => match.test(b.textContent));
}

```

```

 state(`t+${t}`);
 }

 process.stdout.write("\n=== 3. вводим рабочий код x=4\n");
 const ed3 = editor();
 if (!ed3) throw new Error("редактор пропал после комм");
 typeInto(ed3, "x = 41\nx += 1\nprint(x)\n");
 state("ввели");
 for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
 }

 process.stdout.write("\n=== 4. очищаем редактор полностью\n");
 const ed4 = editor();
 if (ed4) {
 typeInto(ed4, "");
 state("очистили");
 for (const t of [200, 800, 2000]) {
 await settle(t);
 state(`t+${t}`);
 }
 } else {
 process.stdout.write(" редактора нет (листинг) — ждем\n");
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("Esc"));
 await sleep(30);
 });
 state("после Esc");
 }
}

```

Настоящее приложение монтируется в jsdom (`App` целиком глава, визуализация, панель Python), сценарии кликают к редакторе.

Группа	Что делает
A. страницы (22 × 3)	каждая глава монтируется; бейдж автозапуск инициализации настоящим Python не роняет <code>[object Object]</code> ; нет кнопок без имени
B. Флойд: размерности (10)	5×5, 10×10 (больше демо - й список, 80×80 (обрезка), пустая матрица, чужое имя
C. матричные страницы (6)	префиксные суммы 2D и раз
D. графы путей (2)	Дейкстра/Форд-Беллман с вершинами грывания гаснут и оживают обратно
E. эталонный код (46)	прогон кода в CPython → финал е показать мусор
F. редактор (6)	код из одних комментариев, синтаксис ляет эталон и возвращает инициализацию; код не перете
G. lite-режим (1)	<code>?lite=1</code> отдаёт текст без навига
H. доступность (2)	в панели подписаны все кнопки в очника

Фильтры: ``HAR_CHAPTERS=floyd,dijkstra``, ``HAR_SKIP=A,E``,

## Как это заведено в приложение

- \* ``window.__algv0PythonRuntime`` – точка подмены интерпр ``PythonCompiler.tsx``. В браузере пользователя поле не грузит Pyodide с CDN как раньше; харнес кладёт туда л
- \* Состояние визуализаций передаётся через ``emitVizState`` (``src/data/vizStepBus.ts``) – те же события, что шлёт идёт по боевому пути, а не через внутренние API компо

```

 bundle: true,
 outfile: `${OUT_DIR}/bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 // Vite-специфичные глобалы, которых нет вне dev-серва
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 logLevel: "warning",
 });
const mod = await import(`${process.cwd()}/${OUT_DIR}/bundle.mjs`);

process.on("unhandledRejection", (reason) => {
 jsdomErrors.push(`unhandledRejection: ${reason instanceof Error ? reason.message : reason}`);
});

log("прогоняю сценарии...\n");
const results = await mod.runSuite({ py, window, document });
const sum = mod.summary();

writeFileSync(`${OUT_DIR}/report.json`, JSON.stringify(results));

const failed = results.filter((r) => !r.ok);
console.log("\n" + "=".repeat(96));
console.log("СВОДКА");
console.log("=".repeat(96));
for (const [group, g] of Object.entries(sum.byGroup)) {
 console.log(` ${group.padEnd(26)} ok: ${String(g.ok)}

```

```
const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
const log = (m) => process.stdout.write(`[verify:probe]

log("гружаю CPython...");
const py = await loadPyodide({ indexURL: "node_modules/"
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/p
log("зондирую панель...");
await mod.probe();
process.exit(0);
```

steps:

- name: Checkout  
uses: actions/checkout@v6
- name: Setup Node.js  
uses: actions/setup-node@v7  
with:  
node-version: 24  
cache: npm
- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Build for the repository subpath  
env:  
VITE\_BASE: /algv0/  
run: npm run build
- name: Make GitHub Pages fallback  
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact  
uses: actions/upload-pages-artifact@v5  
with:  
path: ./dist

deploy:

name: Publish to GitHub Pages  
needs: build  
runs-on: ubuntu-latest  
environment:  
name: github-pages  
url: \${ steps.deployment.outputs.page\_url }

steps:

- # Ранняя и понятная ошибка вместо невнятного «Fail»
- # когда в репозитории Pages не переключён на сбор
- name: Preflight – Pages собирается через GitHub
- env:

```
чтобы коммит самого workflow не запускал бесконечный цикл
- "public/export/**"
- "**/*.md"
workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолчанию 150)"
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}`
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot]')"

 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
```



- name: Step 5 – снимки визуализаций (headless-brun)run: npm run export:shotscontinue-on-error: true
- name: Step 6 – PDF-конспект со снимкамиrun: |  
 npm run export:guide  
 # снимки публикуются вместе с приложением: HTML  
 # в автономный файл прямо из /export/viz-shots  
 mkdir -p public/export/viz-shots  
 cp tmp/viz-shots/\*.png tmp/viz-shots/manifest
- name: Type-check приложенияrun: npx tsc --noEmitcontinue-on-error: true
- name: Summaryrun: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages` | "  
 echo "| \`public/export/full\_code\_all\_pages` | "  
 echo "| \`public/export/guide\_lite.pdf\` | "  
 echo "| Снимков визуализаций | \$(ls tmp/viz-shots | wc -l) | "  
 echo "| PNG-страниц | \$(ls tmp/export-pages | wc -l) | "  
 } >> "\$GITHUB\_STEP\_SUMMARY"
- name: Upload artifacts (PDF + TXT)uses: actions/upload-artifact@v7with:  
 name: full-code-export  
 path: |  
 public/export/full\_code\_all\_pages.pdf  
 public/export/full\_code\_all\_pages.txt

## Универсальное пособие • полный исходный код

```

матрицы чужих размерностей, кнопки,
#
Headless-браузер не нужен: jsdom + pyodide из node_modules
on:
 push:
 branches:
 - "**"
 paths-ignore:
 - "public/export/**"
 - "**/*.md"
 pull_request:
 workflow_dispatch:

concurrency:
 group: verify-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: read

jobs:
 verify:
 name: unit + эталонный код + DOM
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v4

 - uses: actions/setup-node@v4
 with:
 node-version: 22
 cache: npm

 # puppeteer здесь не нужен (он для растеризации PDF)
 # тянет Chromium – пропускаем, чтобы установка не
 - name: Зависимости
 run: npm ci
 env:
```

Универсальное пособие • полный исходный код

-----  
репозитории. На него ссылаются `AGENTS.md` и `GEMINI.md`  
собираем **\*\*только\*\*** из блоков ниже — не придумываем нов  
«голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов  
`src/data/content.ts` (при одинаковом `id` побеждает бо

---

## 1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
  {
    id: "dijkstra", // = ключ VIZ_REGISTRY и
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html", // всегда "html": страни
    category: "Графы. Пути", // группа в навигации
    content: `...HTML ниже...`,
  },
];
```
```

Правила:

- `id` — kebab-case, уникальный, совпадает с ключом `VIZ\_REGISTRY`  
подставляется автоматически, вручную `div#slot-\*` вст  
такие заглушки остались в старых главах и считаются м
- `title` начинается с номера билета: сортировка оглавл  
цифре заголовка. Одна страница может закрывать нескол  
(`mst` → 18, 19, 20) — тогда номер в заголовке один,  
объявляется в `chapterTopics` (`src/data/content.ts`)
- `content` — шаблонная строка. Внутри **\*\*никаких\*\*** ``  
экранирования: это сломает сборку.

---

Универсальное пособие • полный исходный код

```

<div class="bg-slate-900 p-4 rounded-lg mb-6 text-center">
 <p class="text-lg text-emerald-300 italic mb-2">Строчка
 <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

3.2 Обычный абзац

```
```html
<p class="text-slate-300 text-sm mb-4">
 Текст с акцентом и инлайн-кодом
 dist[v]
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html

 ...
```
```

3.3 Сетка ключевых чисел (3 ячейки)

```
```html
<div class="grid grid-cols-1 sm:grid-cols-3 gap-3 my-6">
 <div class="bg-slate-900 border border-slate-700 rounded">
 <p class="text-[10px] uppercase tracking-wider text-white">0
 <p class="font-mono text-emerald-300 text-sm">0((V+H
 </div>
 <!-- 2-я и 3-я ячейки – тот же markup -->
</div>
```
```

3.4 Сетка аналогий – ****обязательна**** на каждой странице

Слева – неформальная аналогия, справа – техническая альфа

```
```html
```

```

 <td class="py-2 pr-3 font-mono">O(log n)</td>
 </tr>
 </tbody>
 </table>
</div>
```
```

3.6 Спойлер ``<details>`` – **обязателен** для кода,

```
```html
<details class="bg-slate-900/40 rounded-lg border border-
 <summary class="p-4 font-bold text-slate-400 outline-ne
 e-700/50">
 ▶ Код: Дейкстра на куче (Python)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <pre class="bg-slate-950 p-4 rounded overflow-x-auto
...</code></pre>
 <p class="text-slate-300 text-sm">Комментарий после
 </div>
</details>
```
```

- Вложенные списки внутри спойлера: ``<ul class="list-di`
- Подзаголовок внутри блока: ``<h4 class="text-lg font-b`
- или ``<p class="text-lg font-bold text-white mt-8 mb-3`

3.7 Ловушки / частые ошибки

```
```html
<div class="bg-rose-950/30 p-5 rounded-lg border border
 <p class="text-sm font-bold text-rose-300 mb-3">⚠ Гд
 <ul class="list-disc pl-5 space-y-2 text-sm text-slat
 Отрицательные веса. Дейкстра их не перев
 Переполнение: <span class="font-mono text-emera

</div>
```
```

6. Экспорт в PDF (`npm run export:guide`)

Текст страниц парсится `scripts/export/html-to-blocks.m`
`public/export/guide\_lite.pdf` (текст + визуализации, без
компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только `h2/h3/h4`; списки – `ul/ol/li`; теги
код – `pre`; определения – обычный `p` (см. 3.1);
- ****смысл не должен держаться на эмодзи****: в PDF использо-
нет эмодзи-глифов – пиктограммы вырезаются, остаётся
« Операции на отрезках» доедет как «Операции на отрез-
а вот « значит да / значит нет» потеряет смысл, пикто-
граммы не выйдут;
- спойлеры в PDF раскрываются полностью – не прячьте ту-
минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из `CHAPTER\_IMAGES` (`scripts`
снимки демонстраций – из `tmp/viz-shots/\*.png` (их пере-
`npm run export:shots` в CI); локально без браузера в
описание вкладок и переменных из `PAGE\_SYNC`.

Цель по объёму: 40–60 страниц, жёсткий предел – 100.